📖

# 취준생이 대규모 트래픽 처리 경험을 쌓는 방법 ?!

## ✅ 최근 백엔드 개발자 채용 공고에 자주 등장하는 '대용량 트래픽 처리 경험'

백엔드 개발자 채용 공고를 살펴보면 '**대용량 트래픽 처리 경험**'에 관한 얘기가 자주 나온다. 이 역량이 서비스가 점점 고도화 될 수록 빼놓을 수 없는 역량이기 때문이다.

**자격요건**

- 컴퓨터 공학 또는 관련 분야 학사 이상
- TypeScript, Python 중 하나 이상의 언어에 대한 깊은 이해
- SQL 및 NoSQL 데이터베이스 경험
- 클라우드 서비스 (AWS, GCP, Azure) 사용 경험

**우대사항**

- 핀테크 또는 금융 서비스 관련 개발 경험
- 마이크로서비스 아키텍처 운영 혹은 대규모 트래픽 처리 경험 혹은 오픈소스 프로젝트 기여 경험
- 문제 해결 능력과 원활한 커뮤니케이션 능력
- 진행중인 캠페인 혹은 공모전에 대한 이해를 바탕으로 자본 시장에 대한 깊은 이해 또는 관심
- 투자한 주식이 대주주의 부당한 행위로 손실을 경험한 이력

**자격요건**

- 경력 3년 이상 선호
- Spring framework + Spring Boot
- Spring + Mybatis or Spring + JPA
- AWS(Amazon Web Services) 운영/개발 경험을 보유하신분 (필수)

**우대사항**

- MSA(Microservices Architecture) 개발 경험 보유
- 높은 AWS 활용 역량을 갖추신 분
- 대용량 트래픽 서비스 설계/개발/운영 경험 보유
- 핀테크 / 금융 관련 서비스 설계/개발/운영 경험 보유
- 웹 프론트엔드 개발 경험 보유
- Git-flow를 사용한 소스관리 경험 보유

**주요업무**

- A/B 테스트 플랫폼 백엔드 서버 개발
- 데이터분석 플랫폼 백엔드 개발
- 인앱/푸시 메시지 서비스 백엔드 개발
- 신규 서비스 설계 및 개발
- 대용량 트래픽을 고려한 서버 설계
- SDK 개발

**자격요건**

- Java/Kotlin 기반 서버 개발 경험
- Spring Framework 사용 경험
- RDBMS, ORM 사용 경험

## 그런데 신입 개발자가 어떻게 '대용량 트래픽 처리 경험'을 쌓죠?

네카라쿠배에 취업을 하더라도 대부분의 부서에서는 대용량 트래픽이 발생하지 않는 경우가 많다. 그래서 실제 대용량 트래픽을 경험하기란 쉽지 않다. 그럼 채용 공고에 나와있는 '대용량 트래픽 처리 경험'은 어떻게 쌓을 수 있는 걸까?

바로 부하 테스트 툴을 활용해서 대용량 트래픽을 경험하면 된다. 부하 테스트 툴은 다양한 형태로 대용량 트래픽을 만들어낼 수 있다. 실제 현업에서도 부하 테스트 툴을 활용해 대용량 트래픽을 직접 만들어내면서 부하를 얼마나 견뎌낼 수 있는 지 테스트한다. **부하 테스트**하는 방법을 배워두면 대용량 트래픽을 어떻게 처리할 지에 대한 경험을 간접적으로 쌓을 수 있다.

따라서 대용량 트래픽 처리 경험을 쌓기 위한 가장 기본인 **부하 테스트**에 대해 본격적으로 배워볼 것이다.

# 현업에서 부하 테스트를 왜 할까?

## ✅ 부하 테스트란?

시스템이 어느 정도의 부하(= 트래픽, 요청)를 견딜 수 있는 지 테스트하는 것

## ✅ 현업에서 부하 테스트를 왜 할까?

백엔드 서버를 구현하고 나서 배포를 한다. 그런데 프로덕션 환경에 서비스를 배포 하기 전에 문득 이런 생각이 들 수도 있다.

***"혹시 요청이 몰려서 서버가 터지면 어떡하지?"***

***"내 서버는 어느 정도 사용자 요청을 견딜 수 있는 거지?"***

이런 불안감을 없애기 위해서 서비스를 배포하기 전에 백엔드 서버가 어느 정도의 요청을 견딜 수 있는 지 **부하 테스트**를 해봐야 한다. 그래야 어느 정도의 트래픽을 감당할 수 있는 지 미리 파악할 수 있다. 이걸 파악할 수 있다는 건 트래픽이 늘어나서 서버가 터질 때쯤 빠르게 대처할 수 있다는 게 장점이다. 즉, 서버가 터져서 죽는 걸 미리 감지하고 대처할 수 있다는 뜻이다.

## ✅ 그냥 걱정 없이 가장 좋은 사양의 서버를 쓰면 되지 않나요?

실제 현업에서도 부하 테스트를 하지 않고 그냥 무작정 고사양의 서버를 배치해서 사용하는 경우가 있다. 당연히 오버 스펙으로 고사양의 컴퓨터를 서버로 사용하면 수많은 트래픽을 처리할 수 있을 것이다. 하지만 여기서 문제가 되는 건 비용이다. 비용을 절약하기 위해 부하 테스트를 통해 딱 필요한 만큼의 컴퓨터 사양을 선택하는 게 중요하다.

kb302507

📖

# 처리량(Throughput), 지연 시간(Latency)

## ✅ 처리량(Throughput)이란?

부하 테스트에서 **서비스가 1초당 처리할 수 있는 트래픽 양**을 보고 **Throughput**이라고 부른다. 단위는 **TPS(Transaction Per Seconds, 1초당 처리한 트랜잭션의 수)**를 많이 활용한다. 만약 내가 만든 서비스가 1초에 최대 100개의 API 요청을 처리할 수 있다면, 이 서비스의 **Throughput**은 **100 TPS**라고 얘기한다.

** **참고)** 현업에서는 '처리량'이라고 잘 얘기하지 않고 'Throughput(쓰루풋)'이라고 많이 얘기하는 편이다.

** **참고)** TPS(Transaction Per Seconds) ≒ RPS(Request Per Second)

→ 이 2가지 단위는 같다고 생각해도 무방하다.

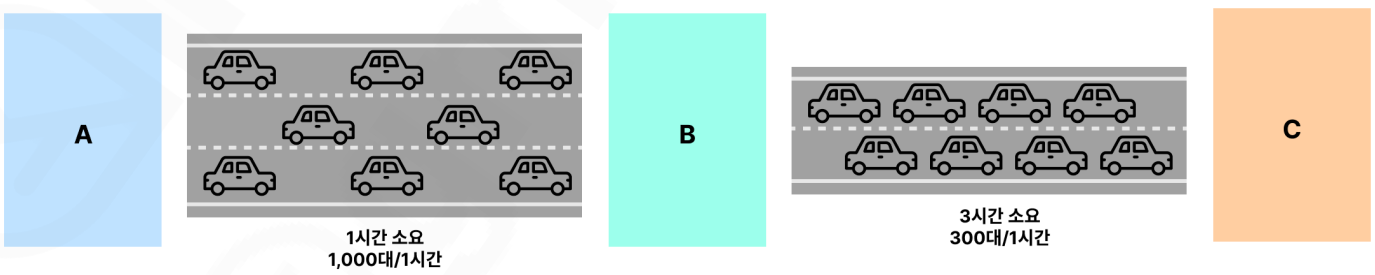
## ✅ 지연 시간(Latency)이란?

부하 테스트에서의 **Latency**는 **요청에 대한 응답 시간**을 의미한다. 만약 내가 만든 서비스에 부하 테스트를 했을 때 평균 응답 시간이 2.5초일 경우, **평균 Latency**가 **2.5초**라고 얘기한다. 조금 더 쉽게 해석하자면 하나의 API에 요청을 보냈을 때 응답받기까지의 시간이 2.5초 정도 걸린다는 뜻이다.

** **참고)** 현업에서는 '지연 시간'이라고 잘 얘기하지 않고 'Latency(레이턴시)'이라고 많이 얘기하는 편이다.

## ✅ 처리량(Throughput), 지연 시간(Latency)

Throughput과 Latency를 조금 더 직관적으로 이해하려면 **고속 도로**를 떠올리면 쉽다.




- 자동차 = **트래픽(요청, 부하)**
- 고속도로 = **네트워크**
- 출발지부터 목적지까지 걸리는 시간 = **Latency**
- 단위 시간당 목적지에 도착하는 자동차의 대수 = **Throughput**

# 부하 테스트 툴 선정 (k6)

## ✅ k6란?

부하테스트 툴에는 다양한 툴이 있다. 그 중 유명한 부하테스트 툴에는 ngrinder, jmeter, ab, locust 등이 있다. 그 중 메모리를 적게 사용하면서 비교적 많은 요청 수를 보낼 수 있는 부하 테스트 툴이 **k6**이다. 또한 사용법이 간단해 쉽게 테스트 해볼 수 있다.

쉽게 테스트할 수 있다고 해서 테스트 결과가 부정확한 건 아니다. k6도 높은 정확도와 많은 트래픽을 발생시킬 수 있는 부하테스트 툴이다. 현업에서도 많이 쓰니까 걱정하지 말고 사용해도 된다.

## ✅ 어떤 부하 테스트 툴이냐는 크게 안 중요하다

취업이나 이직을 할 때 Spring Boot를 쓸 줄 아는 지 Node.js를 쓸 줄 아는 지는 중요하다. 하지만 부하 테스트 툴에서 k6을 쓸 줄 아는 지 ngrinder를 쓸 줄 아는 지는 하나도 안 중요하다. 실제 부하 테스트를 하면서 결과 데이터를 정확하게 해석할 수 있는 지와, 결과 데이터를 바탕으로 적절한 방식으로 성능 개선을 할 수 있는 지의 역량을 가지고 있는 지가 중요하다. 따라서 아무 부하테스트 툴이나 딱 하나만 골라서 익히면 충분하다. 다른 부하 테스트 툴을 써보면 알겠지만 사용법이나 결과 해석하는 방법이 다 거기서 거기다.

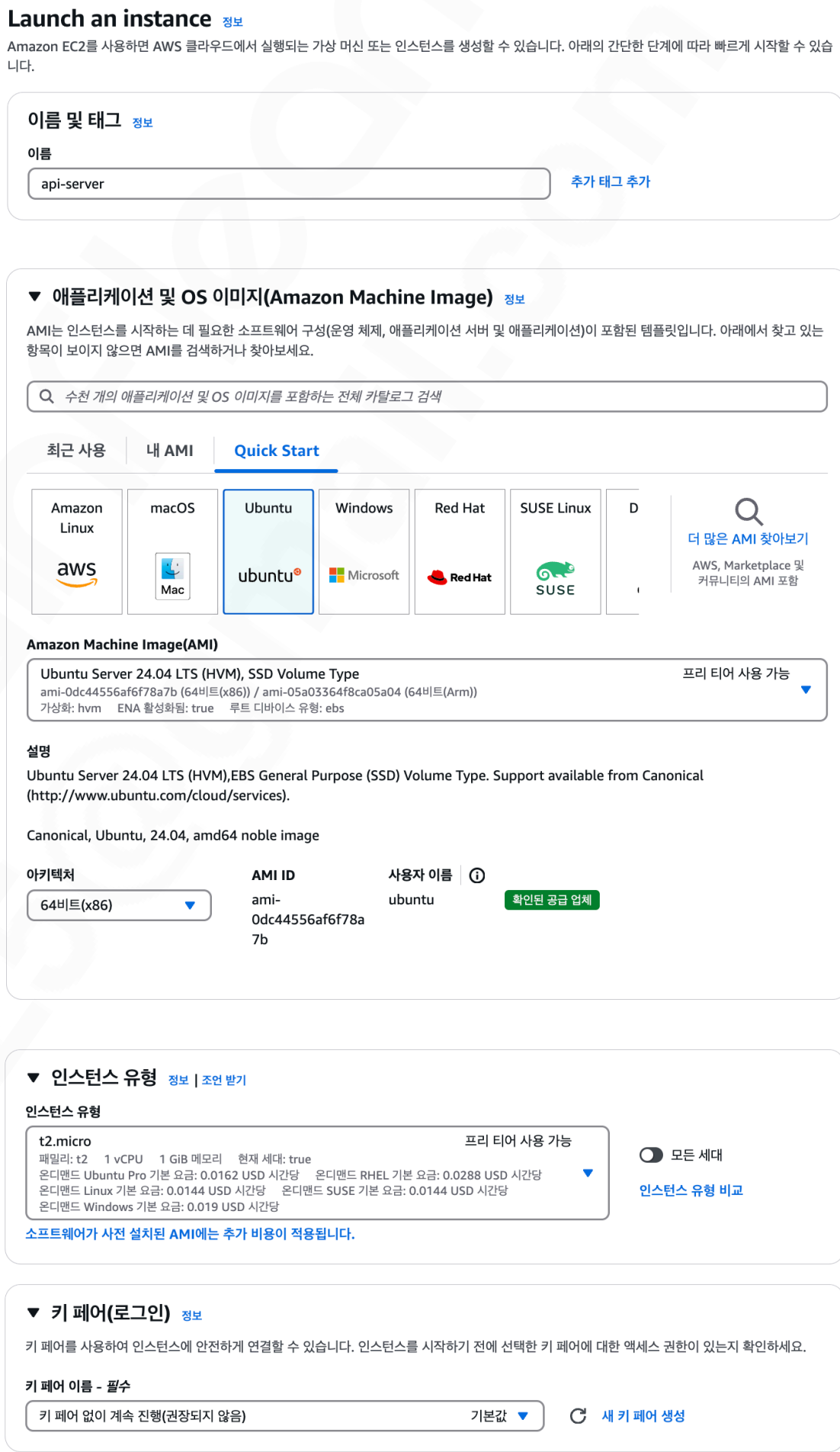
khb3025@gmail.com

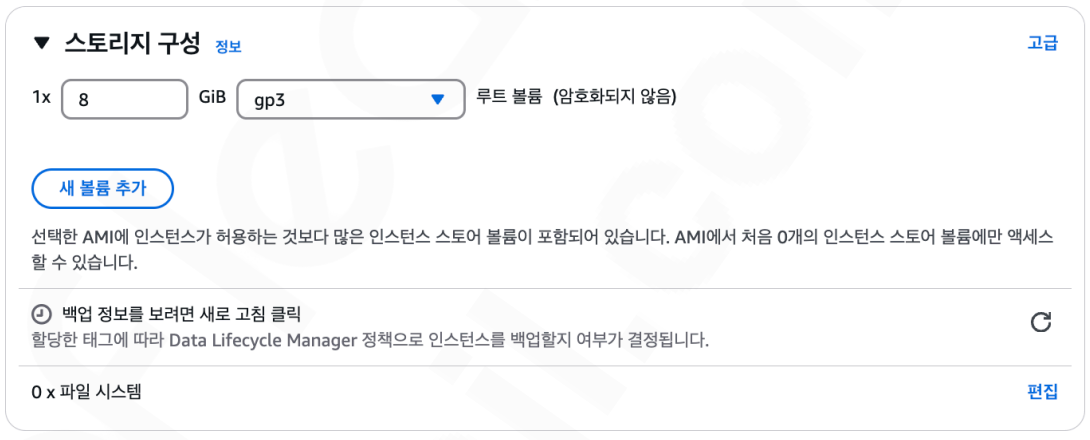
# 🎯 [실습] EC2에 간단한 API 서버 셋팅하기

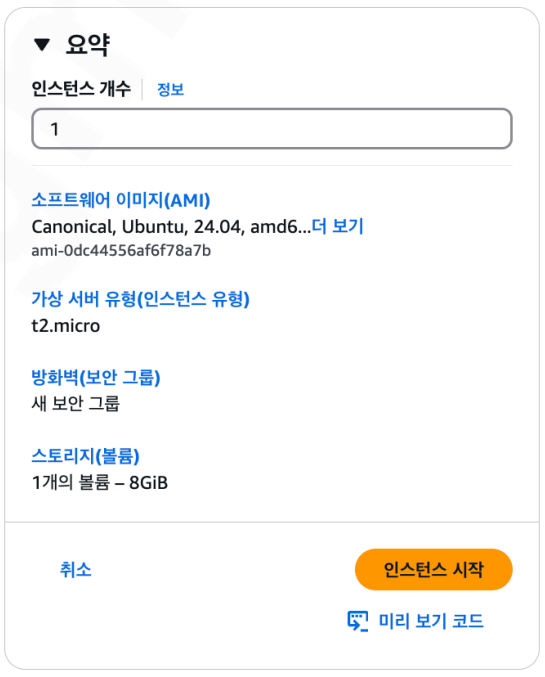
👩🏻 AWS에 초점을 맞춘 강의가 아니기 때문에 EC2, RDS, ELB에 대한 디테일한 설명은 생략할 예정이다. 혹시 AWS에 대한 기본기를 다지고 싶다면 아래 강의를 추천한다.

**비전공자도 이해할 수 있는 AWS 입문/실전 (https://inf.run/73afg)**

## ✅ EC2 인스턴스 생성하기








## ✅간단한 API 서버 셋팅하기

👩🏻‍🏫 **참고)** 이번 실습에서는 Node 기반의 Express 서버를 활용하지만, 이후 실습에서는 Spring Boot를 활용해 테스트를 할 예정이다.

1. **Node 설치**

   Ubuntu 환경에서 Express 서버를 실행시키려면 Node.js가 설치되어 있어야 한다. 그래서 Ubuntu에 우선 Node.js를 설치해보자.

https://github.com/nodesource/distributions

```
$ sudo su
$ apt-get update && /
apt-get install -y ca-certificates curl gnupg && /
mkdir -p /etc/apt/keyrings && /
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key | sudo gpg --dearmor -o /etc/apt/keyrings/
NODE_MAJOR=20 && /
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg] https://deb.nodesource.com/node_$NODE_MAJOR.x nodis
apt-get update && /
apt-get install nodejs -y

# 잘 설치됐는 지 확인하기
$ node -v
```

2. **Express 서버 셋팅하기**

   **기본 프로젝트 셋팅**

```
$ mkdir my-server
$ cd my-server
$ npm init # 의존성 관리 파일 생성
$ npm i express # express 라이브러리 설치
```

   **API 로직 작성**

   **app.js**

```
const express = require('express');
const app = express();
const port = 80;

app.get('/boards', (req, res) => {
  res.send([
    { id: 1, title: '첫 번째 게시글', content: '이것은 첫 번째 게시글의 내용입니다.' },
    { id: 2, title: '두 번째 게시글', content: '이것은 두 번째 게시글의 내용입니다.' },
    { id: 3, title: '세 번째 게시글', content: '이것은 세 번째 게시글의 내용입니다.' }
  ]);
})

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`)
})
```

3. **pm2 설치하기**

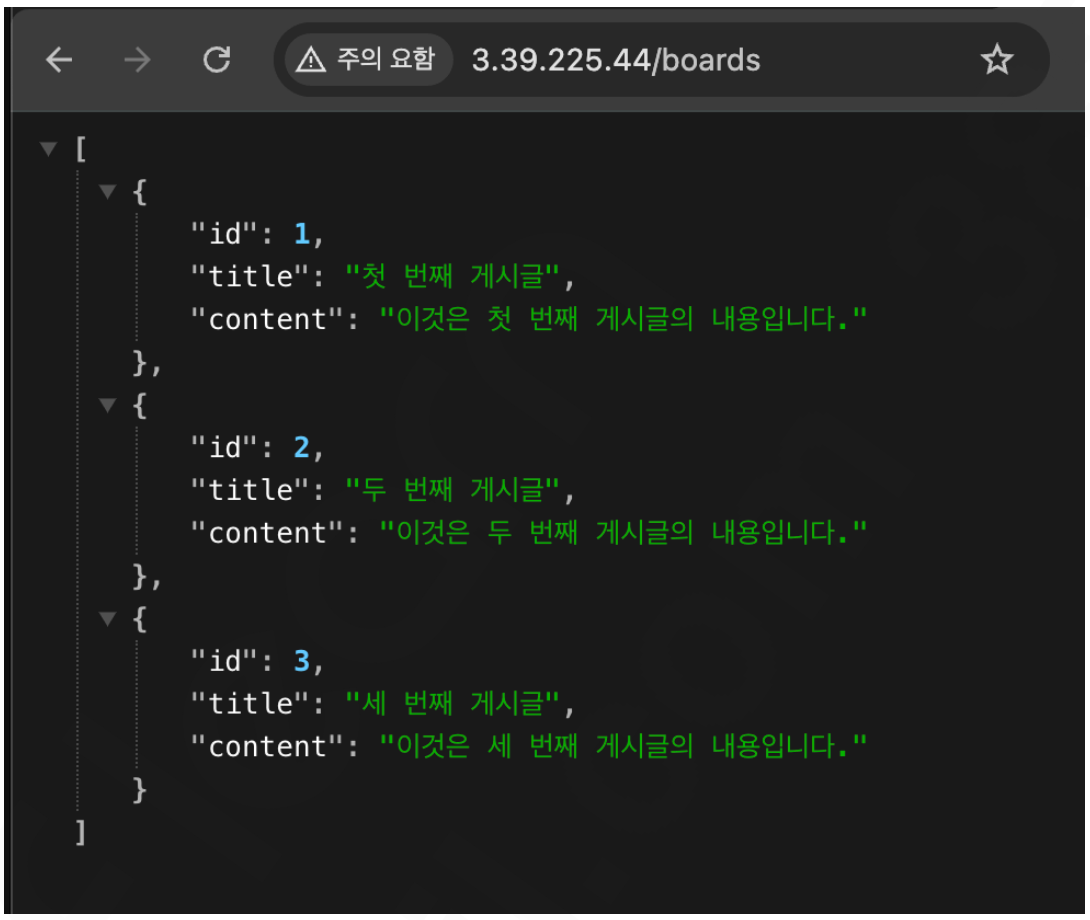
   Node 기반의 서버는 `pm2`를 활용해서 많이 실행시킨다. 서비스를 운영하는 데 있어서 유용한 기능들을 `pm2`가 많이 가지고 있기 때문이다.

```
$ npm i -g pm2
```

4. **API 서버 실행시키기**

```
$ pm2 start app.js
```

5. **잘 잘동하는 지 확인하기**

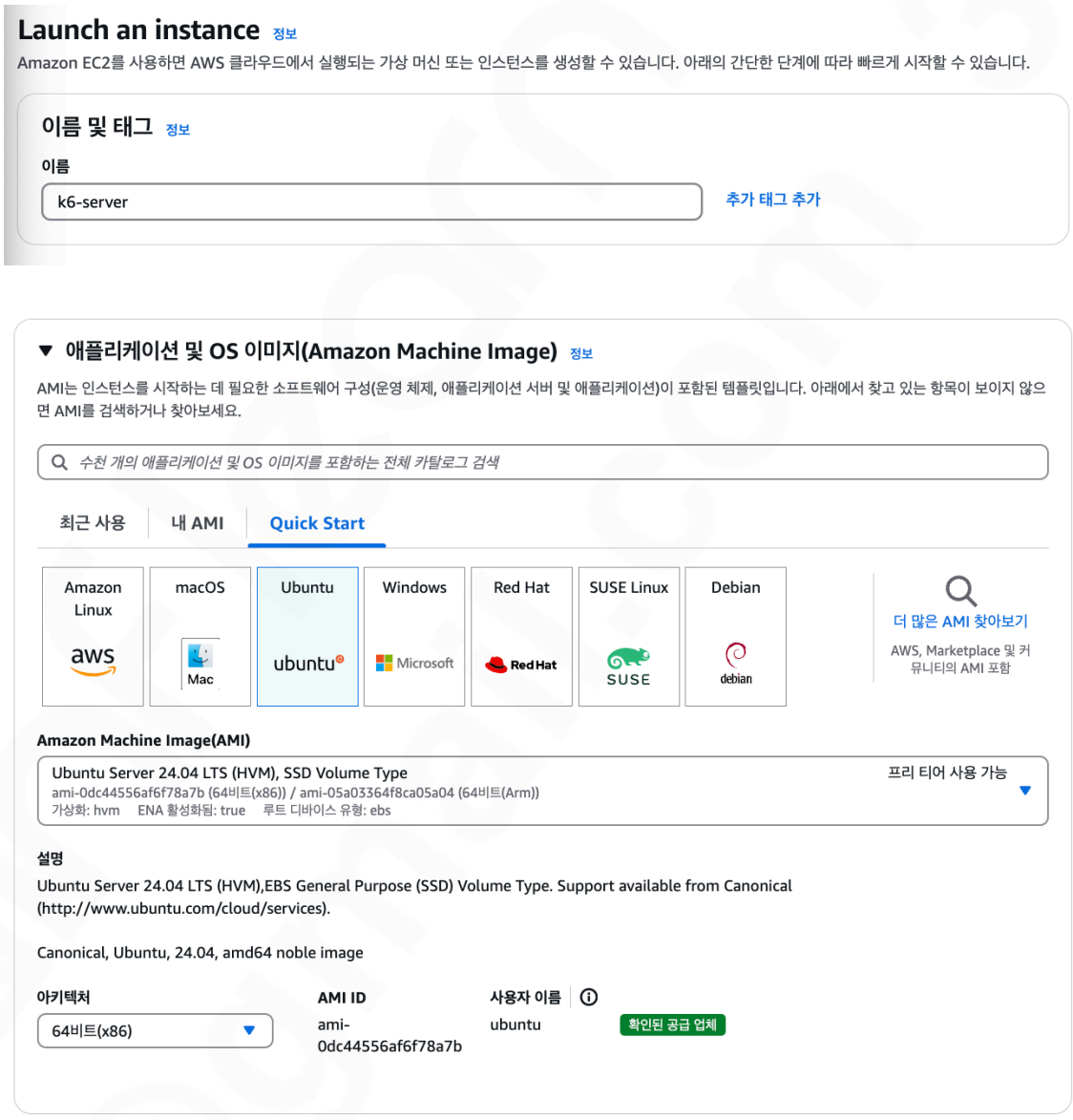



kh3025@gmail.com

# 🎯 [실습] EC2에 부하 테스트 툴 셋팅하기

## ✅ EC2 인스턴스 생성하기

👩🏻‍🏫 충분한 부하를 만들어내기 위해 인스턴스를 **t3a.small**로 생성할 예정이다.



## ▼ 네트워크 설정 정보

**VPC – *필수*** | 정보

vpc-d0897abb (기본값)
172.31.0.0/16

**서브넷** | 정보

기본 설정 없음 — 새 서브넷 생성

**퍼블릭 IP 자동 할당** | 정보

활성화

프리 티어 허용 범위를 벗어나는 경우 추가 요금이 적용됩니다.

**방화벽(보안 그룹)** | 정보

보안 그룹은 인스턴스에 대한 트래픽을 제어하는 방화벽 규칙 세트입니다. 특정 트래픽이 인스턴스에 도달하도록 허용하는 규칙을 추가합니다.

(●) 보안 그룹 생성 ( ) 기존 보안 그룹 선택

보안 그룹 이름 - *필수*

launch-wizard-14

이 보안 그룹은 모든 네트워크 인터페이스에 추가됩니다. 보안 그룹을 만든 후에는 이름을 편집할 수 없습니다. 최대 길이는 255자입니다. 유효한 문자는 a~z, A~Z, 0~9, 공백 및 ._-:/()#,@[]+=&;{}!$*입니다.

**설명 - *필수*** | 정보

launch-wizard-14 created 2024-12-04T10:09:13.506Z

**인바운드 보안 그룹 규칙**

▼ 보안 그룹 규칙 1 (TCP, 22, 0.0.0.0/0) 제거

| 유형 정보 | 프로토콜 정보 | 포트 범위 정보 |
|---|---|---|
| ssh | TCP | 22 |

| 소스 유형 정보 | 원본 정보 | 설명 - *선택 사항* 정보 |
|---|---|---|
| 위치 무관 | CIDR, 접두사 목록 또는 보안 그룹 추 | 예: 관리자 데스크톱용 SSH |
| | 0.0.0.0/0 | |

▼ 보안 그룹 규칙 2 (TCP, 5665, 0.0.0.0/0) 제거

| 유형 정보 | 프로토콜 정보 | 포트 범위 정보 |
|---|---|---|
| 사용자 지정 TCP | TCP | 5665 |

| 소스 유형 정보 | 원본 정보 | 설명 - *선택 사항* 정보 |
|---|---|---|
| 위치 무관 | CIDR, 접두사 목록 또는 보안 그룹 추 | 예: 관리자 데스크톱용 SSH |
| | 0.0.0.0/0 | |

⚠ 소스가 0.0.0.0/0인 규칙은 모든 IP 주소에서 인스턴스에 액세스하도록 허용합니다. 알려진 IP 주소의 액세스만 허용하도록 보안 그룹을 설정하는 것이 좋습니다. ✕

보안 그룹 규칙 추가

## ▼ 스토리지 구성 정보 고급

1x 8 GiB gp3 루트 볼륨 (암호화되지 않음)

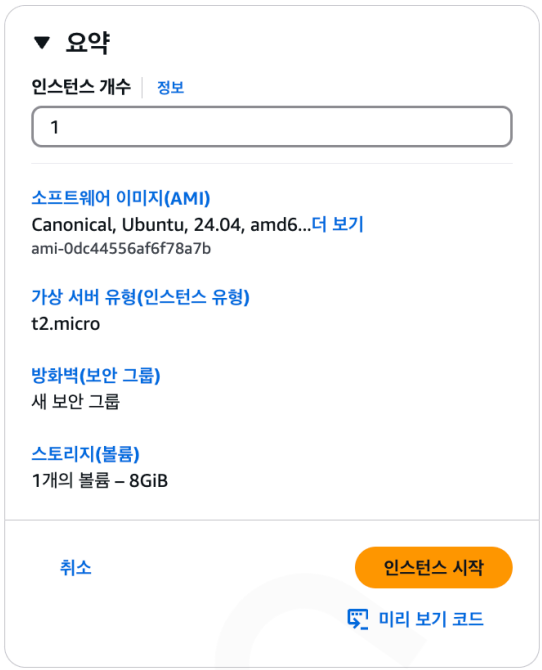
새 볼륨 추가

선택한 AMI에 인스턴스가 허용하는 것보다 많은 인스턴스 스토어 볼륨이 포함되어 있습니다. AMI에서 처음 0개의 인스턴스 스토어 볼륨에만 액세스할 수 있습니다.

백업 정보를 보려면 새로 고침 클릭
할당한 태그에 따라 Data Lifecycle Manager 정책으로 인스턴스를 백업할지 여부가 결정됩니다.

0 x 파일 시스템 편집

## ▶ 고급 세부 정보 정보




## ✅ EC2에 부하테스트 툴 셋팅하기

1. **k6 설치**

```
$ sudo gpg -k && /
sudo gpg --no-default-keyring --keyring /usr/share/keyrings/k6-archive-keyring.gpg --keyserver hkp://keyserver.ubu
echo "deb [signed-by=/usr/share/keyrings/k6-archive-keyring.gpg] https://dl.k6.io/deb stable main" | sudo tee /etc/a
sudo apt-get update && /
sudo apt-get install k6
```

2. **k6 잘 설치됐는 지 확인하기**

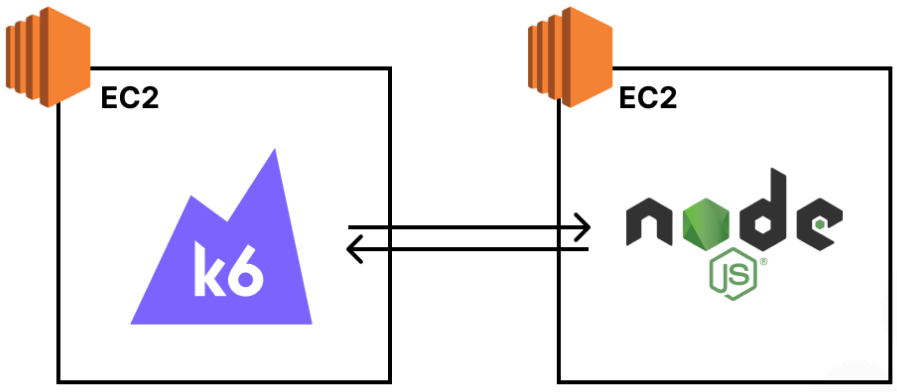
```
$ k6
```

```
ubuntu@ip-172-31-13-107:~$ k6

        /\      Grafana   /‾‾/
     /\  /  \     |\  __   /  /
    /  \/    \    | |/ /  /   ‾‾\
   /          \   |   (  |  (‾)  |
  / __________ \  |_|\_\  \_____/

Usage:
  k6 [command]

Available Commands:
  archive     Create an archive
  cloud       Run a test on the cloud
  completion  Generate the autocompletion script for the specified shell
  help        Help about any command
  inspect     Inspect a script or archive
  new         Create and initialize a new k6 script
  pause       Pause a running test
  resume      Resume a paused test
  run         Start a test
  scale       Scale a running test
  stats       Show test metrics
  status      Show test status
  version     Show application version

Flags:
  -a, --address string      address for the REST API server (default "localhost:6565")
  -c, --config string       JSON config file (default "/home/ubuntu/.config/loadimpact/k6/config.json")
  -h, --help                help for k6
      --log-format string   log output format
      --log-output string   change the output for k6 logs, possible values are stderr,stdout,none,loki[=host:p
ort],file[=./path.fileformat] (default "stderr")
      --no-color            disable colored output
      --profiling-enabled   enable profiling (pprof) endpoints, k6's REST API should be enabled as well
  -q, --quiet               disable progress updates
  -v, --verbose             enable verbose logging
      --version             version for k6

Use "k6 [command] --help" for more information about a command.
ubuntu@ip-172-31-13-107:~$
```

## ✅ 주의점

1. **부하 테스트 환경 독립적으로 분리**

부하 테스트 툴(k6)은 테스트하고자 하는 시스템(백엔드, DB 등)과 반드시 독립적으로 분리해서 구성해야 한다. 왜냐면 부하 테스트 툴 자체도 트래픽을 만들어내면서 컴퓨팅 리소스(CPU, 메모리 등)를 사용하기 때문이다.




2. **부하 테스트 툴을 개인 컴퓨터에 설치하지 않기**

   개인이 가지고 있는 컴퓨터는 Windows 또는 Mac OS일텐데, 컴퓨터 자체적으로 부하를 발생시킬 때 제한이 걸릴 때가 종종 있다. 따라서 EC2 인스턴스를 생성해 Linux 환경에서 부하 테스트를 진행할 것을 권장한다.

**▼ 참고**

K6를 이용한 서버 성능 테스트 이슈

서버 성능 테스트를 위해 k6 셋팅을 하고 테스트로 요청을 하는 도중에 아래와 같은 에러가 발생했다.WARN[0000] Request Failed error="Get \"http://localhost:8080/test/ping\": read tcp 127.0.0.1:62761→127.0.0.1:8080: read: connection reset by peer" K6 scriptexport const options = {

https://san-tiger.tistory.com/entry/K6%EB%A5%BC-%EC%9D%B4%EC%9A%A9%ED%95%9C-%EC%84%9C%EB%B2%84-%EC%84%B1%EB%8A%A5-%ED%85%8C%EC%8A%A4%ED%8A%B8-%EC%9D%B4%EC%8A%88

K6는 맥에서 돌리지 마세요..... 제발(connection reset by peer)

MacOS에서 K6로 부하 테스트를 진행할 때,vuser를 늘리다 보면 높은 확률로 read: connection reset by peer 문제를 마주하게 됩니다.이는 MacOS에서 소켓에 제한을 걸어서 발생되는 문제이므로K6를 Docker Container로 띄우거나다른 운영체제에서 테스트를 돌리면 해결됩니다.

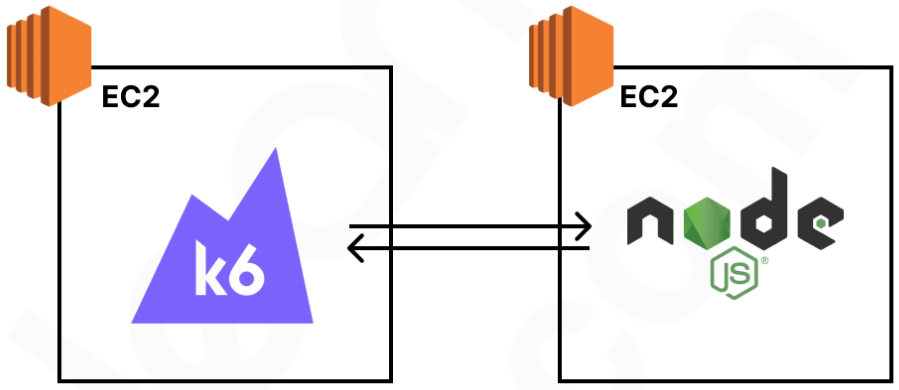
https://kinggodgeneral.tistory.com/72

khb3025@gmail.com

# 🎯 [실습] 내가 구성한 백엔드 서버는 1초당 몇 개의 요청을 견뎌낼 수 있을까?

## ✅ 부하 테스트 할 인프라 구성

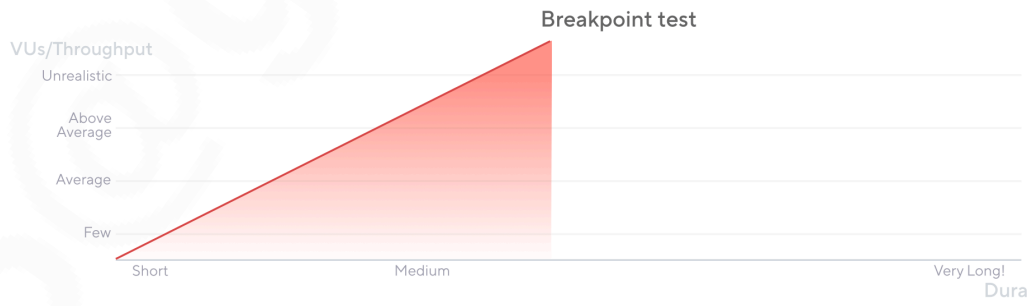
👩🏻‍🏫 부하 테스트를 하기 전에 인프라의 전체 구성을 그림으로 그려놓으면, 전체적인 트래픽의 흐름을 한 눈에 파악할 수 있어서 편리하다.




## ✅ k6를 활용해 부하 테스트 진행하기

1. **백엔드 서버에 부하를 주기 위해 k6 스크립트 작성**

> 구성한 시스템이 1초당 몇 개의 요청을 견딜 수 있는 지 알아보려면, 점진적으로 트래픽을 늘려가게끔 부하 테스트를 셋팅해야 한다.

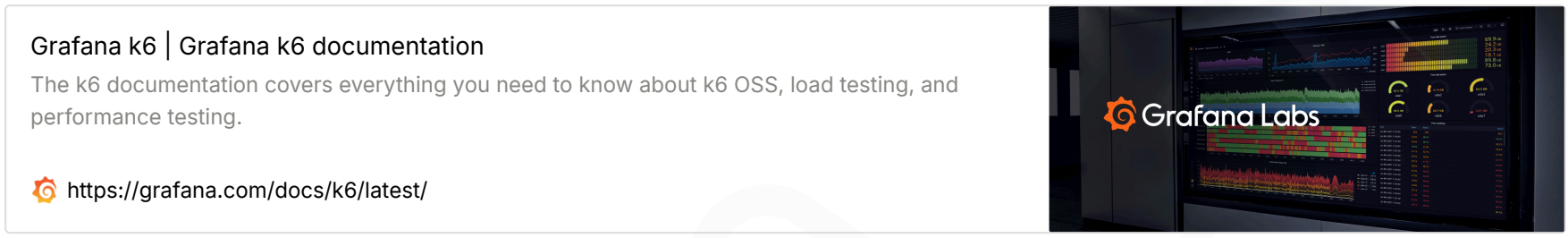



**script.js**

```
import http from 'k6/http';
import { sleep } from 'k6';

export const options = {
  // 부하를 생성하는 단계(stages)를 설정
  stages: [
    // 10분에 걸쳐 vus(virtual users, 가상 유저수)가 6000에 도달하도록 설정
    { duration: '10m', target: 6000 }
  ],
};

export default function () {
```

```
  // API 주소로 GET 요청
  http.get('http://{EC2 IP 주소}/boards');
  // 1초 휴식
  sleep(1);
}
```

▼ **참고)** k6 공식문서

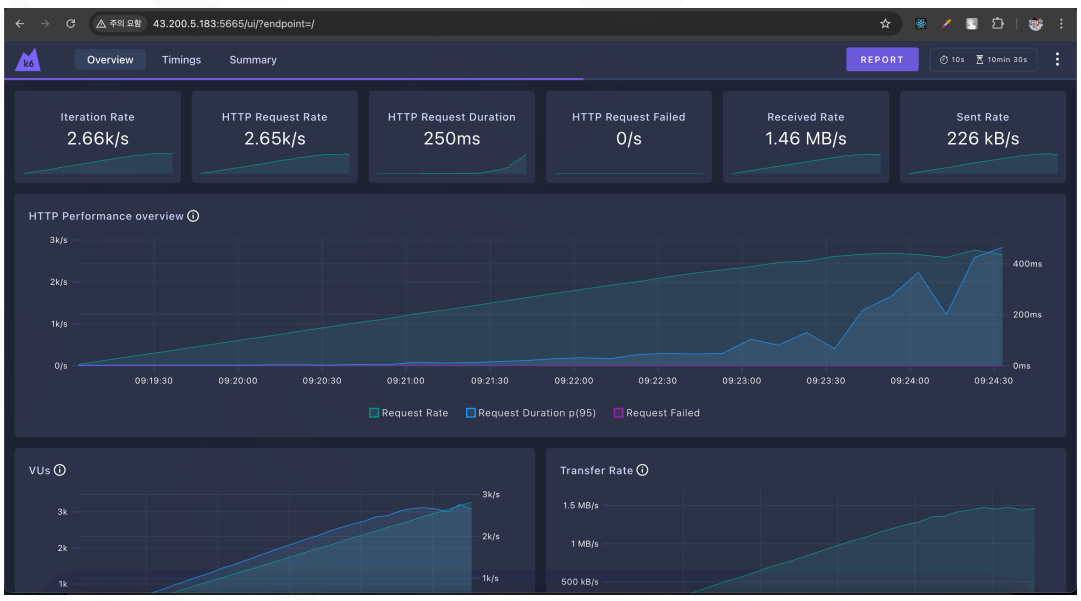



2. **부하 테스트 시작하기**

```
$ K6_WEB_DASHBOARD=true k6 run script.js
```

3. **k6의 웹 대시보드 열기**

`http://{k6가 실행되고 있는 EC2 IP 주소}:5665`으로 접속하기




## ✅ 부하 테스트 결과 해석 방법

k6의 웹 대시보드에서 부하 테스트 결과를 보면 수치가 너무 많아 복잡하게 느껴질 수 있다. 하지만 OT 때 얘기했던 파레토의 법칙에 따라 딱 3가지 수치만 확인하면 된다.

1. **HTTP Request Rate**

   **1초당 처리한 요청 수**를 의미한다. 즉, **Throughput(처리량)**을 나타내는 값이다.

2. **HTTP Request Duration**

   **요청에 대한 평균 응답 시간**을 의미한다. 즉, **Latency(지연 시간)**을 나타내는 값이다.

3. **HTTP Request Failed**

   **1초당 요청 실패 수**를 의미한다. 요청 실패가 발생한다면 어떤 문제로 인해 요청이 실패하는 지 체크해봐야 하는 신호이다.

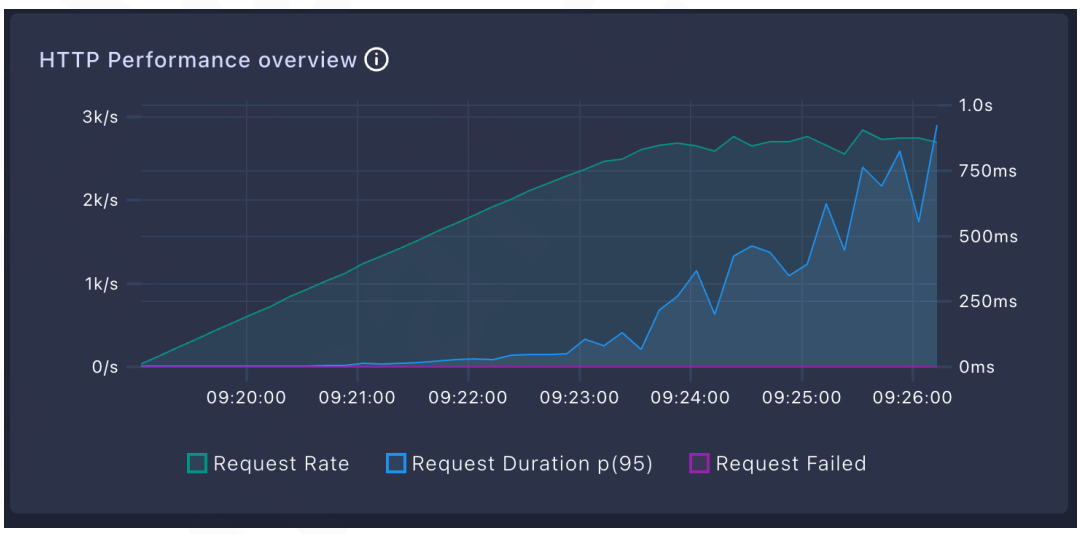
## ✅ 실제 해석 순서

1. **VUs가 늘어나도 HTTP Request Rate가 더 이상 증가하지 않는 구간을 찾는다.**

   더 이상 증가하지 않는 HTTP Request Rate가 현재 시스템의 최대 Throughput이다. 초당 2,700개 정도의 요청을 처리하고 있다. 즉, 현재 시스템은 1초당 2,700개의 요청을 처리할 수 있다는 뜻이다. 이걸 보고 '**현재 시스템의 최대 Throughput은 2,700 TPS**'라고 표현한다.

2. **HTTP Request Duration이 비이상적으로 높은 건 아닌지 체크한다.**

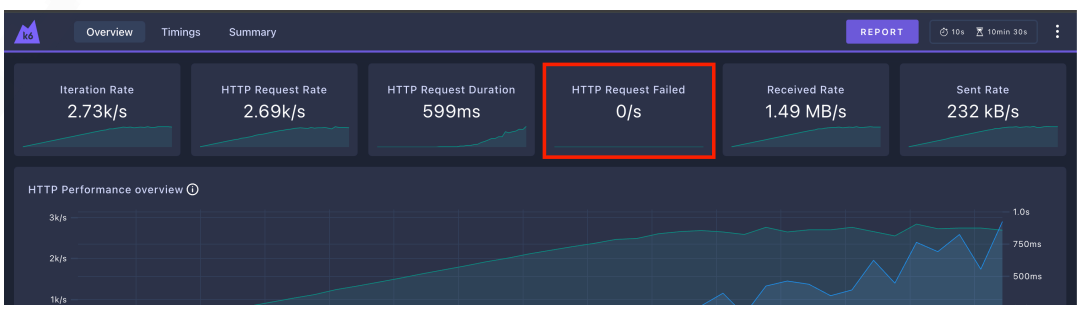
   요청 당 응답 시간(Latency)이 비이상적으로 높을 경우 문제가 있는 건 아닌 지 체크해봐야 한다. 여기서 '비이상적'의 기준은 정하기 나름이다. 특정 서비스에서 응답 시간이 1초가 넘어가는 순간 사용자 이탈률이 크게 증가한다면, 정상적인 응답 시간 기준을 1초로 잡으면 된다.




   **참고)** VUs가 높아짐에 따라 서버가 처리할 수 있는 요청량이 밀려서 대기하는 시간이 늘어나기 때문에, HTTP Request Rate(= Throughput)는 늘어나지 않는데 Request Duration(Latency)는 늘어나는 현상을 확인할 수 있다.

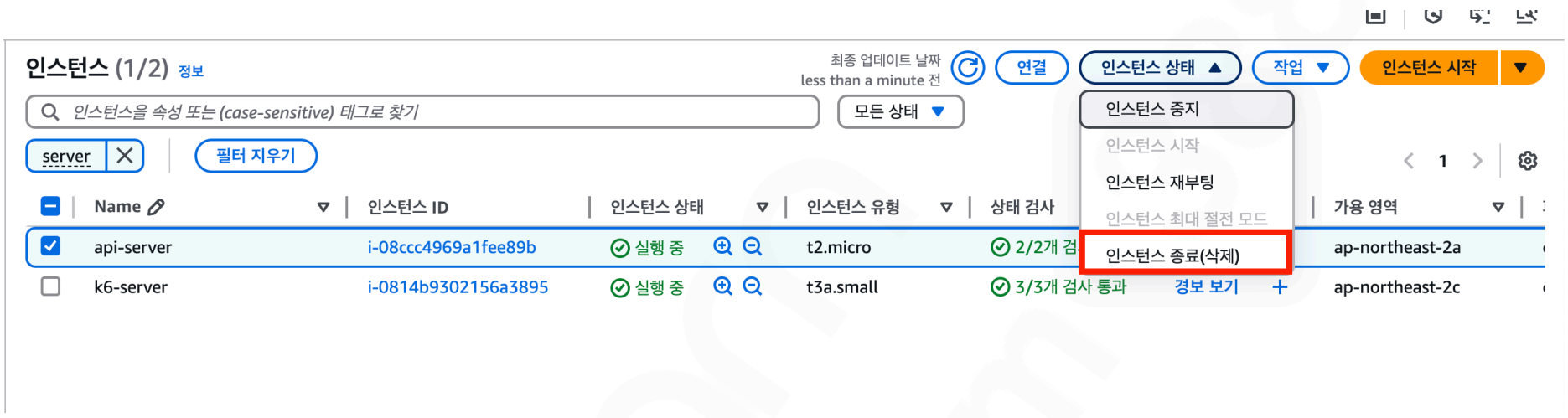
3. **HTTP Request Failed가 있는 건 아닌지 체크한다.**

   만약 실패한 요청이 있다면 요청이 왜 실패했는 지 분석해야 한다.



# [실습] 사용하지 않는 EC2 종료하기

## ✅ 사용하지 않는 EC2 종료하기




`api-server`라고 이름 붙인 EC2 인스턴스는 이제 사용하지 않을 예정이라 종료해주자. 하지만 `k6-server`라고 이름 붙인 EC2 인스턴스는 부하 테스트할 때 써야 하기 때문에 남겨두자.

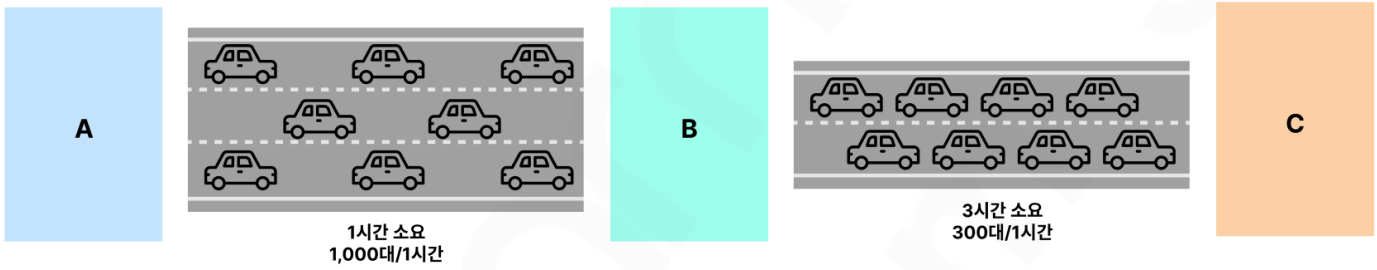
khb302@gmail.com

# 병목 지점 (Bottleneck Point)

## ✅ 병목 지점(Bottleneck Point)이란?

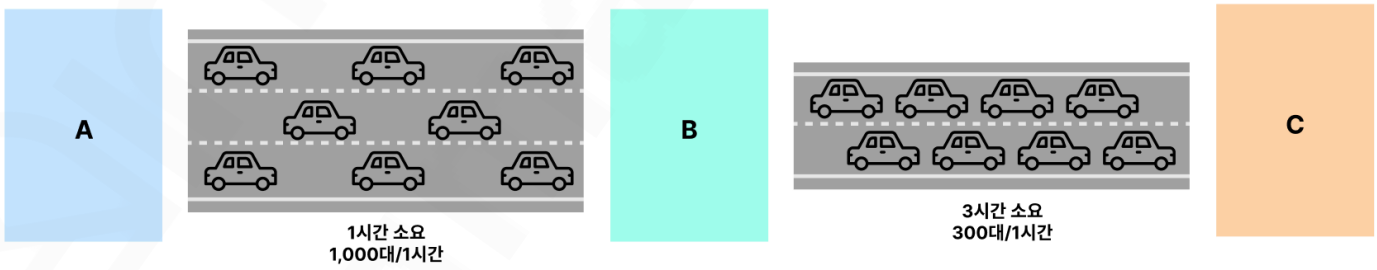
부하테스트에서 **병목 지점(Bottleneck Point)**이라는 말을 많이 사용한다. **병목 지점(Bottleneck Point)**이란, **전체 시스템에서 특정 서버 자원(CPU, Memory 등)이 한계에 도달해 전체 성능이 저하되는 구간**을 의미한다.

고속 도로에 비유하자면 길이 갑자기 좁아지는 구간에서 차가 확 막히는 경우가 많다. 3차선 도로가 소화할만한 차들이 갑자기 2차선으로 몰려들면서 정체가 발생한다. 그럼 이 2차선 구간에서 발생한 정체를 보고 **병목 지점(Bottlenectk Point)**라고 얘기한다.




전체적인 고속 도로 정체를 해결하기 위해서는 교통 정체가 일어난 곳을 해결해야 한다. 정체가 없는 3차선 도로를 아무리 더 넓혀봐야, 2차선 도로의 정체가 나아지지 않는다. 정체가 발생한 2차선 도로를 해결해야 전체적인 고속 도로 정체가 해결된다. 이와 비슷하게 **시스템의 성능을 개선하려면 '병목 지점'을 개선해야 한다.**

## ✅ '병목 지점의 Throughput'이 곧 '전체 Throughput'이다.

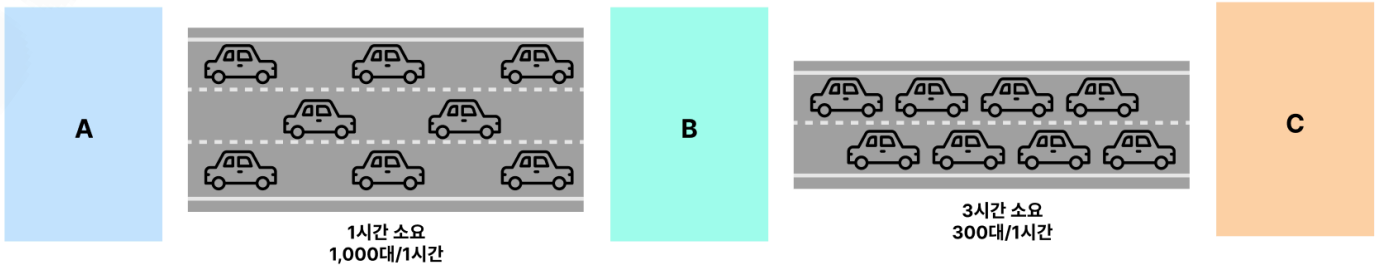



`A~B~C` 까지 1시간에 통과할 수 있는 자동차의 대수가 300대이다. 왜냐하면 `A~B`의 구간에서 아무리 1시간에 1,000대를 통과시킨다고 하더라도, `B~C` 구간에서 1시간에 300대만 통과할 수 있기 때문이다.

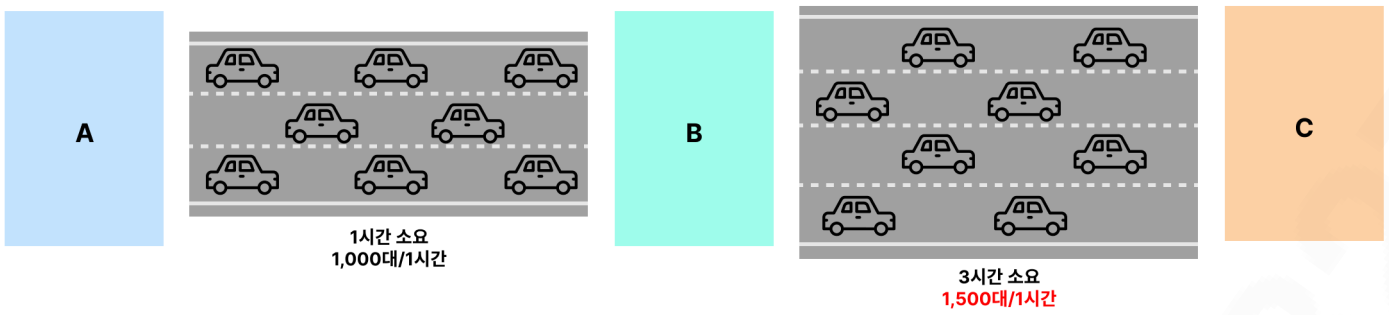
시스템에서도 동일하게 적용된다. `사용자 ↔ 백엔드 서버 ↔ DB`의 구조에서 백엔드 서버가 1초에 1,000개의 요청을 처리할 수 있고, DB가 1초에 300개의 요청을 처리할 수 있다고 가정하자. 그러면 전체적인 시스템에서 1초당 처리할 수 있는 요청의 양은 300개이다.

위 사례를 통해 **'병목 지점의 Throughput'이 곧 '전체 Throughput'**이라는 걸 알 수 있다.

## ✅ 특정 병목 지점을 해소하면 다른 곳에서 새로운 병목 지점이 발생한다.




`A~B~C` 구간에서 통과할 수 있는 자동차의 대수는 시간당 300대이다. `A~B~C` 구간을 통과할 수 있는 자동차 대수를 늘리기 위해 병목 지점인 `B~C` 구간을 아래와 같이 개선해봤다.




이제는 `A~B~C` 구간에서 통과할 수 있는 자동차의 대수는 시간당 1,000대 이다. `B~C` 구간에서는 시간당 1,500대의 자동차가 지나갈 수 있을지라도, `A~B` 구간에서는 시간당 1,000대의 자동차만 지나갈 수 있기 때문이다.

위 예를 통해 **특정 병목 지점을 해소하면 다른 곳에서 새로운 병목 지점이 발생한다는 점**을 알 수 있다.

@kfle@gmail.com
khb302571

📖

# 부하 테스트의 전체 흐름

## ✅ 1. 부하 테스트 필요성 인식

“서비스를 오픈하려고 하는데 사용자들이 몰려서 서버가 터지면 어떡하지?”

“내 서버는 어느 정도 사용자 요청을 견딜 수 있는 거지?”

“개발자님, 이번에 치킨 이벤트를 하려하는데 사용자가 1만명 정도가 들어와도 서버가 괜찮을까요?”

## ✅ 2. 부하 테스트 목표 설정하기

부하 테스트 목표는 주로 **Throughput**과 **Latency**를 활용해 설정한다.

**[예시]**

- **목표 Throughput : 2000TPS**

  → 서비스에 예상 접속자 수, 예상 요청 수 등을 활용해 TPS 목표를 설정한다.

- **평균 Latency : 800ms**

  → 서비스의 특성에 맞게 Latency를 설정한다.

## ✅ 3. 현재 시스템이 어느 정도 트래픽까지 견딜 수 있는 지 부하 테스트 진행

부하 테스트 툴(k6, ngrinder, jmeter 등)을 활용해 시스템에 부하를 준다. 부하의 정도를 올려가면서 어느 정도 부하까지 견딜 수 있는 지를 측정한다. 즉, 시스템의 최대 Throughput을 측정한다.

## ✅ 4. 병목 지점 파악 후 성능 개선하기

목표로 설정한 Throughput, Latency를 달성하기 위해, 기존 시스템의 성능을 개선해야 할 수도 있다. 성능 개선을 하려면 가장 먼저 병목 지점을 파악해서 개선해야 한다.

## ✅ 5. 개선한 시스템이 어느 정도 트래픽까지 견딜 수 있는 지 부하 테스트 진행

개선한 시스템이 어느 정도 트래픽까지 견딜 수 있는 지 부하 테스트를 진행한다. 부하 테스트를 통해 최대 Throughput과 평균 Latency를 측정해서 목표치를 달성했는 지 체크한다.

만약 목표치를 달성하지 못했다면 다시 4번 과정(병목 지점 파악 후 성능 개선)을 거친 후 부하 테스트를 진행해서 목표치 달성 여부를 확인한다. 이 과정을 목표 달성 때까지 반복한다.

📖

# 부하 테스트 시 주의점

## ✅ 적절한 부하 테스트 시간

1분 간격으로 기록되는 모니터링 도구를 가지고 정확하게 성능을 측정하려면 최소 5분간의 부하테스트는 진행해야 한다. 그래야 일관된 결과값을 얻을 수 있다.

## ✅ 프로덕션 환경과 비슷한 데이터 셋팅

데이터베이스는 데이터가 어떻게 저장되어 있는 지와 얼마나 많은 양이 저장되어 있는 지에 따라서 성능 차이가 많이 난다. 따라서 실제 프로덕션 환경과 비슷하게 데이터를 구성해두고 부하 테스트를 해야 보다 정확한 결과값을 얻을 수 있다.

현업에서 부하 테스트를 할 때는 프로덕션 환경에 저장되어 있는 데이터의 양만큼 셋팅을 해두고 부하 테스트를 한다. 이 과정을 조금 더 간소화하기 위해서는 프로덕션 환경의 데이터들이 포함된 DB 복제본을 떠서 그 DB로 부하 테스트를 진행하기도 한다.

## ✅ 프로덕션과 분리된 환경에서 테스트하기

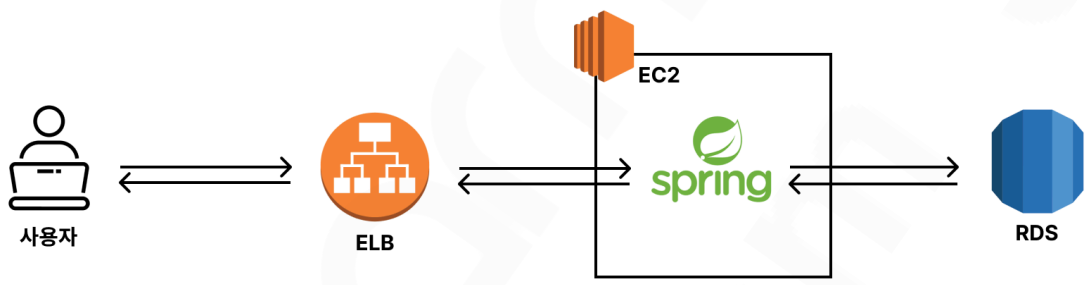
프로덕션 환경에서 부하 테스트를 하면 안 된다. 왜냐하면 부하 테스트를 함으로써 실제 서비스의 성능에 악영향을 끼칠 수 있기 때문이다.

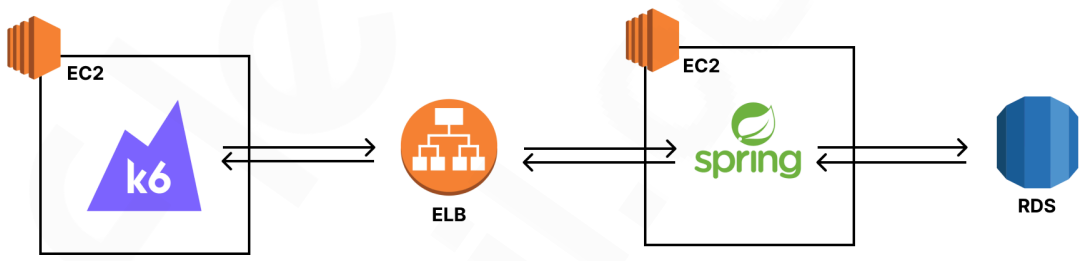
khb302571@gmail.com

# [실습] 부하 테스트 할 인프라 구성을 그림으로 파악하기

부하 테스트를 할 때 병목 지점을 찾는 게 핵심이다. 병목 지점을 쉽게 찾으려면 트래픽이 거쳐가는 인프라가 어떤 것들이 있는 지 파악해야 한다. 따라서 부하 테스트를 하기 전에 인프라 아키텍처에 대한 전체적인 그림을 먼저 그려보는 걸 추천한다.

## ✅ 인프라 아키텍처 구성



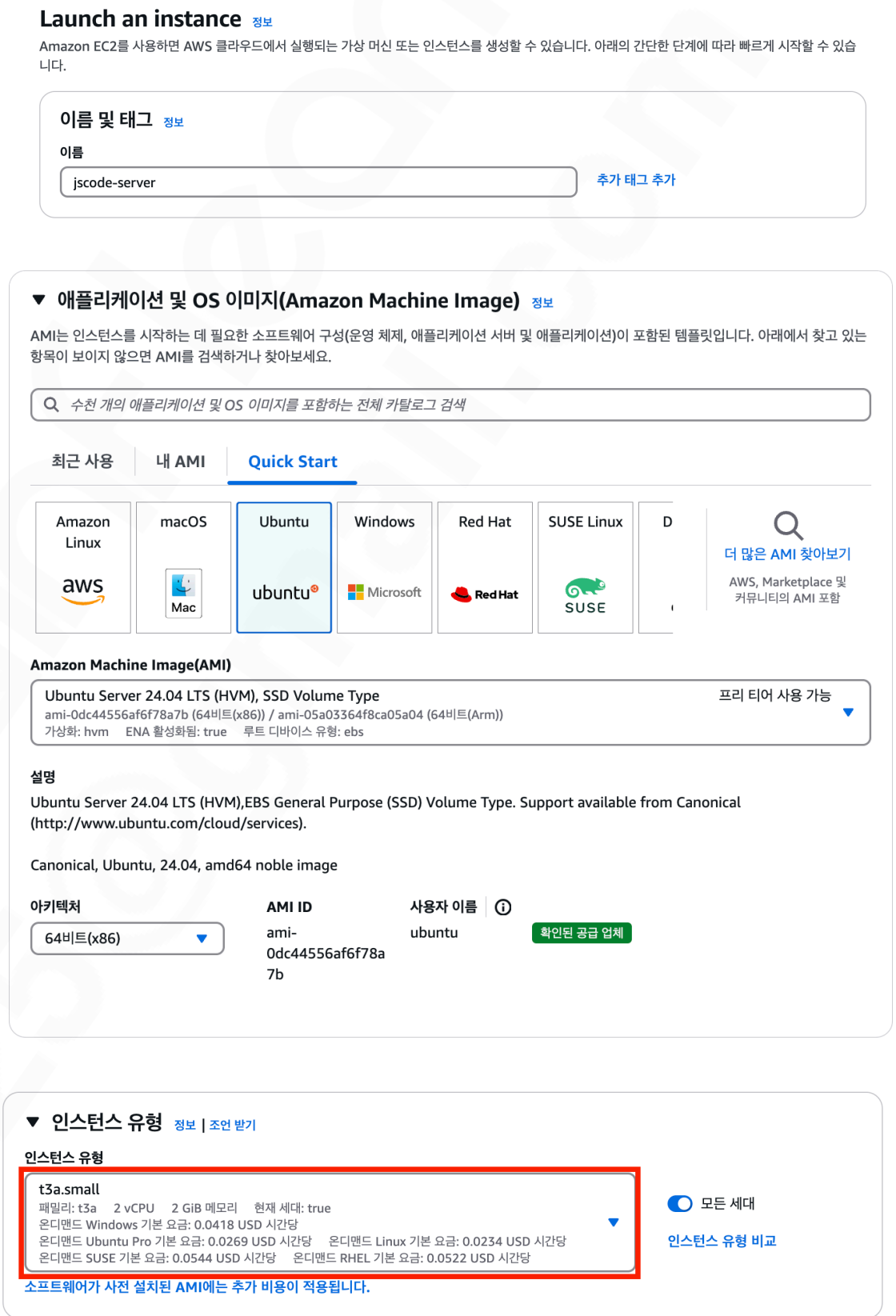

khb3025@gmail.com

# 🎯 [실습] AWS EC2 셋팅하기

👩🏻 AWS에 초점을 맞춘 강의가 아니기 때문에 EC2, RDS, ELB에 대한 디테일한 설명은 생략할 예정이다. 혹시 AWS에 대한 기본기를 다지고 싶다면 아래 강의를 추천한다.

**비전공자도 이해할 수 있는 AWS 입문/실전 (https://inf.run/73afg)**

## ✅ EC2 인스턴스 생성하기




→ Spring Boot가 기본적으로 무겁기 때문에 프리티어인 t2.micro로 돌릴 경우 서버가 멈춰버릴 수 있다.

## ▼ 키 페어(로그인) 정보

키 페어를 사용하여 인스턴스에 안전하게 연결할 수 있습니다. 인스턴스를 시작하기 전에 선택한 키 페어에 대한 액세스 권한이 있는지 확인하세요.

**키 페어 이름 - *필수***

키 페어 없이 계속 진행(권장되지 않음) 기본값 ▼ 새 키 페어 생성

## ▼ 네트워크 설정 정보

편집

**네트워크** | 정보

vpc-d0897abb

**서브넷** | 정보

기본 설정 없음(가용 영역의 기본 서브넷)

**퍼블릭 IP 자동 할당** | 정보

활성화

프리 티어 허용 범위를 벗어나는 경우 추가 요금이 적용됩니다.

**방화벽(보안 그룹)** | 정보

보안 그룹은 인스턴스에 대한 트래픽을 제어하는 방화벽 규칙 세트입니다. 특정 트래픽이 인스턴스에 도달하도록 허용하는 규칙을 추가합니다.

◉ 보안 그룹 생성 ○ 기존 보안 그룹 선택

다음 규칙을 사용하여 **'launch-wizard-12'**(이)라는 새 보안 그룹을 생성합니다.

☑ 다음에서 SSH 트래픽 허용
인스턴스 연결에 도움
위치 무관 0.0.0.0/0

☐ 인터넷에서 HTTPS 트래픽 허용
예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면

☑ 인터넷에서 HTTP 트래픽 허용
예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면

⚠ 소스가 0.0.0.0/0인 규칙은 모든 IP 주소에서 인스턴스에 액세스하도록 허용합니다. 알려진 IP 주소의 액세스만 허용하도록 보안 그룹을 설정하는 것이 좋습니다. ✕

## ▼ 스토리지 구성 정보

고급

1x 8 GiB gp3 루트 볼륨 (암호화되지 않음)

새 볼륨 추가

선택한 AMI에 인스턴스가 허용하는 것보다 많은 인스턴스 스토어 볼륨이 포함되어 있습니다. AMI에서 처음 0개의 인스턴스 스토어 볼륨에만 액세스할 수 있습니다.

백업 정보를 보려면 새로 고침 클릭
할당한 태그에 따라 Data Lifecycle Manager 정책으로 인스턴스를 백업할지 여부가 결정됩니다.

0 x 파일 시스템 편집

## ▶ 고급 세부 정보 정보

## ▼ 요약

**인스턴스 개수** | 정보

1

**소프트웨어 이미지(AMI)**
Canonical, Ubuntu, 24.04, amd6...더 보기
ami-0dc44556af6f78a7b

**가상 서버 유형(인스턴스 유형)**
t2.micro

**방화벽(보안 그룹)**
새 보안 그룹

**스토리지(볼륨)**
1개의 볼륨 – 8GiB

취소 인스턴스 시작

미리 보기 코드

khb302571

# [실습] AWS RDS 셋팅하기

## ✅ AWS RDS 생성하기

1. **RDS 생성하기**

데이터베이스 생성 정보

데이터베이스 생성 방식 선택

- (선택) 표준 생성 — 가용성, 보안, 백업 및 유지 관리에 대한 옵션을 포함하여 모든 구성 옵션을 설정합니다.
- ( ) 손쉬운 생성 — 권장 모범 사례 구성을 사용합니다. 일부 구성 옵션은 데이터베이스를 생성한 후 변경할 수 있습니다.

엔진 옵션

엔진 유형 정보

- ( ) Aurora (MySQL Compatible)
- ( ) Aurora (PostgreSQL Compatible)
- (선택) MySQL
- ( ) PostgreSQL
- ( ) MariaDB
- ( ) Oracle
- ( ) Microsoft SQL Server
- ( ) IBM Db2

에디션

(선택) MySQL Community

엔진 버전 정보

다음 데이터베이스 기능을 지원하는 엔진 버전을 표시합니다.

▼ 필터 숨기기

- [ ] 다중 AZ DB 클러스터를 지원하는 버전만 표시 정보
  기본 DB 인스턴스 1개와 읽기 가능한 대기 DB 인스턴스 2개로 다중 AZ DB 클러스터를 생성합니다. 다중 AZ DB 클러스터는 최대 2배 빠른 트랜잭션 커밋 지연 시간과 일반적으로 35초 미만의 자동 장애 조치를 제공합니다.
- [ ] Amazon RDS 최적화된 쓰기를 지원하는 버전만 표시 정보
  Amazon RDS 최적화된 쓰기는 추가 비용 없이 쓰기 처리량(throughput)을 최대 2배 늘립니다.

엔진 버전

MySQL 8.0.39

- [ ] RDS 확장 지원 활성화 정보
  Amazon RDS 확장 지원은 유료 오퍼링입니다. 이 옵션을 선택하면 해당 버전의 RDS 표준 지원 종료일 이후에 데이터베이스 메이저 버전을 실행하는 경우 오퍼링의 요금이 청구되는 데 동의하는 것으로 간주됩니다. RDS for MySQL 설명서에서 메이저 버전의 표준 지원 종료일을 확인하세요.

템플릿

해당 사용 사례를 충족하는 샘플 템플릿을 선택하세요.

- ( ) 프로덕션 — 고가용성 및 빠르고 일관된 성능을 위해 기본값을 사용하세요.
- ( ) 개발/테스트 — 이 인스턴스는 프로덕션 환경 외부에서 개발 용도로 마련되었습니다.
- (선택) 프리 티어 — RDS 프리 티어를 사용하여 새로운 애플리케이션을 개발하거나, 기존 애플리케이션을 테스트하거나 Amazon RDS에서 실무 경험을 쌓을 수 있습니다. 정보

## 가용성 및 내구성

**배포 옵션** 정보

아래의 배포 옵션은 위에서 선택한 엔진에서 지원하는 배포 옵션으로 제한됩니다.

- 다중 AZ DB 클러스터
  기본 DB 인스턴스와 읽기 가능한 예비 DB 인스턴스 2개가 있는 DB 클러스터를 생성합니다. 각 DB 인스턴스는 서로 다른 가용 영역(AZ)에 있습니다. 고가용성, 데이터 이중화를 제공하고 읽기 워크로드를 처리하기 위한 용량을 늘립니다.
- 다중 AZ DB 인스턴스(다중 AZ DB 클러스터 스냅샷에는 지원되지 않음)
  다른 AZ에 기본 DB 인스턴스와 예비 DB 인스턴스를 생성합니다. 고가용성 및 데이터 이중화를 제공하지만 예비 DB 인스턴스는 읽기 워크로드에 대한 연결을 지원하지 않습니다.
- 단일 DB 인스턴스(다중 AZ DB 클러스터 스냅샷에는 지원되지 않음)
  예비 DB 인스턴스가 없는 단일 DB 인스턴스를 생성합니다.

## 설정

**DB 인스턴스 식별자** 정보

DB 인스턴스 이름을 입력하세요. 이름은 현재 AWS 리전에서 AWS 계정이 소유하는 모든 DB 인스턴스에 대해 고유해야 합니다.

jscode-database

DB 인스턴스 식별자는 대소문자를 구분하지 않지만 'mydbinstance'와 같이 모두 소문자로 저장됩니다. 제약: 1~60자의 영숫자 또는 하이픈으로 구성되어야 합니다. 첫 번째 문자는 글자여야 합니다. 하이픈 2개가 연속될 수 없습니다. 하이픈으로 끝날 수 없습니다.

▼ 자격 증명 설정

**마스터 사용자 이름** 정보

DB 인스턴스의 마스터 사용자에 로그인 ID를 입력하세요.

admin

1~16자의 영숫자. 첫 번째 문자는 글자여야 합니다.

**자격 증명 관리**

AWS Secrets Manager를 사용하거나 마스터 사용자 자격 증명을 관리할 수 있습니다.

- AWS Secrets Manager에서 관리 - *가장 뛰어난 안정성*
  RDS는 자동으로 암호를 생성하고 AWS Secrets Manager를 사용하여 전체 수명 주기 동안 암호를 관리합니다.
- 자체 관리
  사용자가 암호를 생성하거나 RDS에서 암호를 생성하고 사용자가 관리할 수 있습니다.

☐ 암호 자동 생성
Amazon RDS에서 자동으로 암호를 생성하거나 사용자가 직접 암호를 지정할 수 있습니다.

**마스터 암호** | 정보

••••••••

Password strength Very weak

최소 제약 조건: 8자 이상의 인쇄 가능한 ASCII 문자를 사용합니다. / ' " @ 기호는 포함할 수 없습니다.

**마스터 암호 확인** | 정보

••••••••

## 인스턴스 구성

아래의 DB 인스턴스 구성 옵션은 위에서 선택한 엔진에서 지원하는 옵션으로 제한됩니다.

**DB 인스턴스 클래스** | 정보

▼ 필터 숨기기

☐ Amazon RDS 최적화된 쓰기를 지원하는 인스턴스 클래스 표시 정보
Amazon RDS 최적화된 쓰기는 추가 비용 없이 쓰기 처리량(throughput)을 최대 2배 늘립니다.

☐ 이전 세대 클래스 포함

- 스탠다드 클래스(m 클래스 포함)
- 메모리 최적화 클래스(r 및 x 클래스 포함)
- 버스터블 클래스(t 클래스 포함)

db.t4g.micro
2 vCPUs 1 GiB RAM 네트워크: 최대 2,085Mbps

**(나머지는 기본 설정으로 그대로 두기)**

## 연결 정보

**컴퓨팅 리소스**

이 데이터베이스의 컴퓨팅 리소스에 대한 연결을 설정할지를 선택합니다. 연결을 설정하면 컴퓨팅 리소스가 이 데이터베이스에 연결할 수 있도록 연결 설정이 자동으로 변경됩니다.

- (선택) EC2 컴퓨팅 리소스에 연결 안 함
  이 데이터베이스의 컴퓨팅 리소스에 대한 연결을 설정하지 않습니다. 나중에 컴퓨팅 리소스에 대한 연결을 수동으로 설정할 수 있습니다.
- EC2 컴퓨팅 리소스에 연결
  이 데이터베이스의 EC2 컴퓨팅 리소스에 대한 연결을 설정합니다.

**Virtual Private Cloud(VPC)** 정보

VPC를 선택합니다. VPC는 이 DB 인스턴스의 가상 네트워킹 환경을 정의합니다.

Default VPC (vpc-d0897abb)
4 서브넷, 4 가용 영역

해당 DB 서브넷 그룹이 있는 VPC만 나열됩니다.

ⓘ 데이터베이스를 생성한 후에는 VPC를 변경할 수 없습니다.

**DB 서브넷 그룹** 정보

DB 서브넷 그룹을 선택합니다. DB 서브넷 그룹은 선택한 VPC에서 DB 인스턴스가 어떤 서브넷과 IP 범위를 사용할 수 있는지를 정의합니다.

default
2 서브넷, 2 가용 영역

**퍼블릭 액세스** 정보

- (선택) 예
  RDS는 데이터베이스에 퍼블릭 IP 주소를 할당합니다. VPC 외부의 Amazon EC2 인스턴스 및 다른 리소스가 데이터베이스에 연결할 수 있습니다. VPC 내부의 리소스도 데이터베이스에 연결할 수 있습니다. 데이터베이스에 연결할 수 있는 리소스를 지정하는 VPC 보안 그룹을 하나 이상 선택합니다.
- 아니요
  RDS는 퍼블릭 IP 주소를 데이터베이스에 할당하지 않습니다. VPC 내부의 Amazon EC2 인스턴스 및 다른 리소스만 데이터베이스에 연결할 수 있습니다. 데이터베이스에 연결할 수 있는 리소스를 지정하는 VPC 보안 그룹을 하나 이상 선택합니다.

**VPC 보안 그룹(방화벽)** 정보

데이터베이스에 대한 액세스를 허용할 VPC 보안 그룹을 하나 이상 선택합니다. 보안 그룹 규칙이 적절한 수신 트래픽을 허용하는지 확인합니다.

- (선택) 기존 항목 선택
  기존 VPC 보안 그룹 선택
- 새로 생성
  새 VPC 보안 그룹 생성

**기존 VPC 보안 그룹**

*하나 이상의 옵션 선택*

db ×

**가용 영역** 정보

기본 설정 없음

**RDS 프록시**

RDS 프록시는 애플리케이션 확장성, 복원력 및 보안을 개선하는 완전관리형 고가용성 데이터베이스 프록시입니다.

☐ RDS 프록시 생성 정보

RDS는 프록시에 대한 IAM 역할과 Secrets Manager 보안 암호를 자동으로 생성합니다. RDS 프록시에 대한 추가 비용이 있습니다. 자세한 내용은 다음을 참조하세요.Amazon RDS 프록시 요금.

**인증 기관 - *선택 사항*** 정보

서버 인증서를 사용하면 Amazon 데이터베이스에 대한 연결이 이루어지고 있는지 검증하여 추가 보안 계층을 제공합니다. 프로비저닝하는 모든 데이터베이스에 자동으로 설치되는 서버 인증서를 확인하여 이를 수행합니다.

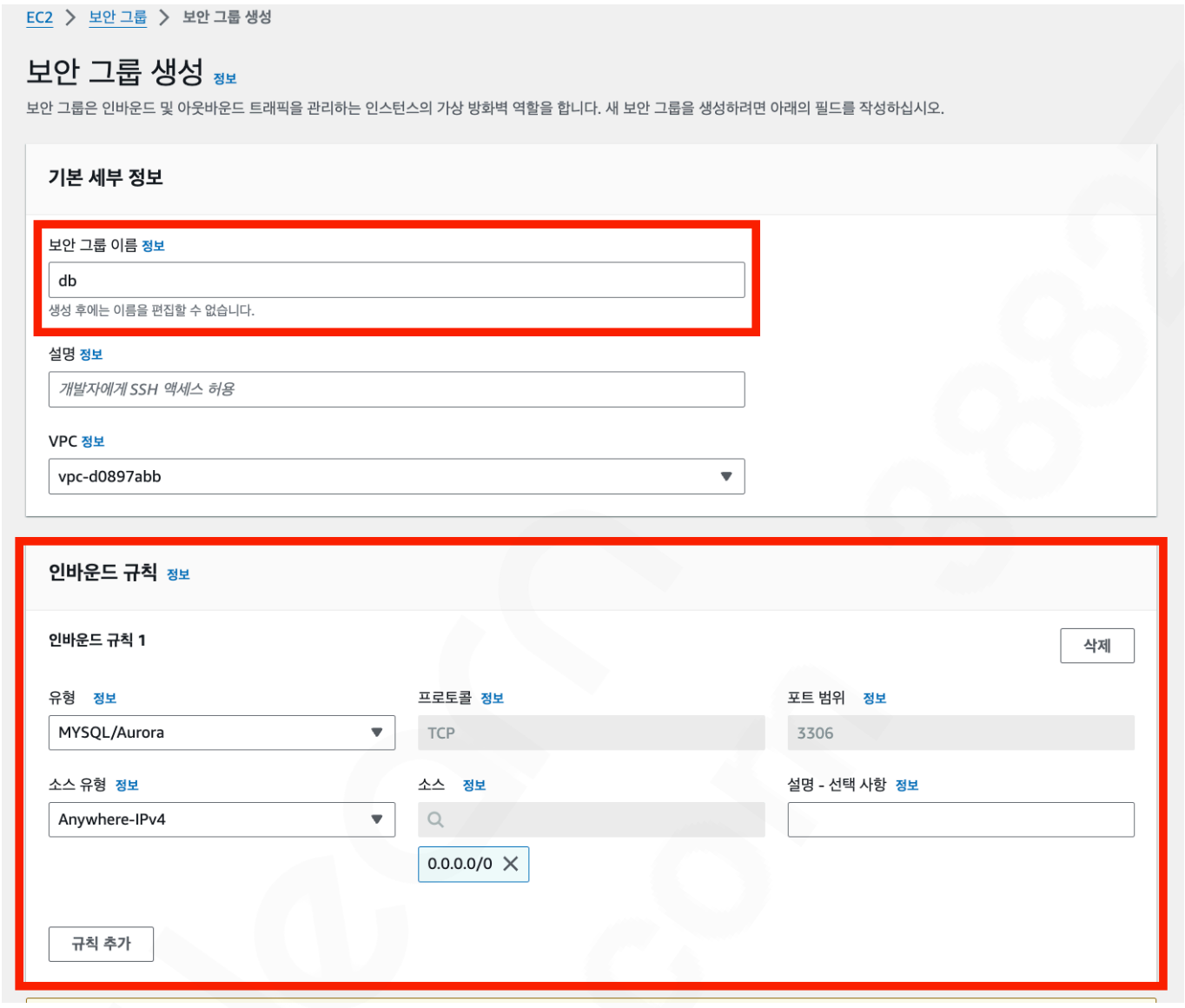
rds-ca-rsa2048-g1 (기본값)
만료: May 21, 2061

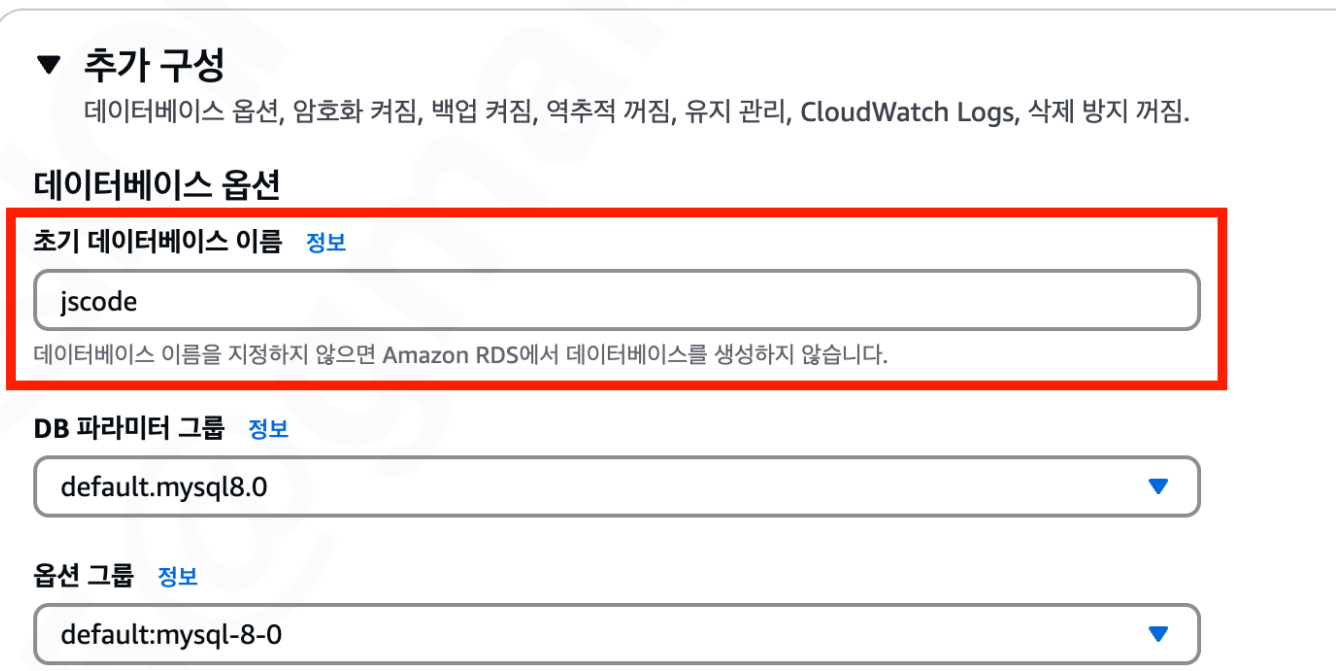
인증 기관을 선택하지 않으면 RDS에서 대신 인증 기관을 선택합니다.

▶ 추가 구성

▼ EC2 페이지에서 보안그룹 생성 한 뒤에 기존 VPC 보안 그룹 선택하기

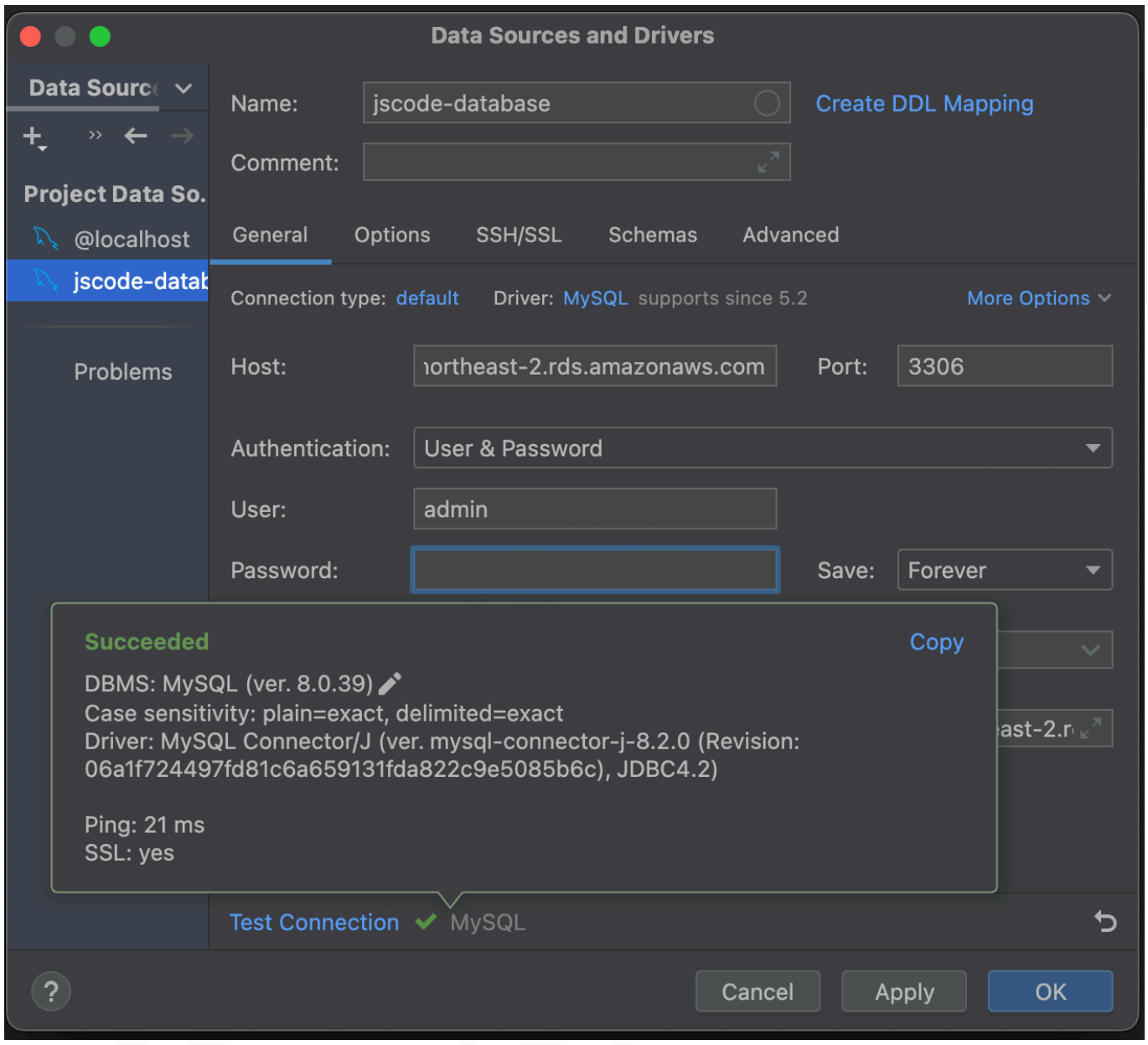
kha3023




**(나머지 옵션은 기본 설정 그대로 두기)**

2. **잘 접속되는 지 확인하기**



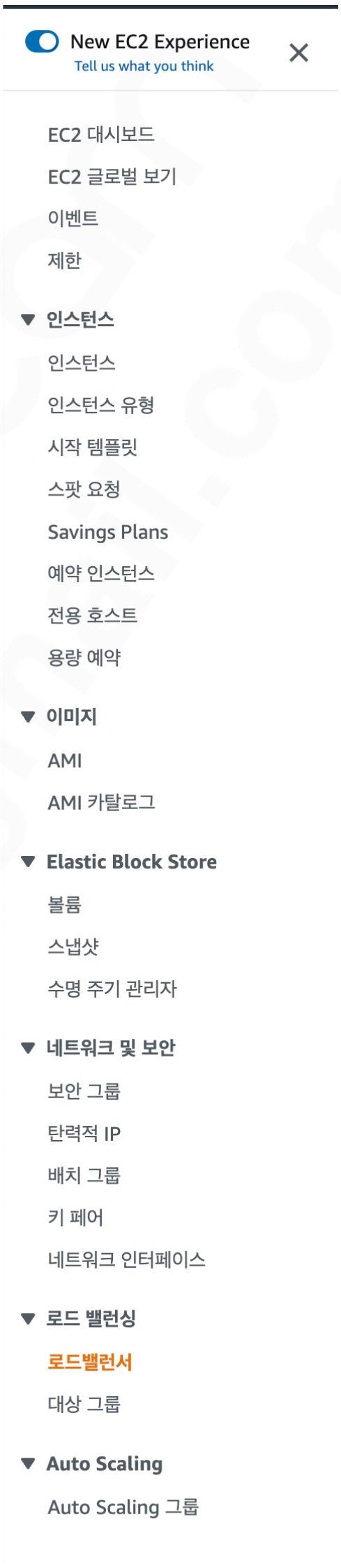

khb3025@gmail.com

# 🎯 [실습] AWS ELB 생성하기

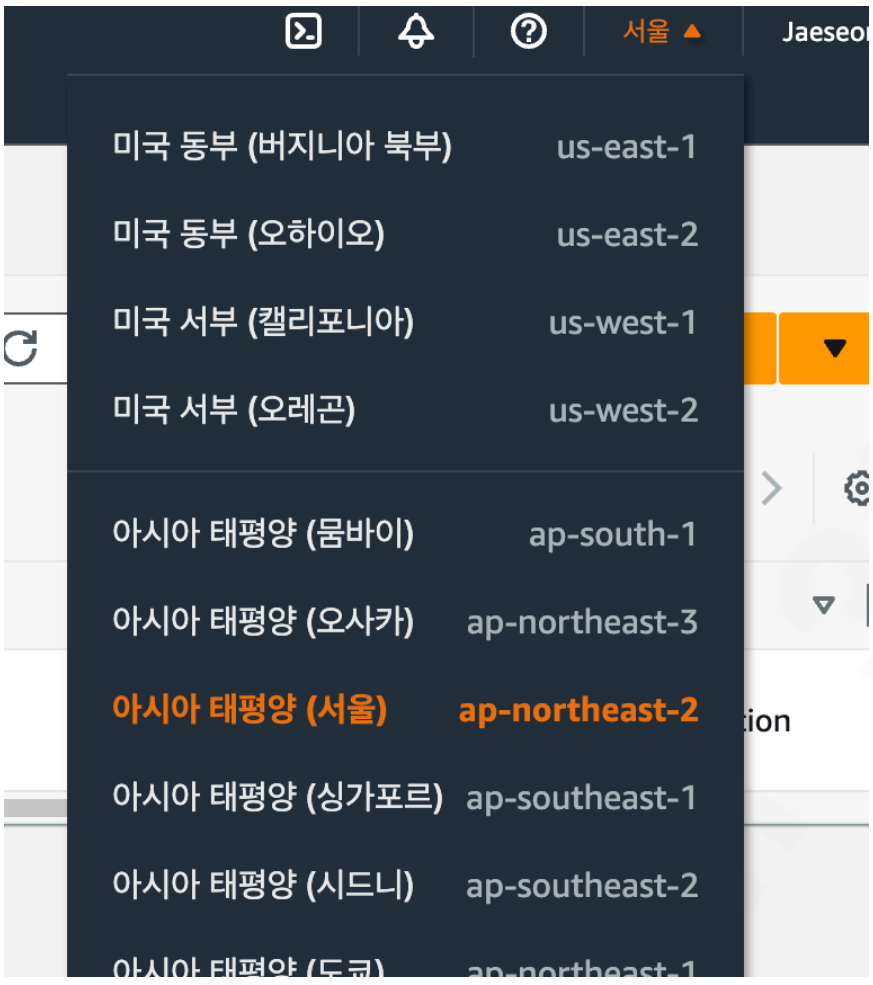
## ✅ AWS ELB 생성하기

1. **리전 선택하기**

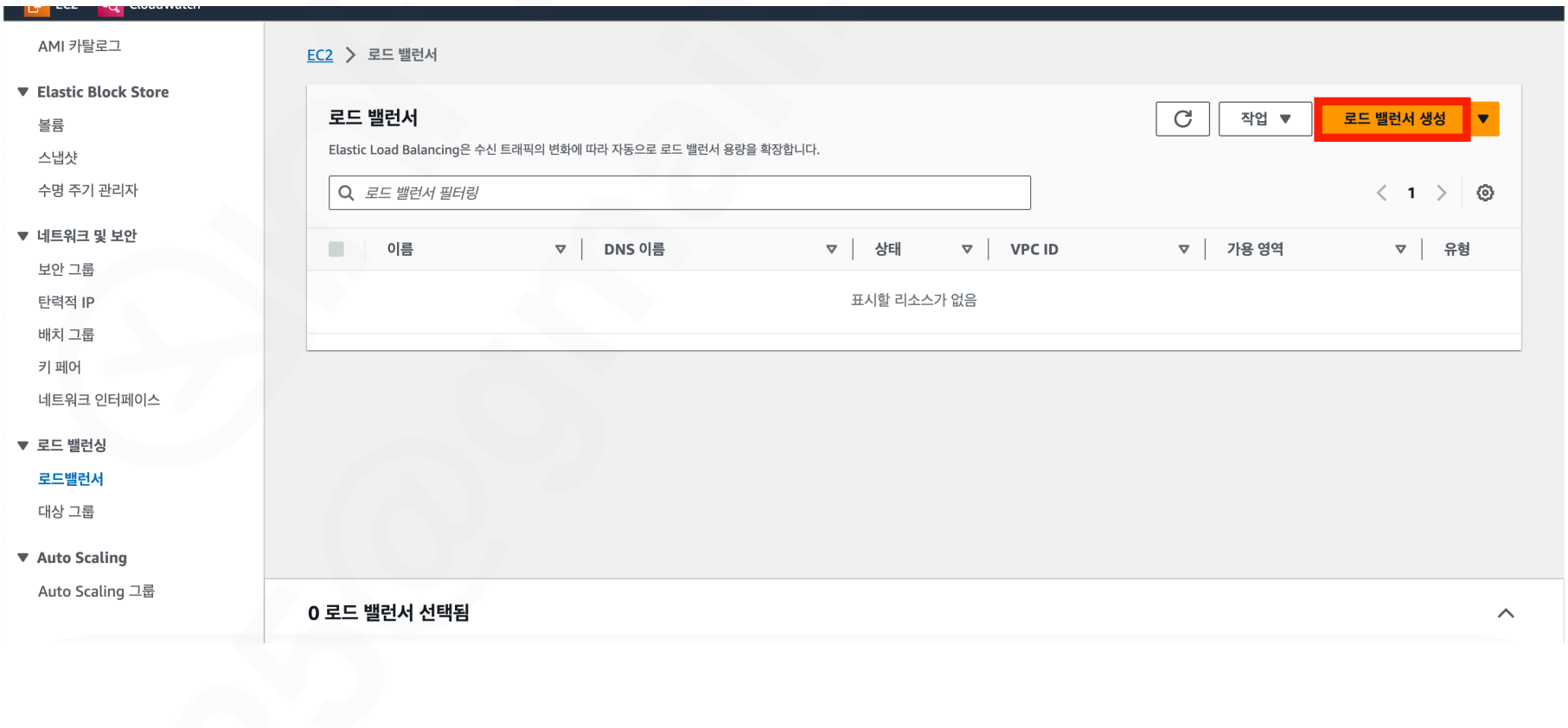
   AWS EC2 로드밸런서 서비스로 들어가서 리전(Region)을 선택해야 한다.




khb3025@gmail.com

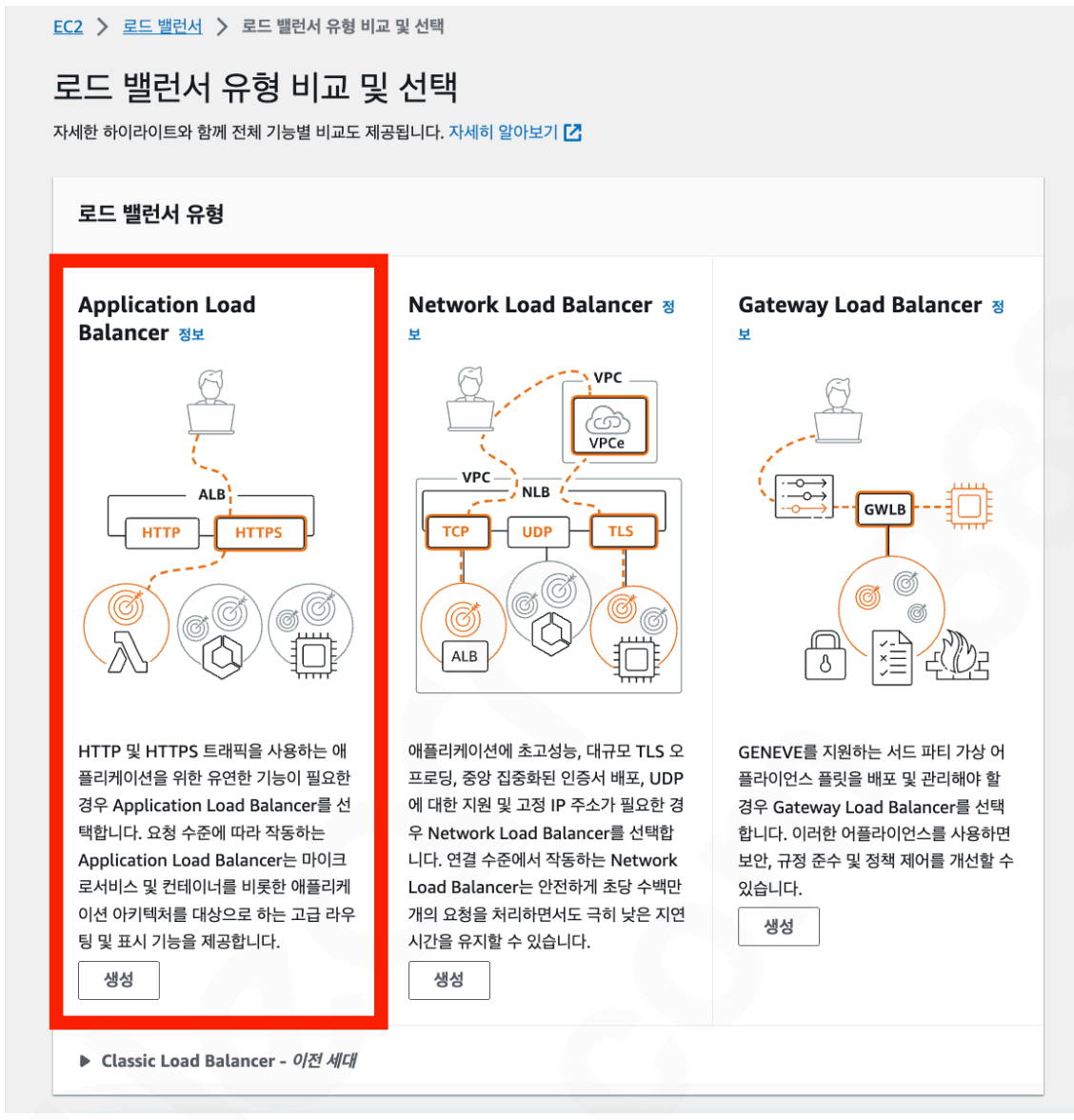



## 2. 로드 밸런서 생성하기

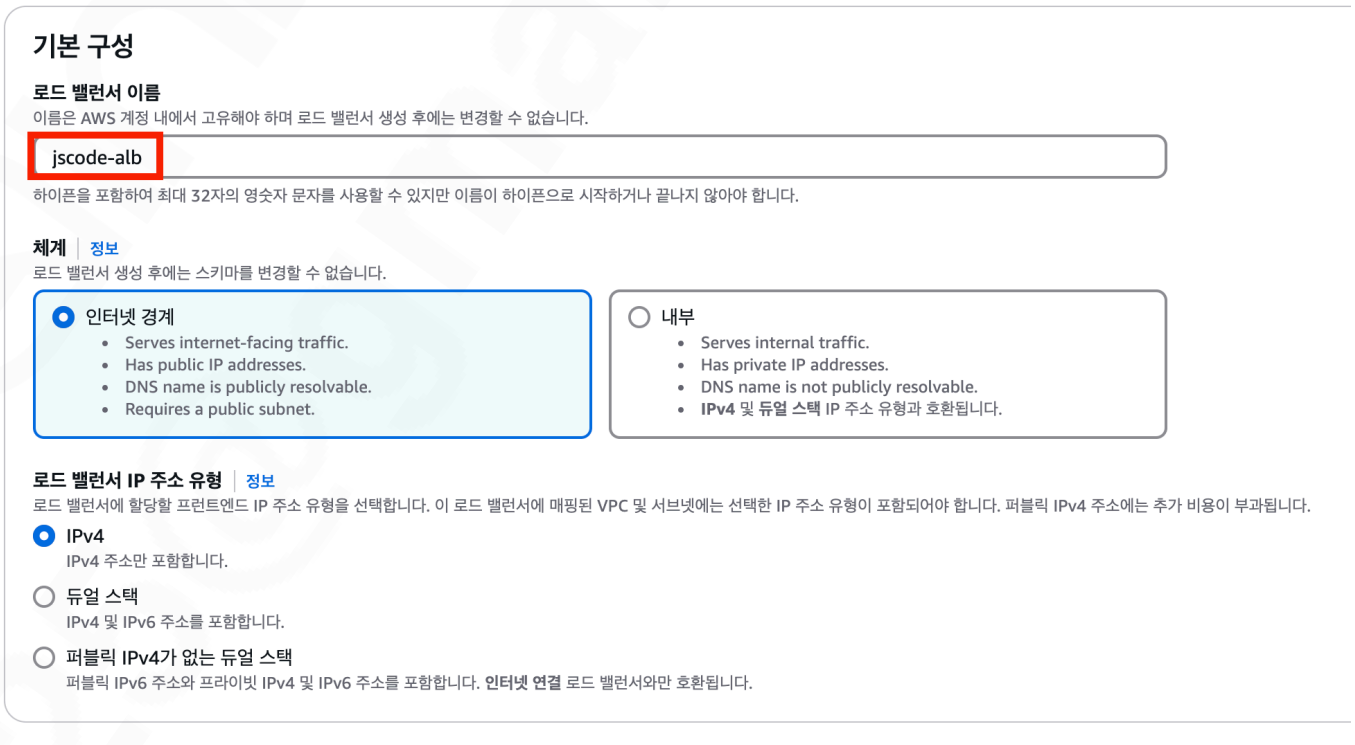



## 3. 로드 밸런서 유형 선택하기

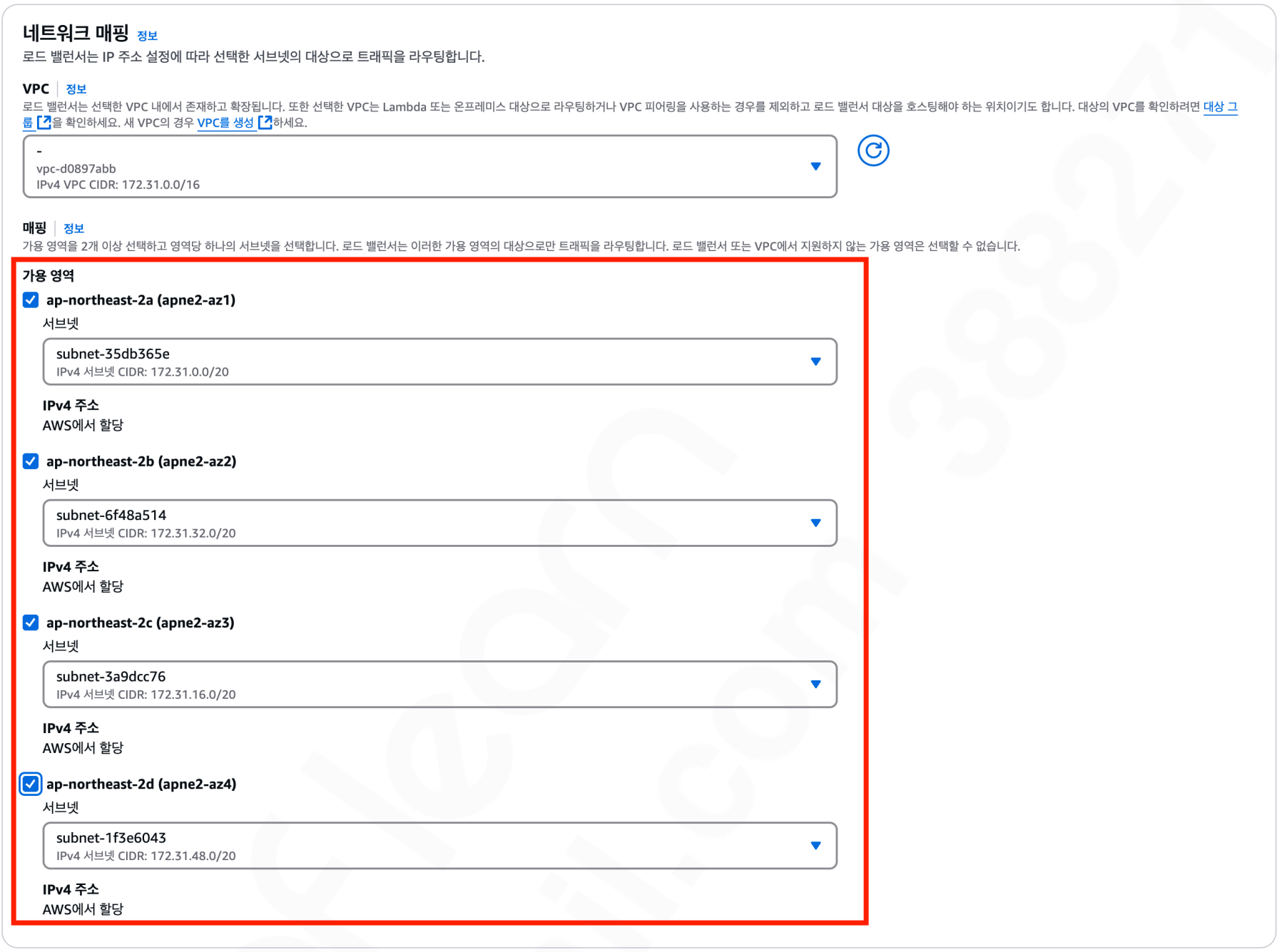
3가지 로드 밸런서 유형 중 Application Load Balancer(ALB)를 선택하면 된다.




## 4. 기본 구성 설정하기

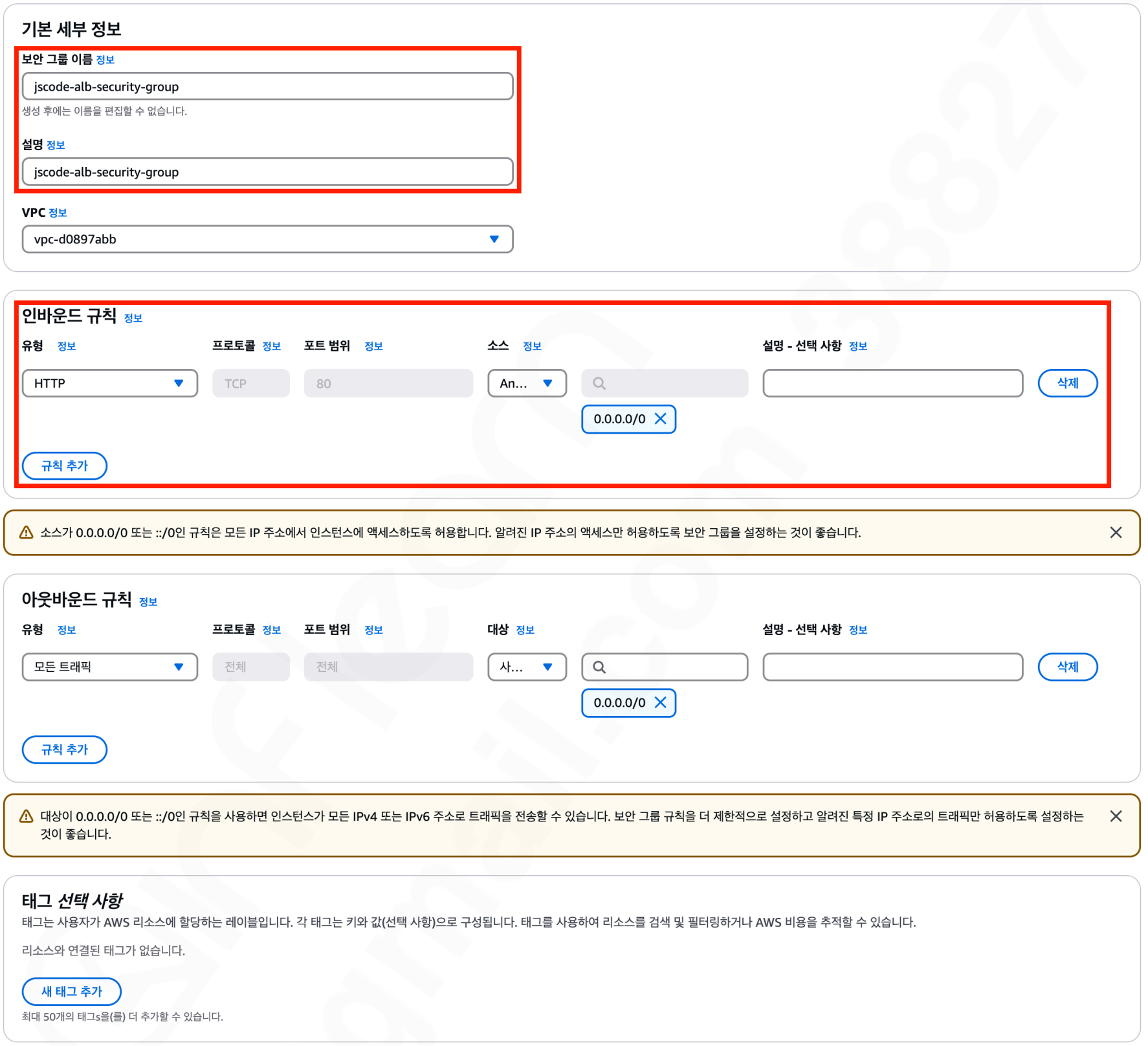



## 5. 네트워크 매핑 설정하기

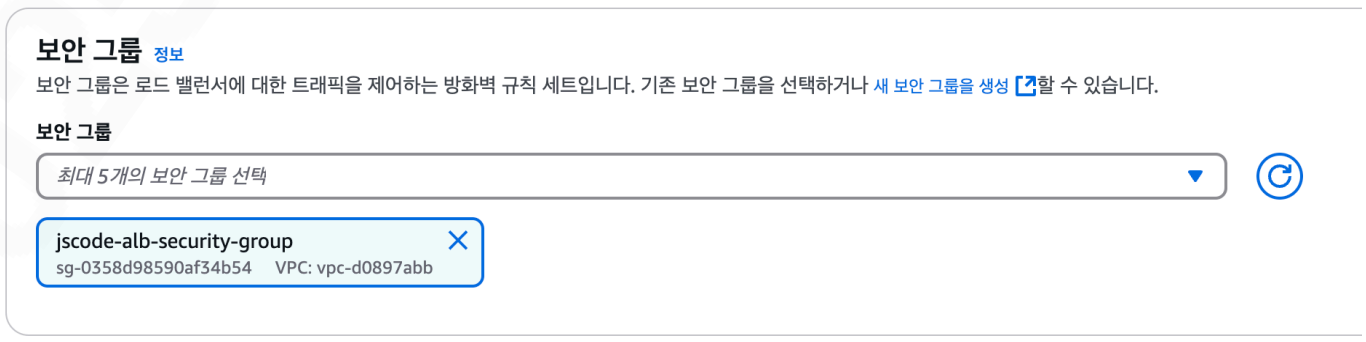



6. **AWS EC2 페이지의 보안 그룹 에서 보안 그룹 생성하기**

khb3025@gmail.com




7. **ELB 만드는 창으로 돌아와서 보안 그룹 등록하기**

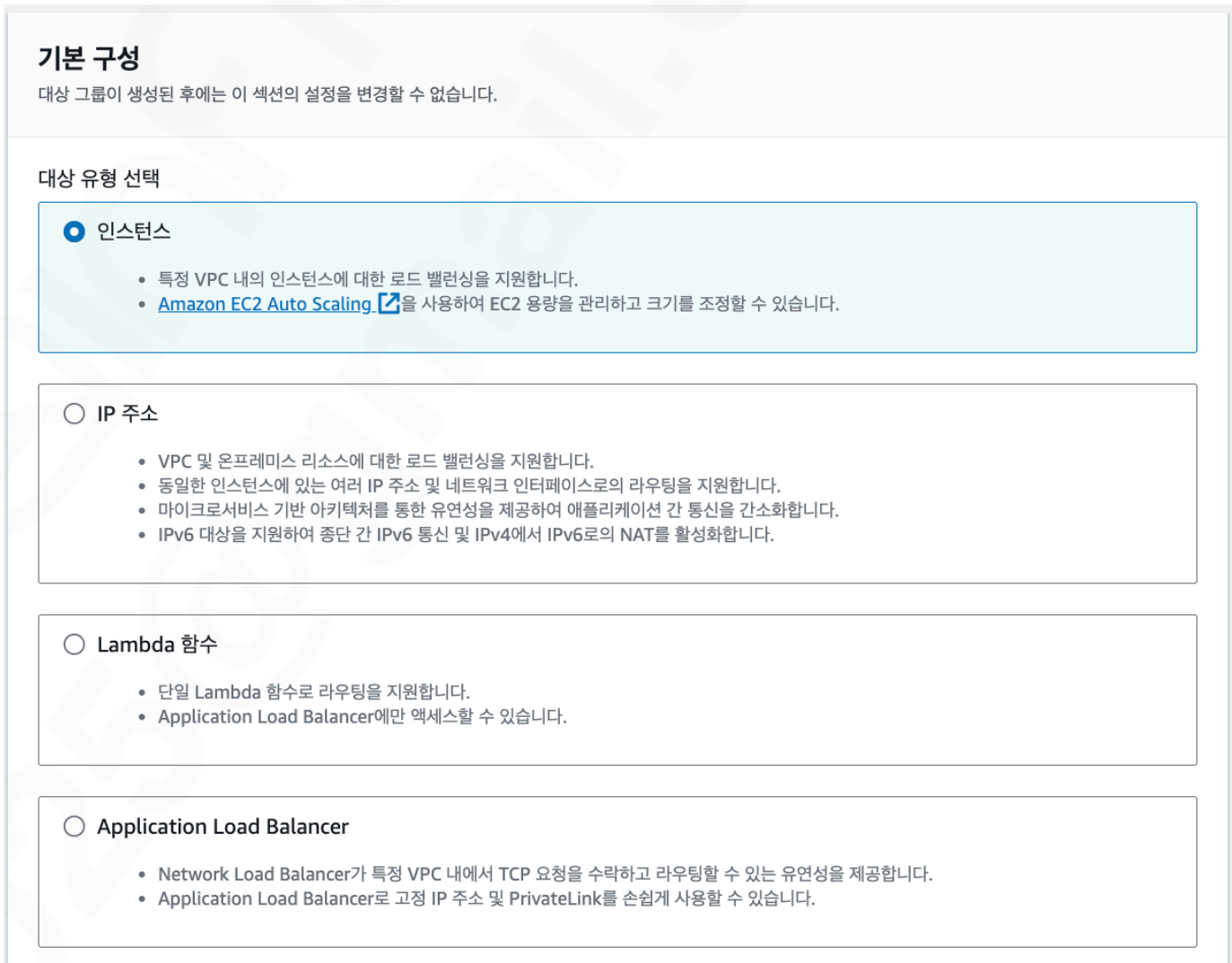



8. 대상 그룹**(Target Group)** 설정하기

   **리스너 및 라우팅** 설정은 ELB로 들어온 요청을 어떤 EC2 인스턴스에 전달할 건지를 설정하는 부분이다.

ELB로 들어온 요청을 **'어떤 곳'**으로 전달해야 하는데, 여기서 '어떤 곳'을 **대상 그룹(Target Group)**이라고 표현한다. 즉, ELB로 들어온 요청을 어디로 보낼 지 **대상 그룹**을 만들어야 한다.

9. **대상 유형 선택하기**



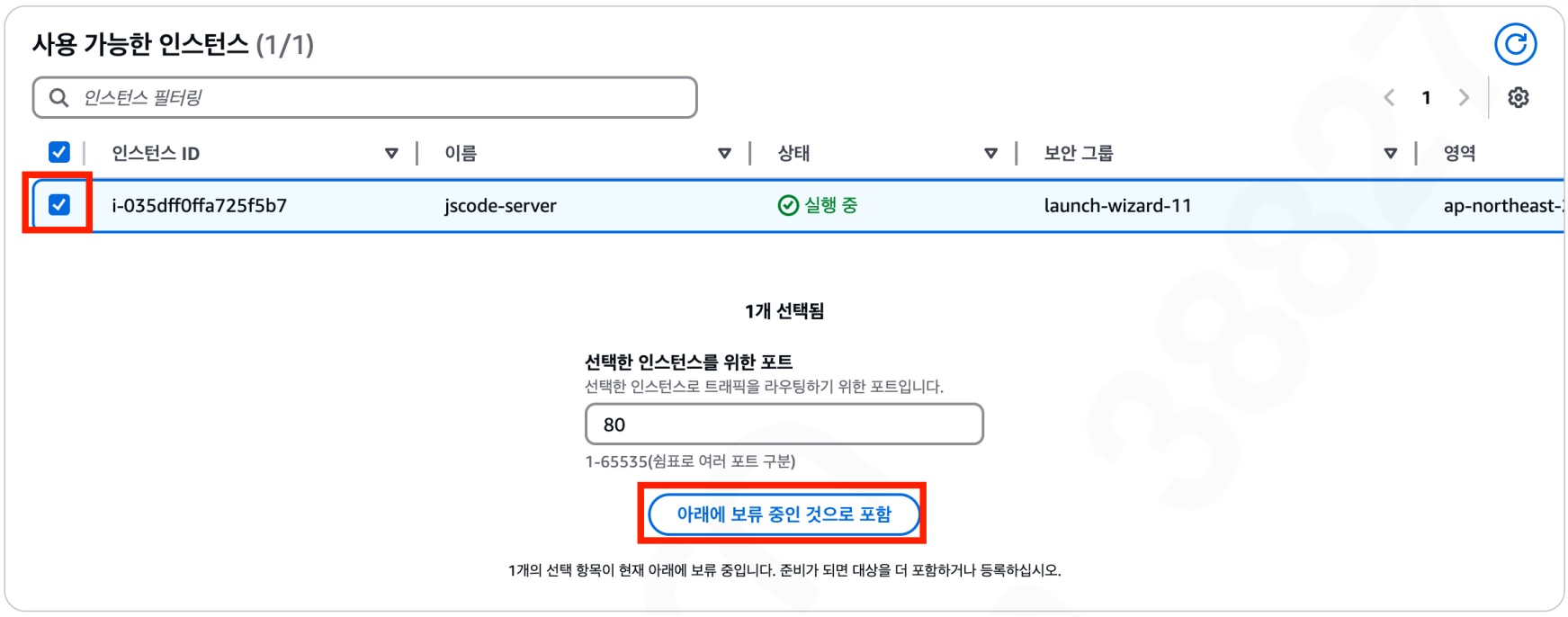

EC2에서 만든 특정 인스턴스로 트래픽을 전달할 것이기 때문에 **인스턴스** 옵션을 선택한다.

10. **프토토콜, IP 주소 유형, 프로토콜 버전 설정**

ELB가 사용자로부터 트래픽을 받아 대상 그룹에게 어떤 방식으로 전달할 지 설정하는 부분이다. 위 그림은 **HTTP(HTTP1), 80번 포트, IPv4 주소**로 통신을 한다는 걸 뜻한다.

11. **상태 검사 설정하기**

5. **대상 등록하기**




6. **ELB 만드는 창으로 돌아와서 대상 그룹(Target Group) 등록하기**

위 설정을 해석하자면 ELB에 **HTTP를 활용해 80번 포트로 들어온 요청(트래픽)**을 설정한 **대상 그룹**으로 전달하겠다는 의미이다.

7. **로드 밸런서 생성하기**

나머지 옵션들은 그대로 두고 로드 밸런서를 생성하면 된다.

키 | 값 | 제거

새 태그 추가

최대 49개의 태그를 추가할 수 있습니다.

## 요약

구성을 검토하고 확인합니다. 비용 예상

### 기본 구성 편집

practice

- 인터넷 경계
- IPv4

### 보안 그룹 편집

- instgram-elb-sg
  sg-017032463f0c448d5

### 네트워크 매핑 편집

VPC vpc-0b80d71048ae406dd

- ap-northeast-2a
  subnet-0ee07996981dd9ed4
- ap-northeast-2b
  subnet-02c9c9ad11a33fa1d
- ap-northeast-2c
  subnet-045c57a59ff4431be
- ap-northeast-2d
  subnet-0dc902977bf0ee676

### 리스너 및 라우팅 편집

- HTTP:80 기본값:
  Practice-taget-group

### 추가 서비스 편집

*없음*

### 태그 편집

*없음*

### 속성

특정 기본 속성이 로드 밸런서에 적용됩니다. 로드 밸런서를 생성한 후 해당 속성을 보고 편집할 수 있습니다.

취소 | 로드 밸런서 생성

khb302@gmail.com

# 🎯 [실습] EC2에 백엔드 서버(Spring Boot) 셋팅하기

## ✅ EC2에 Spring Boot 프로젝트 셋팅하기

1. **JDK 설치하기**

   Spring Boot는 3.x.x 버전을 사용할 예정이고, JDK는 17버전을 사용할 예정이다. 그에 맞게 환경을 설치해보자.

   ```
   $ sudo apt update
   $ sudo apt install openjdk-17-jdk
   ```

2. **잘 설치됐는 지 확인하기**

   ```
   $ java -version
   ```

   ```
   ubuntu@ip-172-31-40-248:~$ java -version
   openjdk version "17.0.8.1" 2023-08-24
   OpenJDK Runtime Environment (build 17.0.8.1+1-Ubuntu-0ubuntu122.04)
   OpenJDK 64-Bit Server VM (build 17.0.8.1+1-Ubuntu-0ubuntu122.04, mixed mode, sharing)
   ```

3. **Spring Boot 프로젝트 클론 받기**

   > 편의를 위해 로컬 환경에서 Spring Boot 프로젝트를 구현했다고 가정하고, 완성된 코드를 EC2에 클론 받자.

   ```
   $ git clone https://github.com/JSCODE-COURSE/load-testing-server.git
   ```

4. **RDS 정보 입력하기**

   **~/load-testing-server/src/main/resources/application.yml**

   ```
   server:
     port: 80
   spring:
     datasource:
       url: jdbc:mysql://{RDS 주소}:3306/jscode
       username: admin
       password: password
       driver-class-name: com.mysql.cj.jdbc.Driver
     jpa:
       hibernate:
         ddl-auto: create
   ```

3. **코드 살펴보기**

   - JPA를 활용해 DB와 연동하고 있는 상태
   - `/health GET` : ELB의 Health Check용 API
   - `/boards GET` : 게시글 최신순으로 조회 API
   - `/boards POST` : 게시글 작성 API

- 80번 포트에서 서버를 실행
- 편의상 `application.yml` 파일을 `.gitignore`에 추가하지 않음
  - 개인 프로젝트 또는 현업에서는 민감한 파일은 반드시 .gitignore에 추가해 Github Repository에 올라가지 않도록 해야 한다.

## ✅ Spring Boot 서버 실행시키기

1. **Spring Boot 서버 빌드 및 실행**

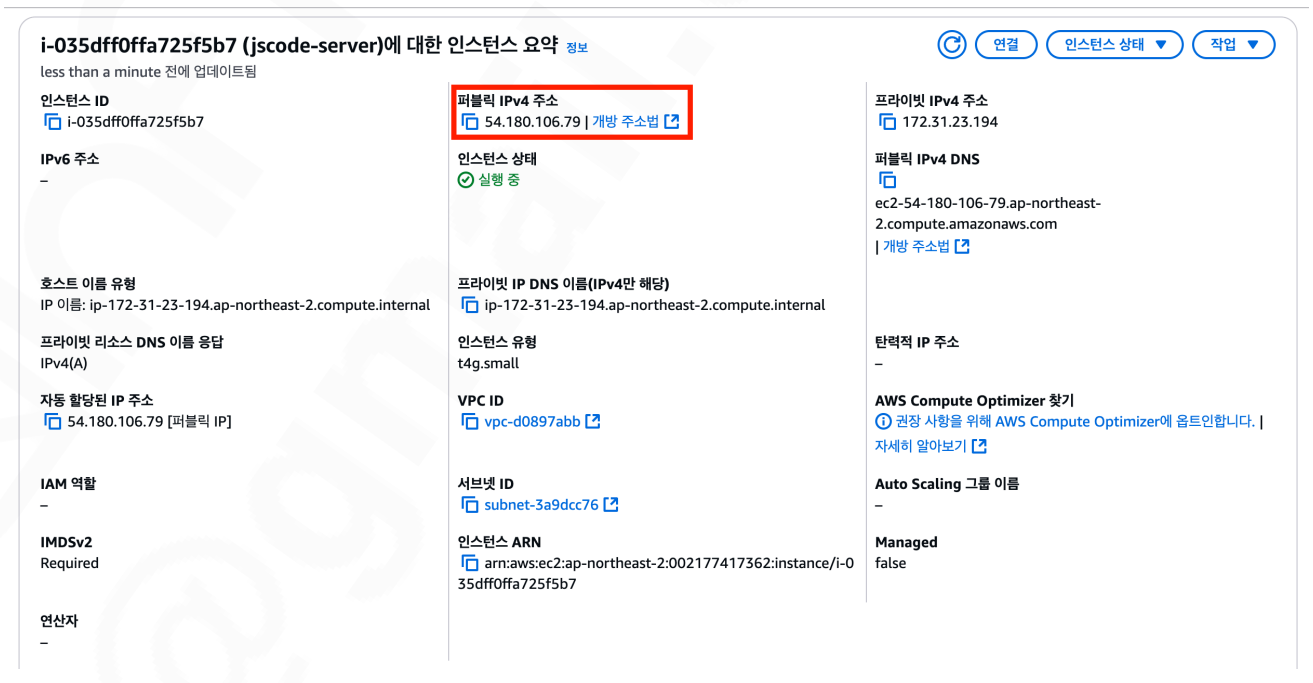
```
$ cd ~/load-testing-server
$ ./gradlew clean build -x test
$ cd build/libs
$ sudo nohup java -jar jscode-0.0.1-SNAPSHOT.jar &
```

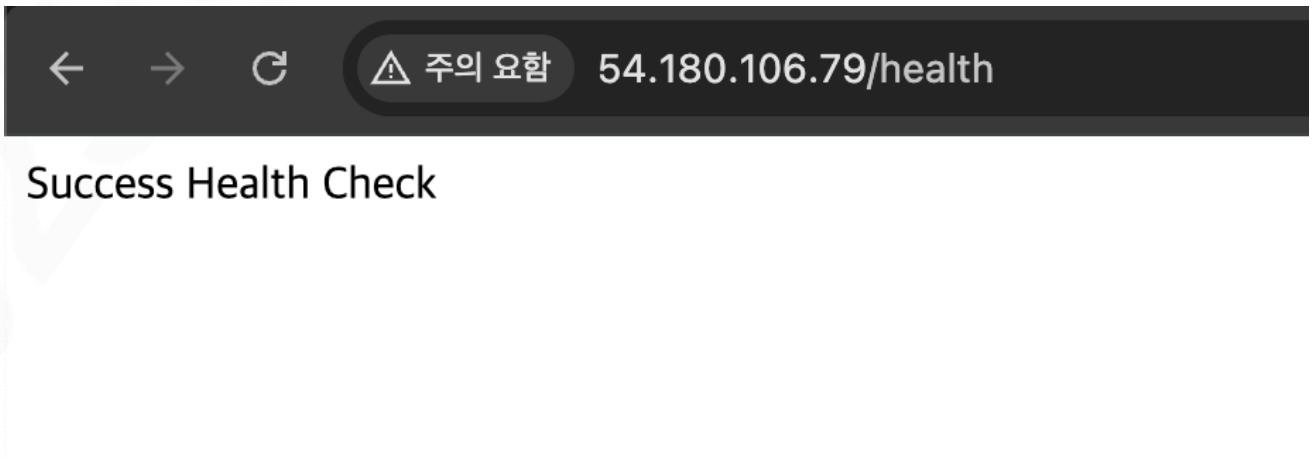
2. **서버 잘 실행됐는 지 확인**

a. 80번 포트에서 실행되고 있는 프로세스 확인

```
$ sudo lsof -i:80
```

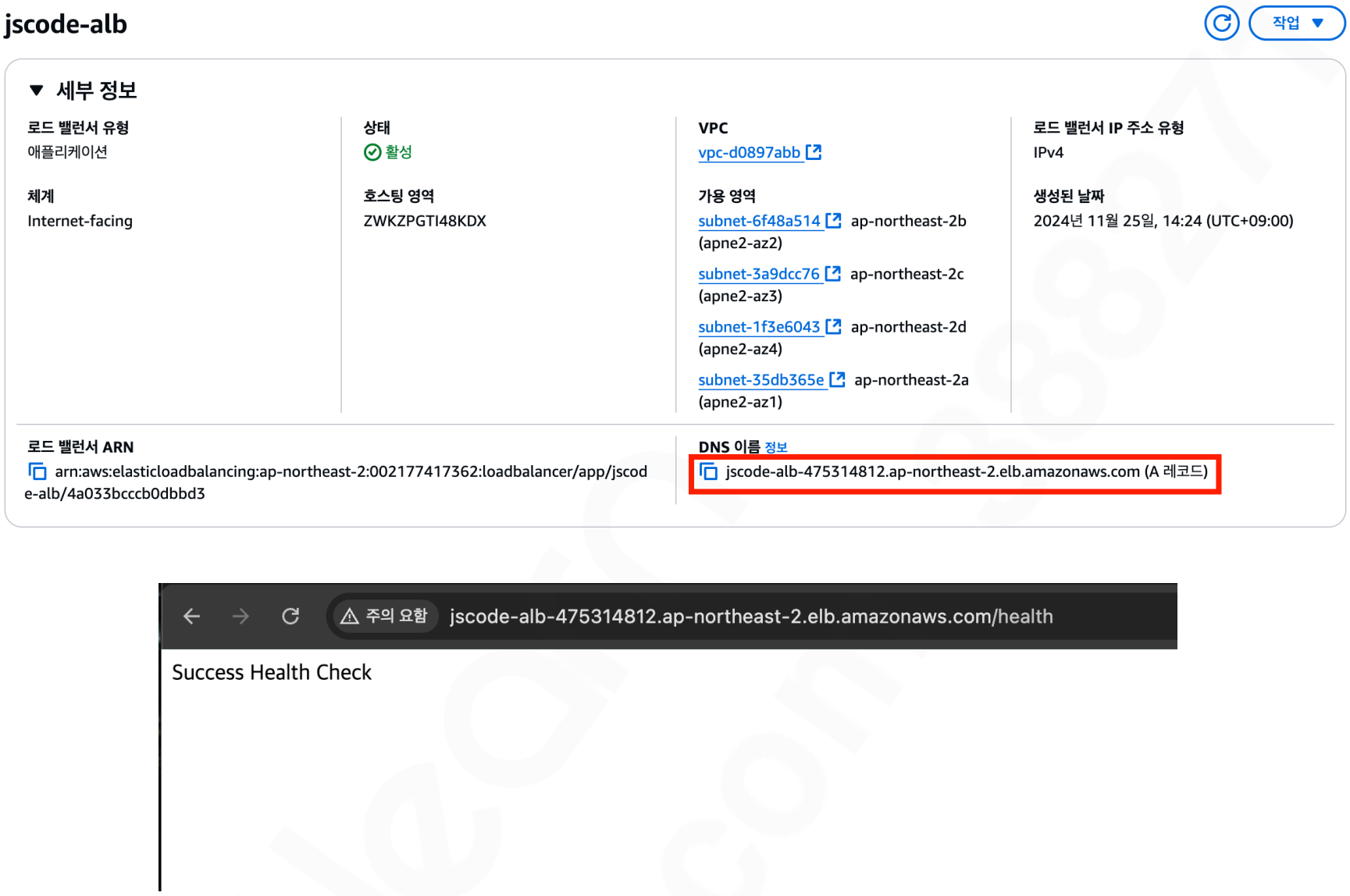
b. **EC2 IP 주소로 접속해보기**




**주의)** HTTPS 설정을 하지 않은 상태이기 때문에 `https://`가 아닌 `http://{EC2 IP 주소}/health`로 접속해야 한다.

c. **ELB 주소로 접속해보기**

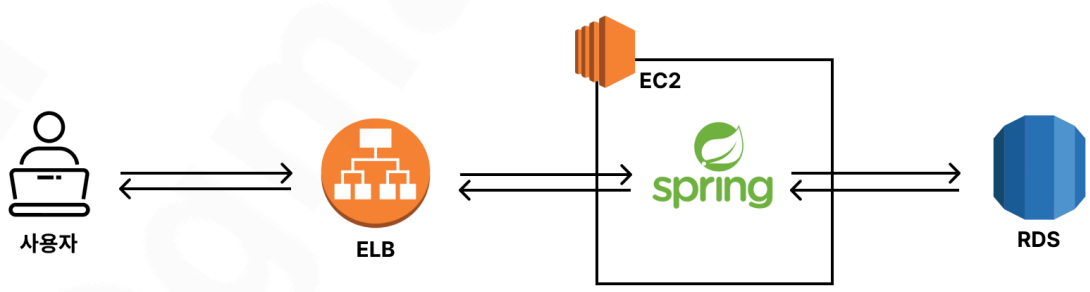






**주의)** HTTPS 설정을 하지 않은 상태이기 때문에 `https://` 가 아닌 `http://{ELB 주소}/health` 로 접속해야 한다.

d. **인프라 그림 다시 살펴보기**

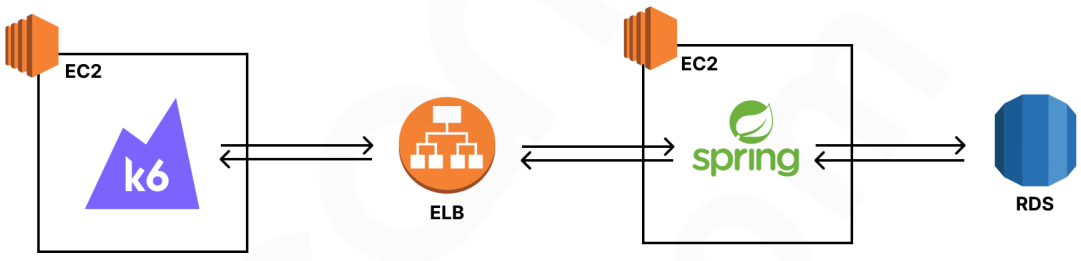



khb3025@gmail.com

# 🎯 [실습] 내가 구성한 인프라는 1초당 몇 개의 요청을 견뎌낼 수 있을까?

## ✅ 부하 테스트 할 인프라 구성

👩🏻‍🏫 부하 테스트를 하기 전에 인프라의 전체 구성을 그림으로 그려놓으면, 전체적인 트래픽의 흐름을 한 눈에 파악할 수 있어서 편리하다.




## ✅ k6를 활용해 부하 테스트 진행하기

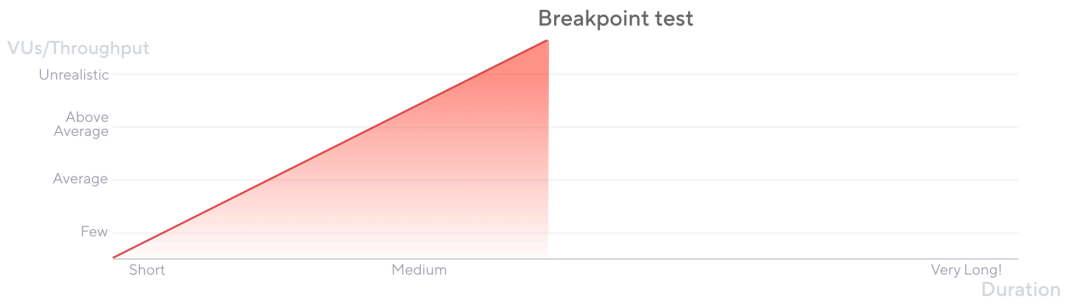
1. **프로덕션 환경과 비슷하게 구성하기 위해 더미 데이터 넣기**

   DB는 데이터가 얼마나 들어있느냐에 따라 성능 차이가 많이 난다. 따라서 정확한 부하 테스트를 하기 위해서는 프로덕션 환경의 DB에 들어있는 데이터와 비슷하게 구성하는 것이 좋다. 기존 프로덕션 환경에 100만개의 게시글 데이터가 들어있다고 가정하자.

```
-- 높은 재귀(반복) 횟수를 허용하도록 설정
-- (아래에서 생성할 더미 데이터의 개수와 맞춰서 작성하면 된다.)
SET SESSION cte_max_recursion_depth = 1000000;

-- 더미 데이터 삽입 쿼리
INSERT INTO boards (content, title, created_at)
WITH RECURSIVE cte (n) AS
(
  SELECT 1
  UNION ALL
  SELECT n + 1 FROM cte WHERE n < 1000000 -- 생성하고 싶은 더미 데이터의 개수
)
SELECT
    CONCAT('Content', LPAD(n, 7, '0')), -- 'Content' 다음에 7자리 숫자로 구성된 이름 생성
    CONCAT('Title', LPAD(n, 7, '0')), -- 'Title' 다음에 7자리 숫자로 구성된 이름 생성
    TIMESTAMP(DATE_SUB(NOW(), INTERVAL FLOOR(RAND() * 3650) DAY) + INTERVAL FLOOR(RAND() * 86400) SEC
FROM cte;

-- 잘 생성됐는 지 확인
SELECT COUNT(*) FROM boards;
```

2. **부하를 주기 위해 k6 스크립트 작성**

   > 구성한 시스템이 1초당 몇 개의 요청을 견딜 수 있는 지 알아보려면, 점진적으로 트래픽을 늘려가게끔 부하 테스트를 셋팅해야 한다.




> **실제 프로덕션 환경에서 발생하는 트래픽과 최대한 비슷하게 부하를 만들어서 테스트 해야한다.** 실제 서비스에서 100명의 사용자 중 95명이 게시글 조회( `GET /boards` )를 하고, 5명이 게시글 작성( `POST /boards` )을 한다고 가정하자. 실제 프로덕션 환경과 유사하게 트래픽을 만들어낼 수 있게 k6 코드를 작성해보자.

**script.js**

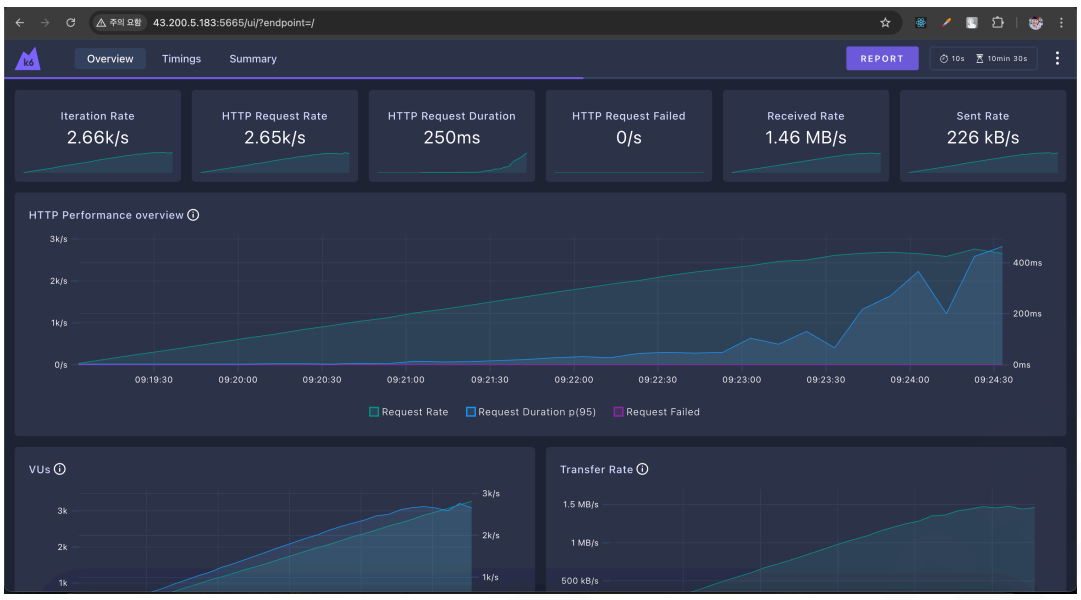
```
import http from 'k6/http';
import { sleep } from 'k6';

export const options = {
  // 부하를 생성하는 단계(stages)를 설정
  stages: [
    // 10분에 걸쳐 vus(virtual users, 가상 유저수)가 50에 도달하도록 설정
    { duration: '10m', target: 50 }
  ],
};

export default function () {
  let random = Math.random();
  // 100명 중 5명의 비율로 게시글을 작성
  if (random < 0.05) {
    const data = { title: '제목', content: '내용' };
    http.post('http://{ELB 주소}/boards', JSON.stringify(data), {
      headers: { 'Content-Type': 'application/json' },
    });

  // 100명 중 95명의 비율로 게시글을 조회
  } else {
    http.get('http://{ELB 주소}/boards');
  }

  // 1초 휴식
  sleep(1);
}
```

2. **부하 테스트 시작하기**

```
$ K6_WEB_DASHBOARD=true k6 run script.js
```

3. **k6의 웹 대시보드 열기**

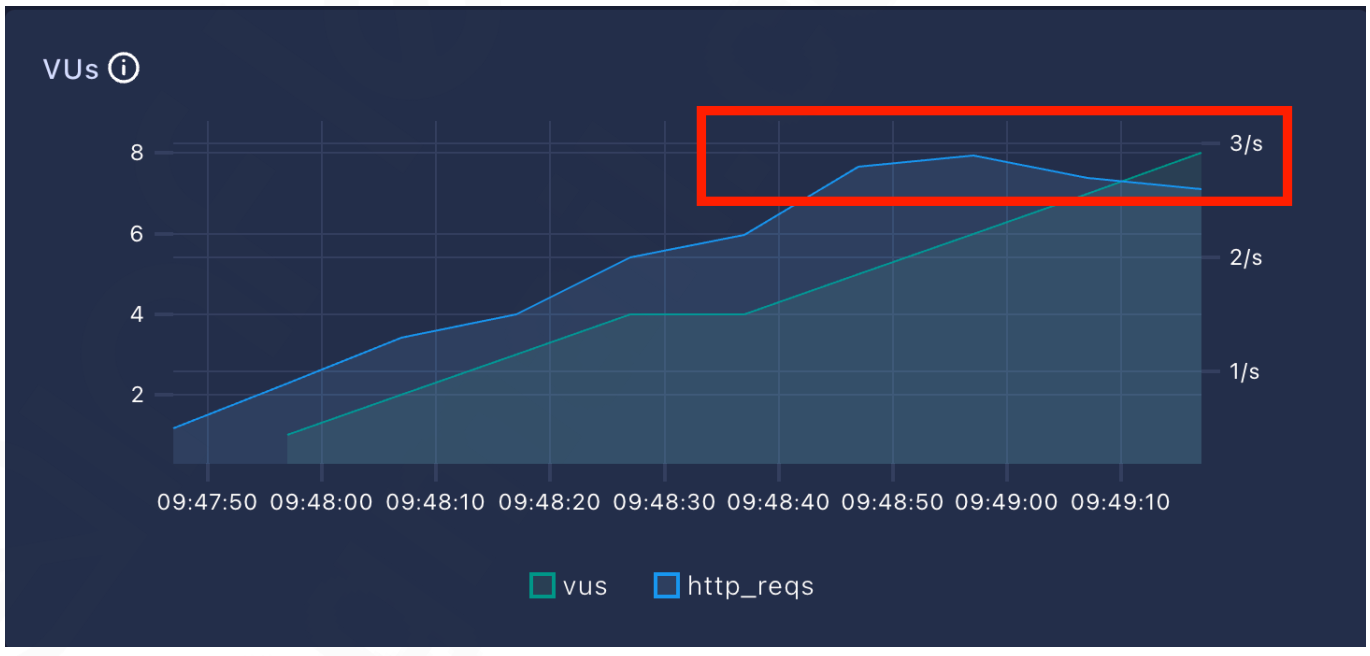
`http://{k6가 실행되고 있는 EC2 IP 주소}:5665` 으로 접속하기




## ✅ 부하테스트 결과 해석

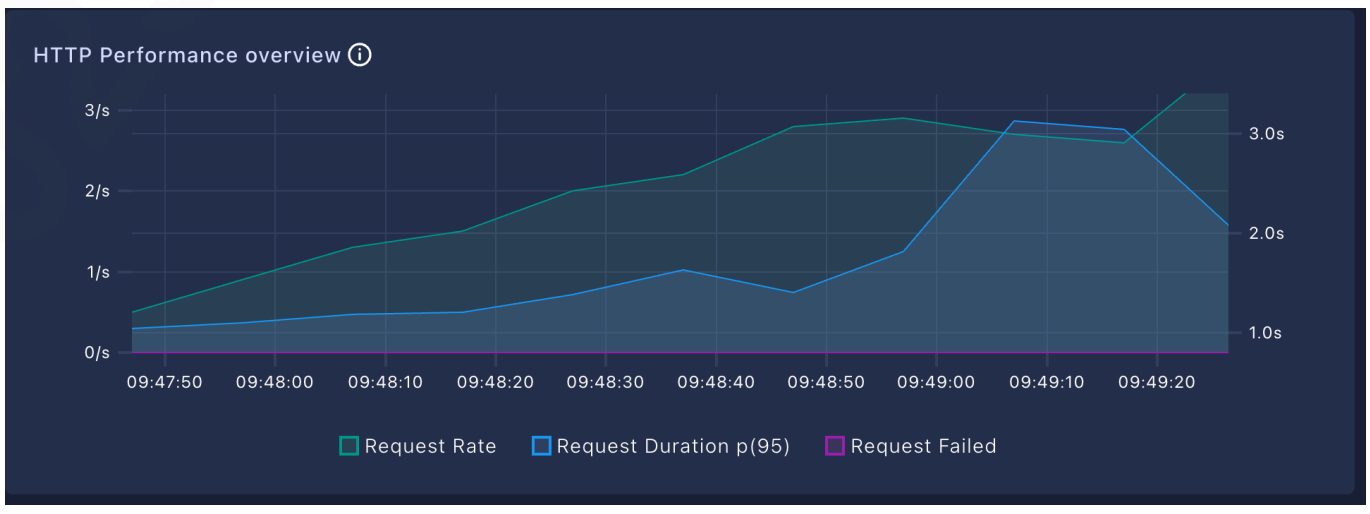
1. **VUs가 늘어나도 HTTP Request Rate가 더 이상 증가하지 않는 구간을 찾는다.**

   더 이상 증가하지 않는 HTTP Request Rate가 현재 시스템의 최대 Throughput이다. 초당 3개 정도의 요청을 처리하고 있다. 즉, 현재 시스템은 1초당 3개의 요청을 처리할 수 있다는 뜻이다. 이걸 보고 '**현재 시스템의 최대 Throughput은 3 TPS**'라고 표현한다.




2. **HTTP Request Duration이 비이상적으로 높은 건 아닌지 체크한다.**

   요청 당 응답 시간이(Latency)이 최대 TPS를 넘은 이후로부터 점점 응답 시간이 길어지는 걸 확인할 수 있다. 하지만 최대 TPS에 도달하기 전에는 1초 미만의 Latency인 걸로 봐서 크게 문제가 없는 걸로 판단된다.




**참고)** VUs가 높아짐에 따라 서버가 처리할 수 있는 요청량이 밀려서 대기하는 시간이 늘어나기 때문에, HTTP Request Rate(= Throughput)는 늘어나지 않는데 Request Duration(Latency)는 늘어나는 현상을 확인할 수 있다.

3. **HTTP Request Failed가 있는 건 아닌지 체크한다.**

   만약 실패한 요청이 있다면 요청이 왜 실패했는 지 분석해야 한다.

👩🏻‍🏫 부하 테스트를 통해 현재 구성한 인프라가 1초당 몇 개의 요청을 견뎌내는 지 알아봤다. 여기서 더 많은 요청을 견디게끔 성능을 개선하고 싶다고 가정해보자. 이 때, 로드밸런서(ELB)의 성능을 개선해야 하는 지, 백엔드 서버(EC2)의 성능을 개선해야 하는 지, DB(RDS)의 성능을 개선해야 하는 지 판단할 수 있어야 한다. 이걸 판단하려면 '병목 지점'을 찾아내야 한다. 다음 강의에서 '병목 지점'을 어떻게 찾아내는 지 알아보자.

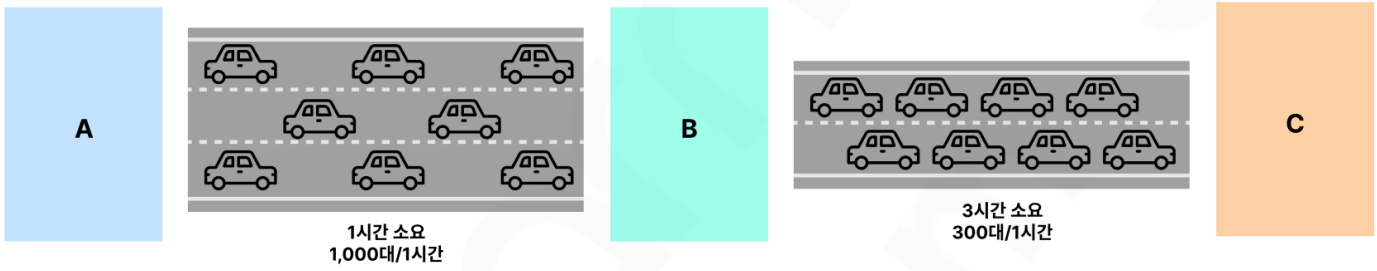
@kfc1cc@gmail.com
khb302500

# 모니터링(Monitoring), 메트릭(Metric)

## ✅ 모니터링(Monitoring)이란?

**모니터링(Monitoring)**이란 **어떤 대상을 지속적으로 감시**한다는 뜻이다.

### 부하 테스트에서 모니터링(Monitoring)은 왜 필요할까?




부하 테스트 과정 중에 성능을 개선하려면 **병목 지점**을 먼저 파악해야 한다고 했다. 병목 지점을 파악하기 위해서는 각 **'컴퓨터'**의 **CPU 사용률**과 **메모리 사용률**과 같은 값들을 지속적으로 확인할 수 있어야 한다.

여기서 얘기하는 **컴퓨터**는 로드 밸런서(ELB), 백엔드 서버(EC2), DB(RDS), 캐시(ElastiCache) 등을 얘기한다. 무언가 요청을 받아서 처리하는 형태의 서비스는 전부 컴퓨터에서 처리하기 때문이다.

컴퓨터가 무언가를 처리하려면 **CPU**와 **메모리** 같은 자원이 필요하다. 만약 컴퓨터가 처리할 수 있는 최대한의 요청을 처리하고 있다면, CPU 사용률 또는 메모리 사용률과 같은 값이 100%일 것이다. 여기서 **특정 자원 사용률이 100%인 컴퓨터를 보고 병목 지점이라고 판단한다.**

위 내용을 정리해보면, **병목 지점을 파악하기 위해서 부하 테스트를 하면서 컴퓨터의 자원(CPU, 메모리 등)을 모니터링(= 어떤 대상을 지속적으로 감시)해야 한다.**

(글로만 봤을 때는 무슨 말인지 이해하기 어려울 수 있다. 실습을 해보면 쉽게 이해할 수 있으니, 지금 이해가 안 되더라도 크게 걱정하지 말자.)

## ✅ 메트릭(Metric)이란?

**메트릭(Metric)**이란 **수치 데이터** 또는 **측정값**을 의미한다. 모니터링을 통해 측정하는 **CPU, 메모리** 등의 값을 전부 메트릭(Metric)이라고 부른다.

khb3025@gmail.com 1388271

📖

# CPU, 메모리(Memory), 디스크(Disk)

## ✅ CPU, 메모리(Memory), 디스크(Disk) 의미

**[CPU]**

- 명령어(코드)를 해석하고 실행
- 연산 작업

**[메모리(Memory)]**

- CPU가 작업하기 위해 임시로 사용하는 공간
  - 디스크보다 메모리로부터 데이터를 가져올 때 속도가 훨씬 빠르기 때문이다.
  - CPU는 명령어를 해석해서 실행하거나 연산 작업을 할 때는 무조건 메모리를 통해서만 데이터를 가져온다. 디스크에 직접 접근하지 않는다.
- 실행할 프로그램의 코드, 변수(전역 변수, 매개 변수...)
- RAM이라 부르기도 한다.

**[디스크(Disk)]**

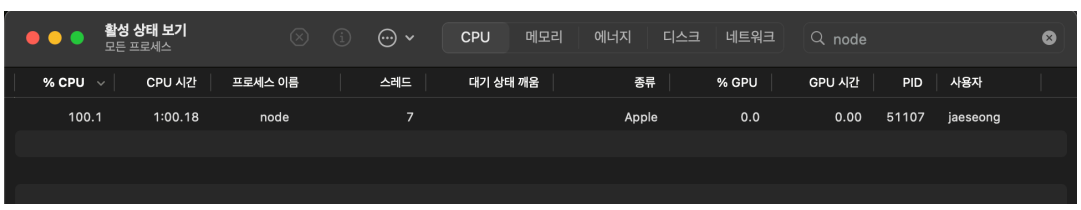
- C드라이브와 같은 **컴퓨터 저장 공간**
- **영구적**으로 데이터, 파일 등을 저장
- 데이터를 가져올 때 속도가 상대적으로 느림

## ✅ CPU, 메모리(Memory)의 직관적인 이해

1. **CPU를 많이 사용하는 작업**

**app.js**

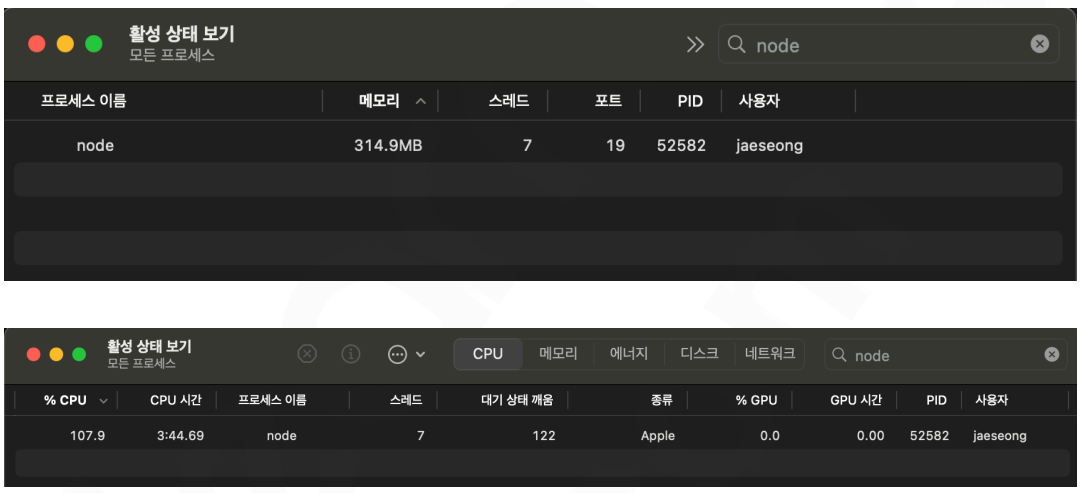
```
let num = 1;
while (true) {
  num += 1;
}
```



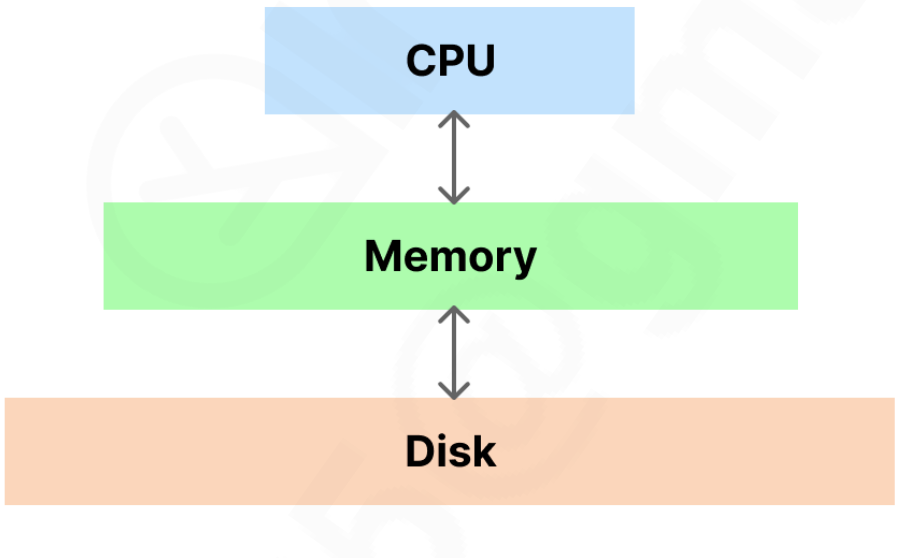
2. **메모리를 많이 사용하는 작업**

**app.js**

```
while (true) {
  let largeArray = [];
  for (let i = 0; i < 10000000; i++) {
    largeArray.push(i);
  }
}
```




## ✅ CPU, 메모리, 디스크의 직관적인 이해를 위한 연습




1. **DB는 CPU, 메모리, 디스크를 어떻게 사용할까?**
   - `디스크`로부터 데이터를 조회해서 `메모리`에 올림
   - `메모리`에서 연산, 필터링, 집계 등의 작업을 `CPU`가 처리함

   ⇒ DB의 특성상 많은 양의 데이터를 가지고 작업을 해야 하는 경우가 빈번해서,

   `CPU`, `메모리` 둘 다를 많이 사용하는 편이다.

2. **백엔드 서버는 CPU, 메모리, 디스크를 어떻게 사용할까?**
   - 백엔드 서버의 로직에 파일을 읽어오거나, 파일을 생성하는 로직이 없다면 디스크는 거의 사용할 일이 없다.
     - 파일 업로드나 파일 조회 같은 건 S3와 같은 서비스를 활용해 별도로 분리하는 추세다.
   - DB로부터 불러오는 데이터의 양이 그렇게 크지 않을 경우 `메모리`를 많이 사용할 일이 없다.
   - 많은 요청에 대해 로직을 처리해야 하기 때문에 `CPU`를 많이 사용하는 편이다.

- 복잡한 연산 작업(인코딩, 디코딩, 암호화 등)을 사용할 경우 `CPU`를 많이 사용한다.

3. **로드밸런서는 CPU, 메모리, 디스크를 어떻게 사용할까?**
   - 요청이 들어오면 적절하게 서버들한테 골고루 트래픽을 분배해주는 역할을 한다.
     - 임시로 저장해야 하는 데이터가 많은 게 아니기 때문에 `메모리`를 많이 사용할 일이 없다.
   - 많은 요청을 분배해주는 로직(= 연산)을 처리하기 때문에 `CPU`를 많이 사용한다.

4. **캐시(Redis)는 CPU, 메모리, 디스크를 어떻게 사용할까?**
   - 캐시의 특성상 빠르게 데이터를 조회해야 하기 때문에 디스크에 데이터를 저장하지 않고 `메모리`에 데이터를 저장해둔다.
     - 디스크보다 `메모리`가 데이터 접근 속도(읽기, 쓰기)가 훨씬 빠르기 때문이다.
   - 캐시의 주 역할은 데이터 조회다. 연산할만한 작업은 많이 없다. 따라서 `CPU`보다는 `메모리`를 많이 사용하는 편이다.

## ✅ Disk I/O, Network I/O란?

서버의 성능을 측정할 때 CPU, 메모리 이외에도 Disk I/O, Network I/O와 같은 다양한 값들이 있다. 하지만 입문자 입장에서는 너무 많은 메트릭을 한 번에 이해하려고 하면 어렵게 느껴진다. 파레토의 법칙에 따라 CPU, 메모리에 최대한 초점을 맞춰서 부하 테스트를 진행할 예정이다.

실제 AWS 클라우드를 사용하면 대부분 CPU, 메모리 부족으로 인해 병목 현상이 발생한다. 따라서 CPU, 메모리 이 2가지 메트릭만 가지고 부하 테스트 및 성능 개선을 연습해보자.

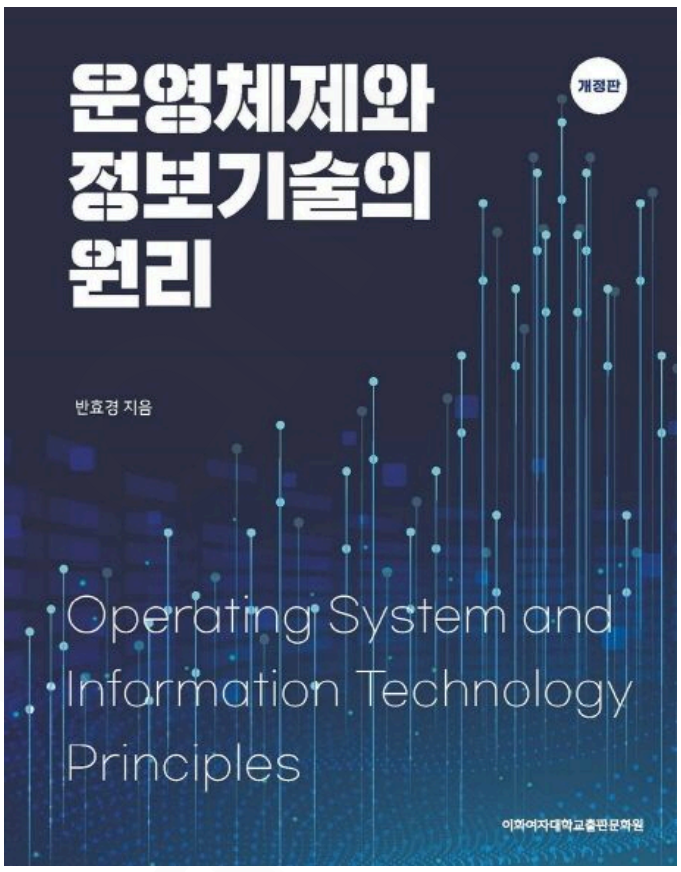
khb3025@gmail.com

CPU, 메모리, 디스크와 같은 장치에 대해 더 자세히 알고 싶다면 **운영체제(OS)**에 대해 추가적으로 학습해보는 걸 추천한다. 개발자가 필수적으로 공부해야 하는 과목 중 하나가 **운영체제(OS)**이다.

**[추천 책]**

product.kyobobook.co.kr

https://product.kyobobook.co.kr/detail/S000001772604




khb3025@gmail.com

# 🎯 [실습] EC2 모니터링 셋팅하기

## ✅ EC2가 수집하는 기본 지표들

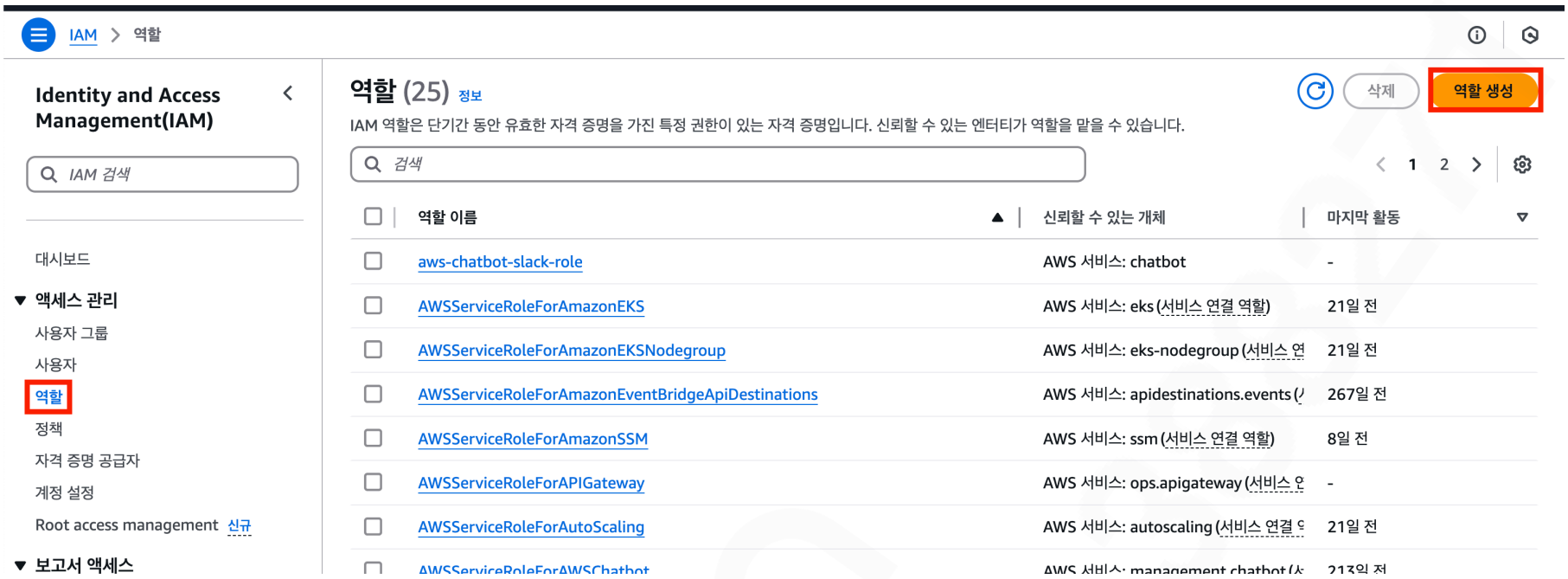
EC2 인스턴스를 생성하면 기본적으로 수집하는 지표들이 있다.

- CPU 사용률(CPUUtilization)
- 네트워크 사용률(NetworkIn, NetworkOut)
- 디스크 성능(DiskReadOps, DiskWriteOps)
- 디스크 읽기/쓰기(DiskReadBytes, DiskWriteBytes)
- 등등

단순히 인스턴스가 멈추지 않고 가동되고 있는지를 확인하기에는 위의 지표를 감시하는 것만으로 충분할 지도 모르지만, 확실한 모니터링을 위해서는 **메모리(Memory)**도 반드시 모니터링 할 필요가 있다. **CloudWatch Agent**를 인스턴스에 설치하고 구성 파일을 설정한 후 실행하면 메모리(Memory)를 포함한 더 많은 지표를 수집할 수 있다.

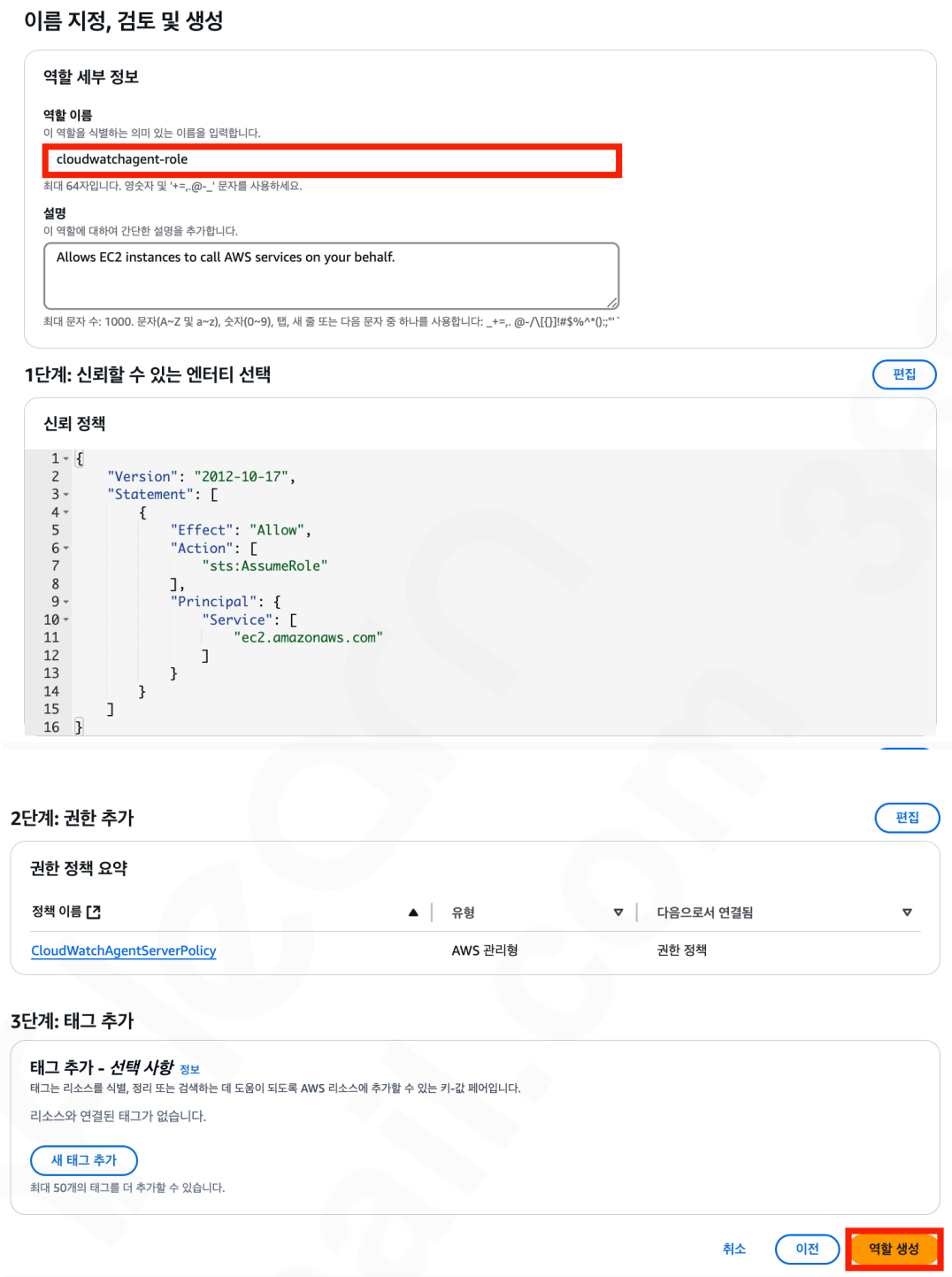
## ✅ EC2에 Cloudwatch Agent 설치하기

1. **IAM Role 생성**

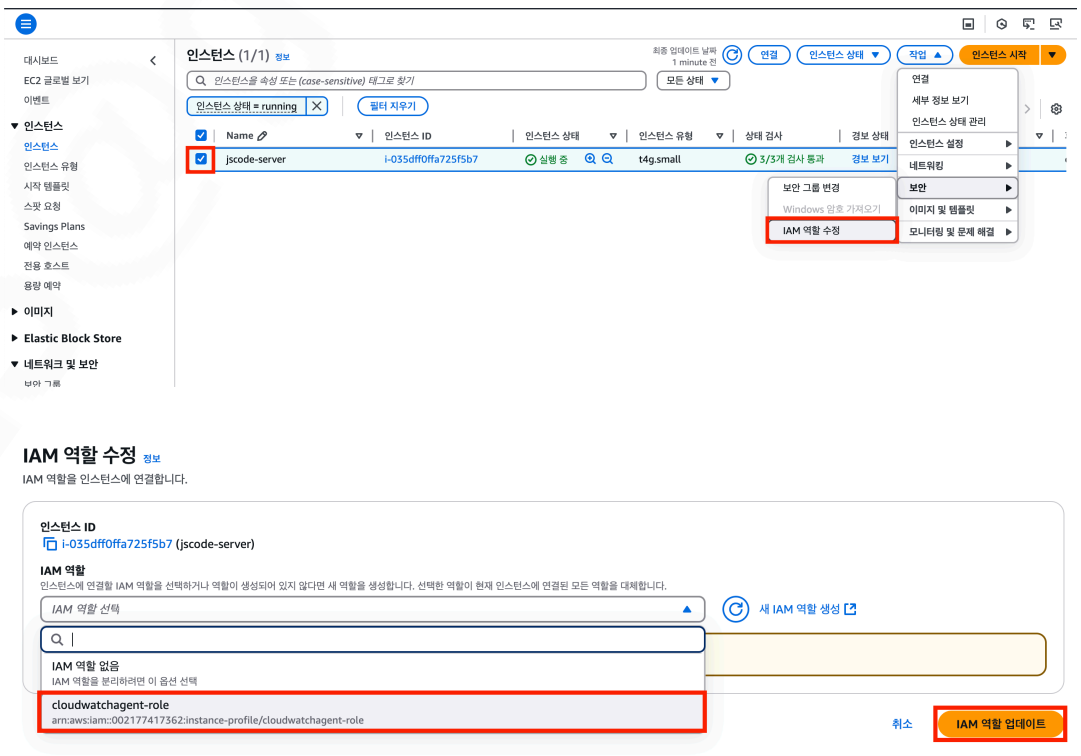


## 2. 정책 선택하기

## 3. 역할 이름 설정하기




## 4. EC2 인스턴스에 IAM Role 연결




## 5. EC2 인스턴스로 접속해 Cloudwatch Agent 설치하기

```
# Ubuntu x86-64 Cloudwatch Agent 다운로드
$ wget https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb
```

```
# 다운받은 패키지 설치
$ sudo dpkg -i -E ./amazon-cloudwatch-agent.deb
```

6. **CloudWatch Agent 설정 파일 생성하기**

```
$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard
```

```
================================================================
= Welcome to the Amazon CloudWatch Agent Configuration Manager =
=                                                              =
= CloudWatch Agent allows you to collect metrics and logs from =
= your host and send them to CloudWatch. Additional CloudWatch =
= charges may apply.                                           =
================================================================

# 사용하고 있는 운영체제는 무엇입니까?
On which OS are you planning to use the agent?
1. linux
2. windows
3. darwin
default choice: [1]:
1

# EC2 인스턴스 환경인지? 아니면 온프레미스 환경인지?
Trying to fetch the default region based on ec2 metadata...
I! imds retry client will retry 1 timesAre you using EC2 or On-Premises hosts?
1. EC2
2. On-Premises
default choice: [1]:
1

# 리눅스에서 어떤 user가 Cloudwatch Agent를 사용할 건지?
Which user are you planning to run the agent?
1. cwagent
2. root
3. others
default choice: [1]:
2

# StatsD 데몬(애플리케이션 지표를 수집)을 사용할 건지?
Do you want to turn on StatsD daemon?
1. yes
2. no
default choice: [1]:
2

# CollectD라는 시스템 메트릭 모니터링 도구를 사용할 건지?
Do you want to monitor metrics from CollectD? WARNING: CollectD must be installed or the Agent will fail to start
1. yes
2. no
default choice: [1]:
2

# 컴퓨터(host)의 메트릭(CPU, 메모리 등)을 모니터링 할 건지?
Do you want to monitor any host metrics? e.g. CPU, memory, etc.
1. yes
```

```
2. no
default choice: [1]:
1

# 코어별로 CPU 메트릭을 모니터링 할 건지?
Do you want to monitor cpu metrics per core?
1. yes
2. no
default choice: [1]:
1

# 모든 메트릭에 EC2 Dimension(이미지 ID, 인스턴스 ID, 인스턴스 타입, 오토스케일링 그룹 이름)의 정보를 추가해서 수집할 건지?
Do you want to add ec2 dimensions (ImageId, InstanceId, InstanceType, AutoScalingGroupName) into all of your metr
1. yes
2. no
default choice: [1]:
1

# EC2 Dimension(InstanceId)를 기준으로 집계를 할 건지?
Do you want to aggregate ec2 dimensions (InstanceId)?
1. yes
2. no
default choice: [1]:
1

# 메트릭을 몇 초 간격으로 수집을 할 건지?
Would you like to collect your metrics at high resolution (sub-minute resolution)? This enables sub-minute resolution f
1. 1s
2. 10s
3. 30s
4. 60s
default choice: [4]:
4

# 기본 메트릭 구성을 어떻게 설정할 건지?
# Basic : 메모리 사용량(mem_used_percent), 디스크 사용량(disk_used_percent))
Which default metrics config do you want?
1. Basic
2. Standard
3. Advanced
4. None
default choice: [1]:
1

Current config as follows:
{
	"agent": {
		"metrics_collection_interval": 60,
		"run_as_user": "root"
	},
	"metrics": {
		"aggregation_dimensions": [
			[
				"InstanceId"
			]
		],
		"append_dimensions": {
			"AutoScalingGroupName": "${aws:AutoScalingGroupName}",
```

```
            "ImageId": "${aws:ImageId}",
            "InstanceId": "${aws:InstanceId}",
            "InstanceType": "${aws:InstanceType}"
        },
        "metrics_collected": {
            "disk": {
                "measurement": [
                    "used_percent"
                ],
                "metrics_collection_interval": 60,
                "resources": [
                    "*"
                ]
            },
            "mem": {
                "measurement": [
                    "mem_used_percent"
                ],
                "metrics_collection_interval": 60
            }
        }
    }
}

# 위의 구성에 만족하는 지?
Are you satisfied with the above config? Note: it can be manually customized after the wizard completes to add additi
1. yes
2. no
default choice: [1]:
1

# 합치고 싶은 CloudWatch Agent의 설정 파일이 있는 지?
Do you have any existing CloudWatch Log Agent (http://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/Agent
1. yes
2. no
default choice: [2]:
2

# 로그 파일을 모니터링 할 건지?
Do you want to monitor any log files?
1. yes
2. no
default choice: [1]:
2

# CloudWatch Agent가 X-ray(AWS의 분산 추적 시스템)의 정보를 수집하게 할 건지?
Do you want the CloudWatch agent to also retrieve X-ray traces?
1. yes
2. no
default choice: [1]:
2

Existing config JSON identified and copied to:  /opt/aws/amazon-cloudwatch-agent/etc/backup-configs
Saved config file to /opt/aws/amazon-cloudwatch-agent/bin/config.json successfully.
Current config as follows:
{
    "agent": {
        "metrics_collection_interval": 60,
```

```
        "run_as_user": "root"
    },
    "metrics": {
        "aggregation_dimensions": [
            [
                "InstanceId"
            ]
        ],
        "append_dimensions": {
            "AutoScalingGroupName": "${aws:AutoScalingGroupName}",
            "ImageId": "${aws:ImageId}",
            "InstanceId": "${aws:InstanceId}",
            "InstanceType": "${aws:InstanceType}"
        },
        "metrics_collected": {
            "disk": {
                "measurement": [
                    "used_percent"
                ],
                "metrics_collection_interval": 60,
                "resources": [
                    "*"
                ]
            },
            "mem": {
                "measurement": [
                    "mem_used_percent"
                ],
                "metrics_collection_interval": 60
            }
        }
    }
}
Please check the above content of the config.
The config file is also located at /opt/aws/amazon-cloudwatch-agent/bin/config.json.
Edit it manually if needed.

# 설정 정보를 AWS의 SSM parameter store(AWS의 값 저장소)에 저장할 건지?
Do you want to store the config in the SSM parameter store?
1. yes
2. no
default choice: [1]:
2

Program exits now.
```

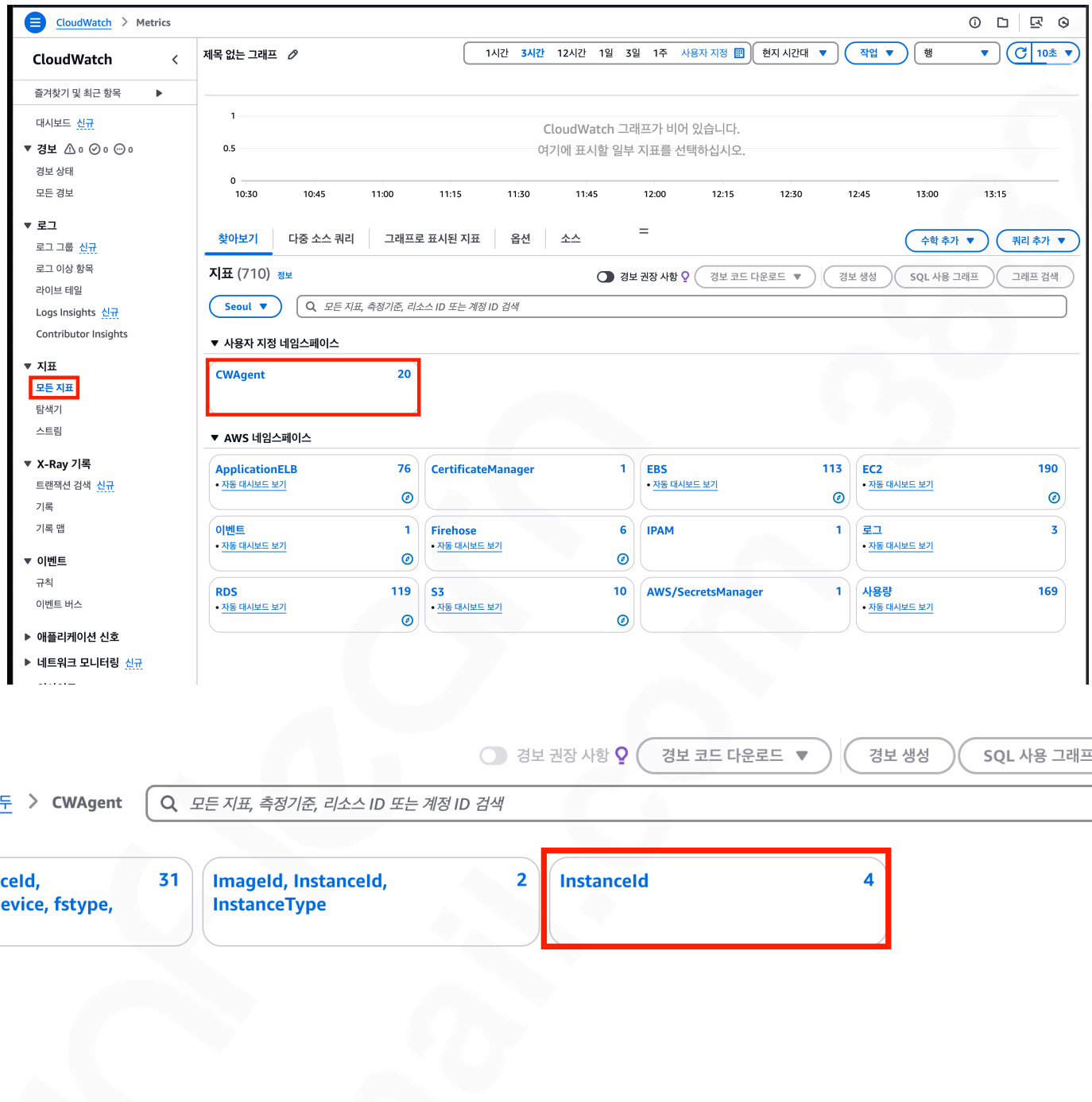
7. **CloudWatch Agent 실행하기**

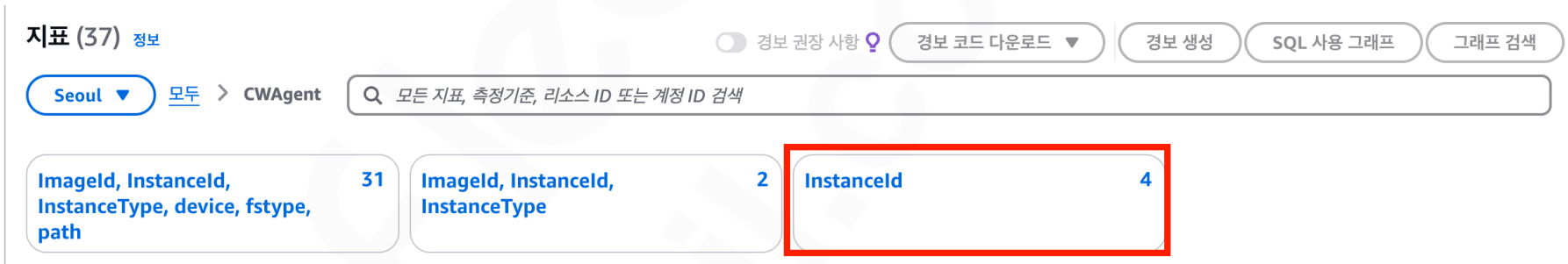
```
$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -s -c file:/opt/
```

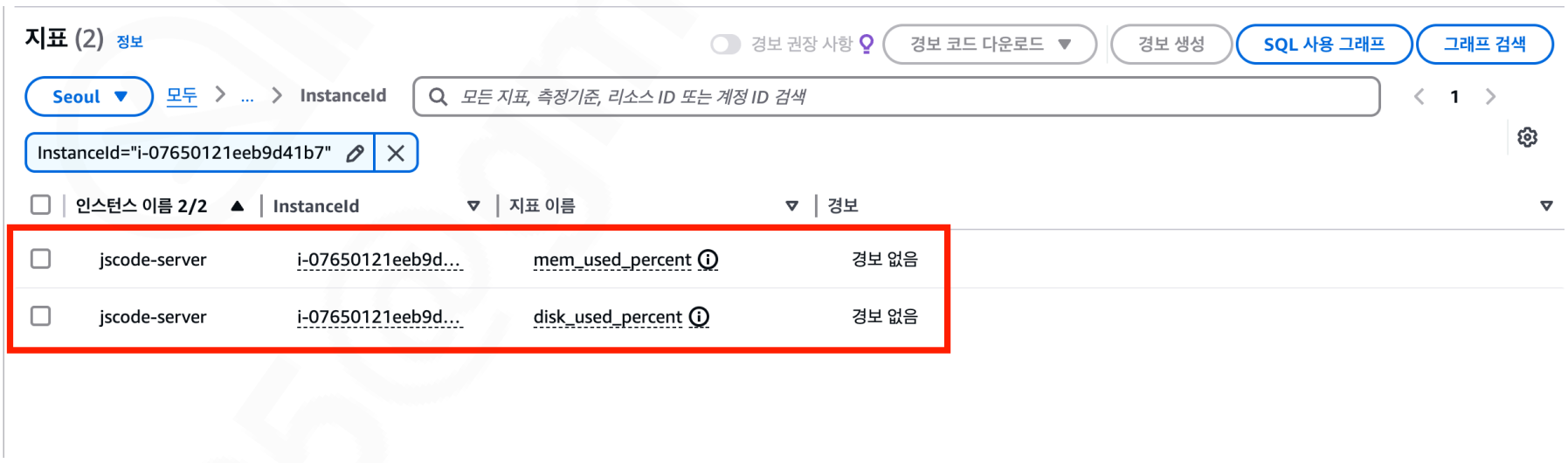
8. **CloudWatch Agent가 잘 실행되고 있는 지 확인하기**

```
$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -m ec2 -a status
```

9. **CloudWatch로 가서 메트릭이 잘 수집되고 있는 지 확인하기**

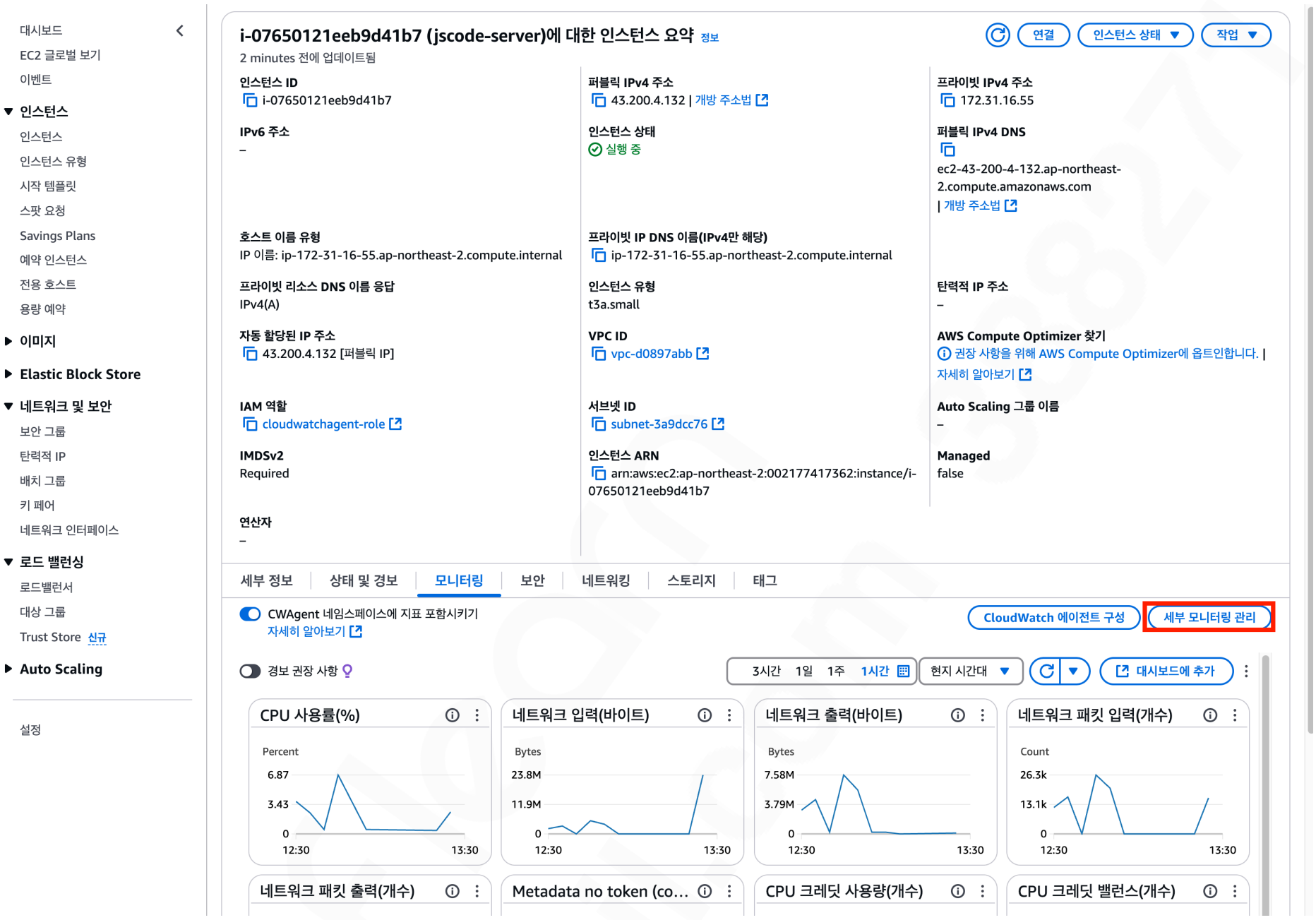



## ✅ 세부 모니터링 설정

EC2에 기본적으로 설정되어 있는 모니터링은 5분 간격으로 메트릭(CPU 등)을 수집한다. 부하 테스트를 통해 정확한 모니터링을 하기 위해 5분 간격이 아닌 1분 간격으로 메트릭을 수집하게끔 바꿔보자.


대시보드
EC2 글로벌 보기
이벤트
인스턴스
인스턴스
인스턴스 유형
시작 템플릿
스팟 요청
Savings Plans
예약 인스턴스
전용 호스트
용량 예약
이미지
Elastic Block Store
네트워크 및 보안
보안 그룹
탄력적 IP
배치 그룹
키 페어
네트워크 인터페이스
로드 밸런싱
로드밸런서
대상 그룹
Trust Store 신규
Auto Scaling
설정
i-07650121eeb9d41b7 (jscode-server)에 대한 인스턴스 요약 정보
2 minutes 전에 업데이트됨
연결
인스턴스 상태
작업
인스턴스 ID
i-07650121eeb9d41b7
IPv6 주소
–
퍼블릭 IPv4 주소
43.200.4.132 | 개방 주소법
인스턴스 상태
실행 중
프라이빗 IPv4 주소
172.31.16.55
퍼블릭 IPv4 DNS
ec2-43-200-4-132.ap-northeast-2.compute.amazonaws.com | 개방 주소법
호스트 이름 유형
IP 이름: ip-172-31-16-55.ap-northeast-2.compute.internal
프라이빗 IP DNS 이름(IPv4만 해당)
ip-172-31-16-55.ap-northeast-2.compute.internal
프라이빗 리소스 DNS 이름 응답
IPv4(A)
인스턴스 유형
t3a.small
탄력적 IP 주소
–
자동 할당된 IP 주소
43.200.4.132 [퍼블릭 IP]
VPC ID
vpc-d0897abb
AWS Compute Optimizer 찾기
권장 사항을 위해 AWS Compute Optimizer에 옵트인합니다. | 자세히 알아보기
IAM 역할
cloudwatchagent-role
서브넷 ID
subnet-3a9dcc76
Auto Scaling 그룹 이름
–
IMDSv2
Required
인스턴스 ARN
arn:aws:ec2:ap-northeast-2:002177417362:instance/i-07650121eeb9d41b7
Managed
false
연산자
–
세부 정보
상태 및 경보
모니터링
보안
네트워킹
스토리지
태그
CWAgent 네임스페이스에 지표 포함시키기
자세히 알아보기
CloudWatch 에이전트 구성
세부 모니터링 관리
경보 권장 사항
3시간 1일 1주 1시간
현지 시간대
대시보드에 추가
CPU 사용률(%)
Percent
6.87
3.43
0
12:30
13:30
네트워크 입력(바이트)
Bytes
23.8M
11.9M
0
12:30
13:30
네트워크 출력(바이트)
Bytes
7.58M
3.79M
0
12:30
13:30
네트워크 패킷 입력(개수)
Count
26.3k
13.1k
0
12:30
13:30
네트워크 패킷 출력(개수)
Metadata no token (co…
CPU 크레딧 사용량(개수)
CPU 크레딧 밸런스(개수)

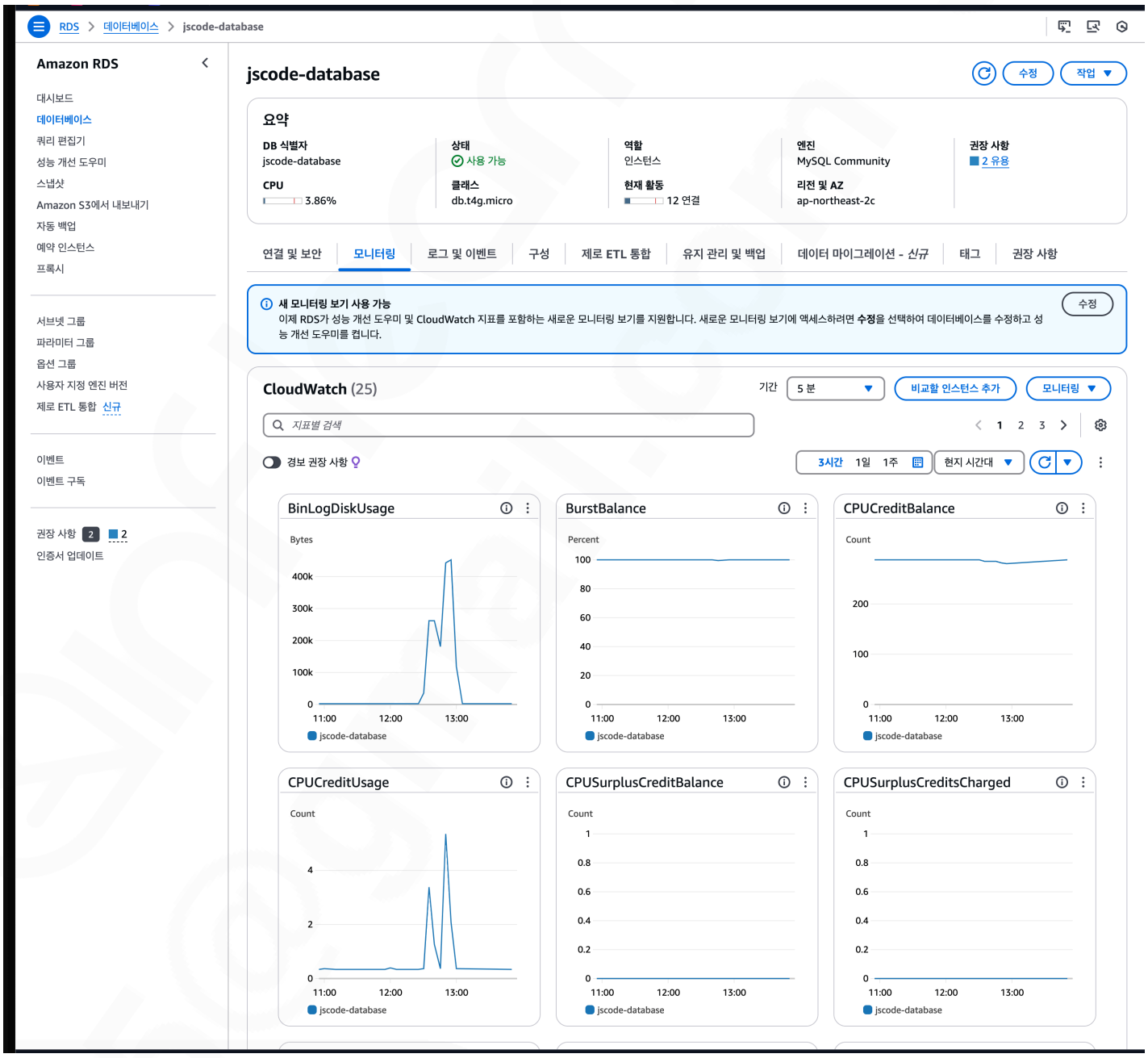
서브넷 ID
subnet-3a9dcc76
Auto Scaling 그룹 이름
–
세부 모니터링
인스턴스에 대한 세부 모니터링을 활성화한 후 모니터링 데이터를 1분 동안 사용할 수 있습니다. 자세히 알아보기
인스턴스 ID
i-07650121eeb9d41b7 (jscode-server)
세부 모니터링
활성화
세부 모니터링을 활성화하면 Amazon EC2 콘솔에 인스턴스에 대한 1분 기간의 모니터링 그래프가 표시됩니다.
추가 요금 적용
취소
확인
모니터링
포함시키기
Bytes
23.8M
Bytes
7.58M
Count
26.3k

# [실습] RDS 모니터링 셋팅하기

## ✅ RDS 모니터링 셋팅하기

EC2와 비슷하게 RDS도 생성하면 기본적으로 수집하는 지표들이 있다.

- CPU 사용률 (CPUUtilization)
- FreeableMemory (사용 가능한 메모리 용량)
- 등등




RDS는 별도의 설정을 하지 않더라도 1분 간격으로 지표를 수집하기 때문에 별도의 설정을 추가적으로 해주지 않아도 된다.

# [보충 강의] ELB의 CPU, 메모리는 왜 측정하지 않나요?

## ✅ ELB의 CPU, 메모리는 왜 측정하지 않나요?

AWS에서 ELB를 만들 때 1초당 수백만 개의 요청을 처리할 수 있게끔 고사양 컴퓨터를 여러 대 셋팅해뒀다. 다른 말로 표현하자면, ELB 때문에 성능 저하가 일어날 가능성은 아주 적으니 신경쓰지 않아도 된다는 뜻이다. 이러한 이유 때문에 생성한 ELB(정확히는 ALB)의 모니터링 지표에는 CPU, 메모리에 대한 정보가 포함되어 있지 않다.

### 고가용성

Elastic Load Balancer는 가용성이 매우 뛰어납니다. 수신되는 트래픽을 단일 가용 영역 또는 여러 가용 영역의 Amazon EC2 인스턴스 전체에 걸쳐 배포할 수 있습니다. Elastic Load Balancer는 수신되는 애플리케이션 트래픽에 대응하여 요청 처리 용량을 자동으로 조정합니다. 대상이 사용 가능하고 정상 상태인지 확인하기 위해 Elastic Load Balancer는 구성 가능한 케이던스로 대상에 대해 상태 확인을 실행합니다.

### 높은 처리량

Elastic Load Balancer는 트래픽 증가를 처리할 수 있도록 설계되었으며 초당 수백만 개의 요청을 로드 밸런싱할 수 있습니다. 또한, 갑작스럽고 변동이 심한 트래픽 패턴도 처리할 수 있습니다.

출처 : https://aws.amazon.com/ko/elasticloadbalancing/features/#Product_comparisons

## ✅ ELB의 모니터링 지표

khb3025@gmail.com

EC2 > 로드 밸런서 > jscode-alb

대시보드
EC2 글로벌 보기
이벤트
▼ 인스턴스
인스턴스
인스턴스 유형
시작 템플릿
스팟 요청
Savings Plans
예약 인스턴스
전용 호스트
용량 예약
▶ 이미지
▶ Elastic Block Store
▼ 네트워크 및 보안
보안 그룹
탄력적 IP
배치 그룹
키 페어
네트워크 인터페이스
▼ 로드 밸런싱
로드밸런서
대상 그룹
Trust Store 신규
▶ Auto Scaling
설정

# jscode-alb

작업

## 세부 정보

**로드 밸런서 유형**
애플리케이션

**상태**
활성

**VPC**
vpc-d0897abb

**로드 밸런서 IP 주소 유형**
IPv4

**체계**
Internet-facing

**호스팅 영역**
ZWKZPGTI48KDX

**가용 영역**
subnet-6f48a514 ap-northeast-2b (apne2-az2)
subnet-3a9dcc76 ap-northeast-2c (apne2-az3)
subnet-1f3e6043 ap-northeast-2d (apne2-az4)
subnet-35db365e ap-northeast-2a (apne2-az1)

**생성된 날짜**
2024년 11월 25일, 14:24 (UTC+09:00)

**로드 밸런서 ARN**
arn:aws:elasticloadbalancing:ap-northeast-2:002177417362:loadbalancer/app/jscode-alb/4a033bcccb0dbbd3

**DNS 이름** 정보
jscode-alb-475314812.ap-northeast-2.elb.amazonaws.com (A 레코드)

리스너 및 규칙 | 네트워크 매핑 | 리소스 맵 - *신규* | 보안 | 모니터링 | 통합 | 속성 | 용량 - *신규* | 태그

1시간 3시간 12시간 1일 3일 1주 사용자 지정 | 현지 시간대 | 대시보드에 추가

### 대상 응답 시간
Seconds
6e-3
3e-3
0
11:12 14:11
jscode-alb

### 요청
Count
53
26.5
0
11:12 14:11
jscode-alb

### 규칙 평가
None
1
0.5
0
사용 가능한 데이터가 없습니다.
대시보드 시간 범위를 조정해 보십시오.
11:12 14:11
jscode-alb

### 대상 5XXs
None
1
0.5
0
사용 가능한 데이터가 없습니다.
대시보드 시간 범위를 조정해 보십시오.
11:12 14:11
jscode-alb

### 대상 4XXs
Count
53
26.5
0
11:12 14:11
jscode-alb

### ELB 5XXs
None
1
0.5
0
사용 가능한 데이터가 없습니다.
대시보드 시간 범위를 조정해 보십시오.
11:12 14:11
jscode-alb

### ELB 4XXs
None
1
0.5
0
사용 가능한 데이터가 없습니다.
대시보드 시간 범위를 조정해 보십시오.
11:12 14:11
jscode-alb

### ELB 500s
None
1
0.5
0
사용 가능한 데이터가 없습니다.
대시보드 시간 범위를 조정해 보십시오.
11:12 14:11
jscode-alb

### ELB 502s
None
1
0.5
0
사용 가능한 데이터가 없습니다.
대시보드 시간 범위를 조정해 보십시오.
11:12 14:11
jscode-alb

### ELB 503s
None
1
0.5
0
사용 가능한 데이터가 없습니다.
대시보드 시간 범위를 조정해 보십시오.
11:12 14:11
jscode-alb

### ELB 504s
None
1
0.5
0
사용 가능한 데이터가 없습니다.
대시보드 시간 범위를 조정해 보십시오.
11:12 14:11
jscode-alb

### 대상 연결 오류
None
1
0.5
0
사용 가능한 데이터가 없습니다.
대시보드 시간 범위를 조정해 보십시오.
11:12 14:11
jscode-alb

### 거부된 연결 개수 합계
None
1
0.5
사용 가능한 데이터가 없습니다.
대시보드 시간 범위를 조정해 보십시오.

### 대상 TLS 협상 오류
None
1
0.5
사용 가능한 데이터가 없습니다.
대시보드 시간 범위를 조정해 보십시오.

### 클라이언트 TLS 협상 오류
None
1
0.5
사용 가능한 데이터가 없습니다.
대시보드 시간 범위를 조정해 보십시오.

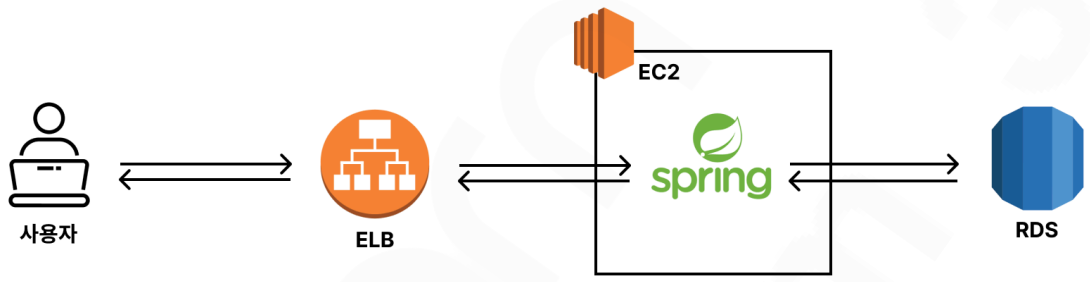
### 대상 3XXs
None
1
0.5
사용 가능한 데이터가 없습니다.
대시보드 시간 범위를 조정해 보십시오.

khb302500

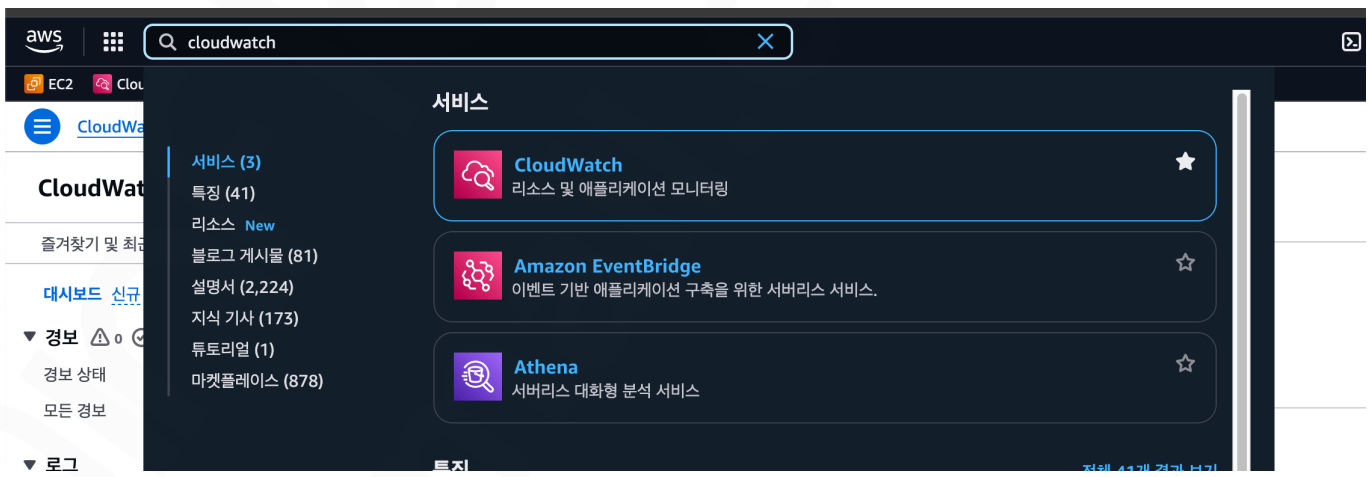
# [실습] 각 서버의 CPU, 메모리를 한 눈에 볼 수 있도록 셋팅하기

## ✅ 각 서버의 CPU, 메모리를 한 눈에 볼 수 있도록 셋팅하기

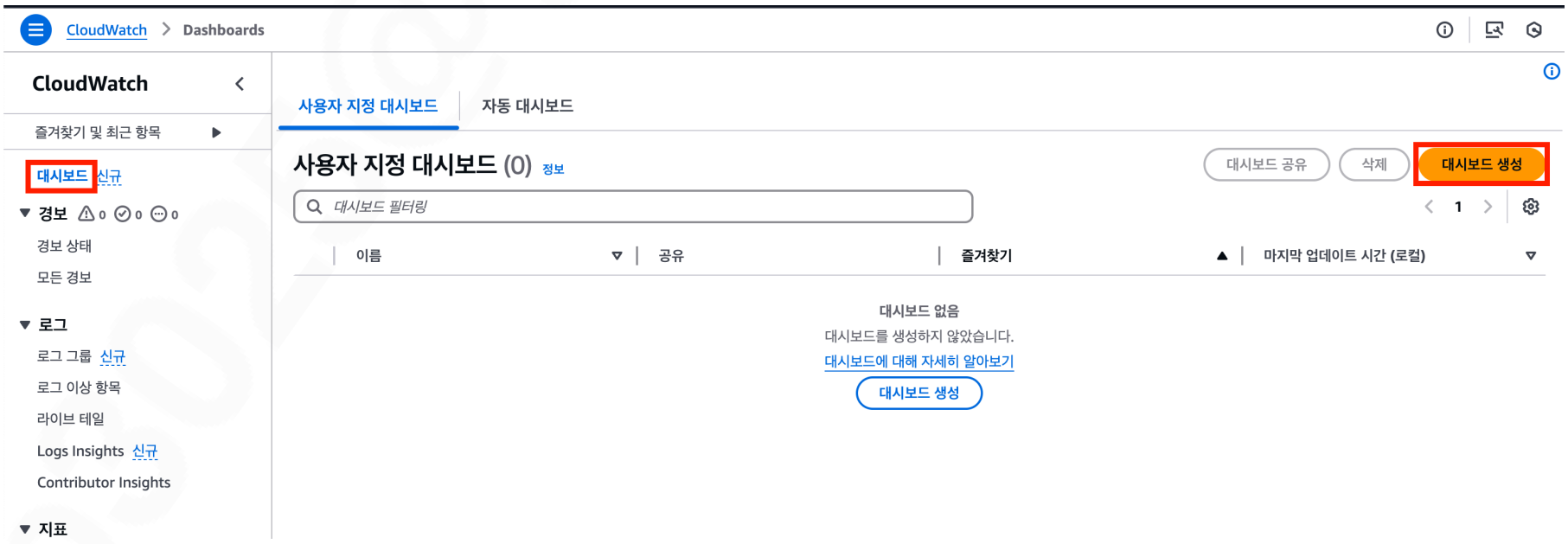
병목 지점을 알아내기 위해 EC2, RDS의 **CPU 사용률**과 **메모리 사용률**을 한 눈에 볼 수 있도록 셋팅해볼 것이다.

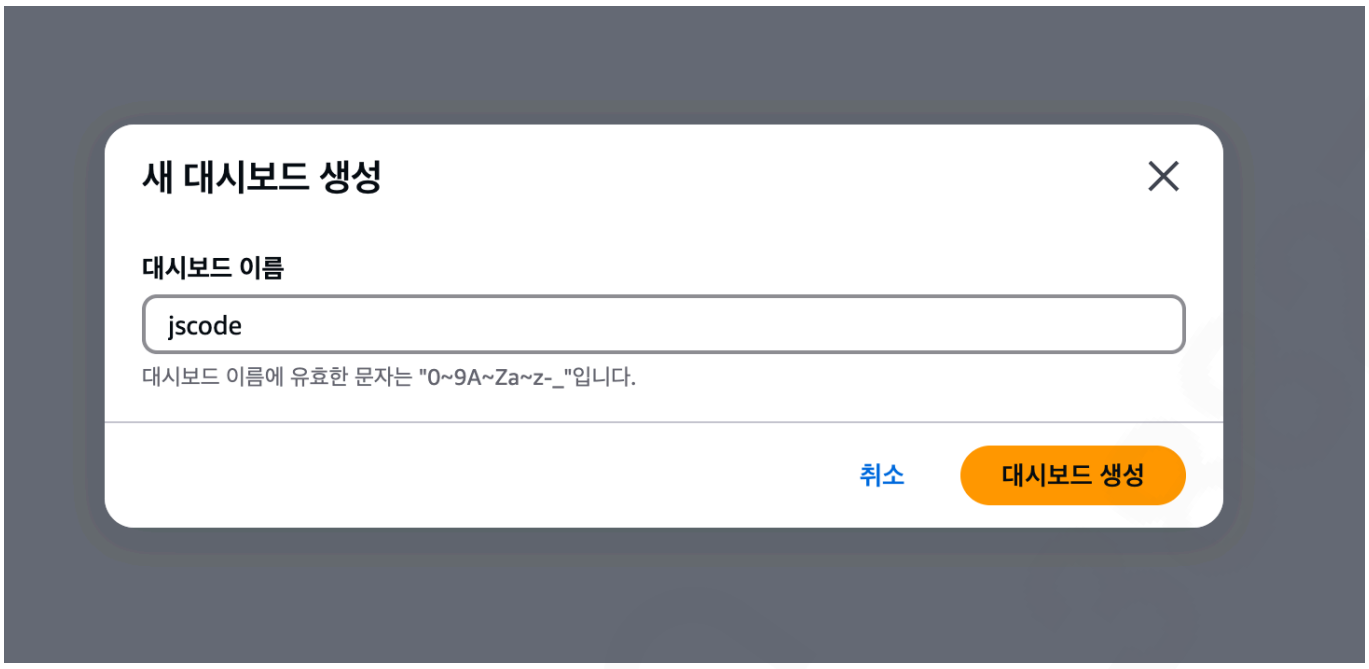



1. **CloudWatch 페이지 들어가기**

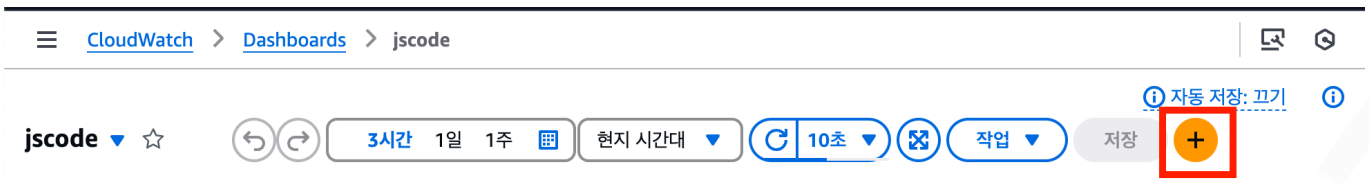



2. **대시보드 생성하기**





3. **위젯(EC2 CPU 사용률) 추가하기**



knb3025@gmail.com

지표 그래프 추가

제목 없는 그래프

1시간 3시간 12시간 1일 3일 1주 사용자 지정 | 현지 시간대 | 행 | 10초

1
0.5
0

CloudWatch 그래프가 비어 있습니다.
여기에 표시할 일부 지표를 선택하십시오.

12:45 13:00 13:15 13:30 13:45 14:00 14:15 14:30 14:45 15:00 15:15 15:30

찾아보기 | 다중 소스 쿼리 | 그래프로 표시된 지표 | 옵션 | 소스

수학 추가 | 쿼리 추가

지표 (752)

경보 권장 사항 | SQL 사용 그래프 | 그래프 검색

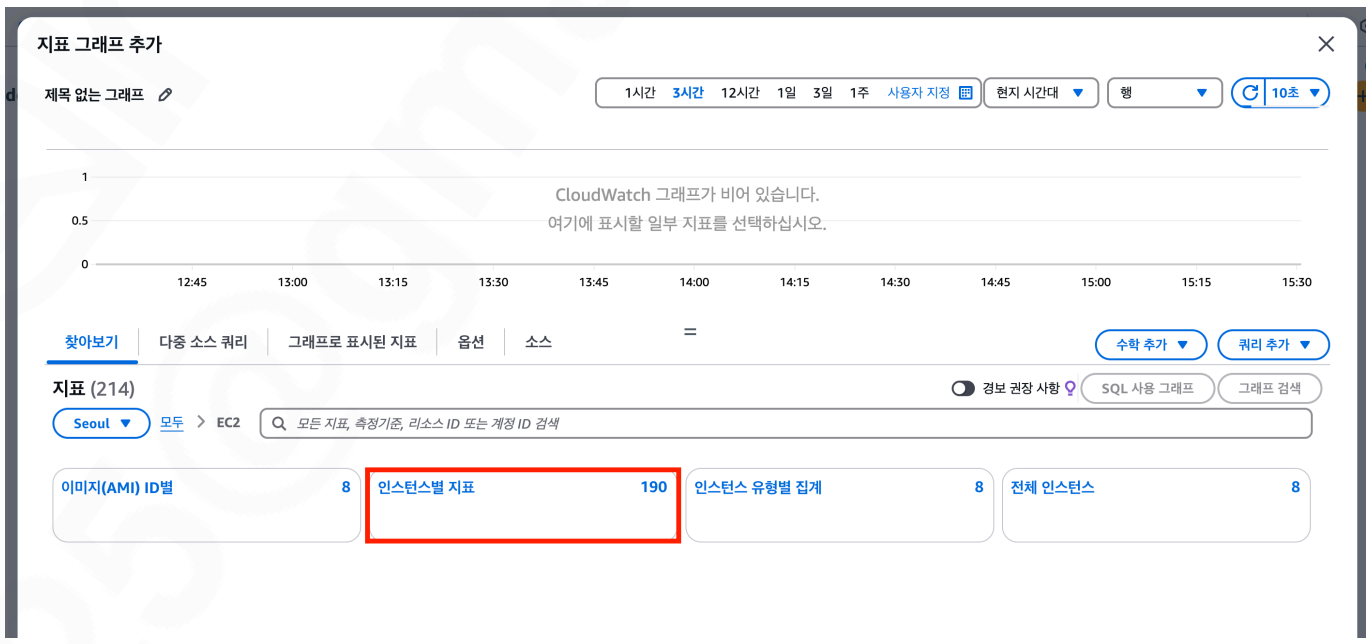
Seoul | 모든 지표, 측정기준, 리소스 ID 또는 계정 ID 검색

▼ 사용자 지정 네임스페이스

| | |
|---|---|
| CWAgent | 37 |

▼ AWS 네임스페이스

| | | | |
|---|---|---|---|
| ApplicationELB 76 | CertificateManager 1 | EBS 113 | EC2 214 |
| 이벤트 1 | Firehose 6 | IPAM 1 | 로그 3 |
| RDS 119 | S3 10 | AWS/SecretsManager 1 | 사용량 170 |




khb3023@gmail.com

지표 그래프 추가

제목 없는 그래프

시간 범위 지속 | 1시간 3시간 12시간 1일 3일 1주 사용자 지정 | 현지 시간대 | 행 | 10초

Percent
6.87
3.62
0.365
12:30 12:45 13:00 13:15 13:30 13:45 14:00 14:15 14:30 14:45 15:00 15:15 15:30
CPUUtilization

찾아보기 | 다중 소스 쿼리 | 그래프로 표시된 지표(1) | 옵션 | 소스 =

수학 추가 | 쿼리 추가

지표 (10)

경보 권장 사항 | SQL 사용 그래프 | 그래프 검색

Seoul | 모두 > ... > 인스턴스별 지표 | cpuutilization | 1

cpuutilization

| | 인스턴스 이름 10/10 | InstanceId | 지표 이름 | 경보 |
|---|---|---|---|---|
| ☐ | 지정된 이름이 없음 | i-08ccc4969a1fee... | CPUUtilization | 경보 없음 |
| ☐ | 지정된 이름이 없음 | i-0fb1d938a9061... | CPUUtilization | 경보 없음 |
| ☐ | 지정된 이름이 없음 | i-0e4cd9ecbec5c0... | CPUUtilization | 경보 없음 |
| ☐ | 지정된 이름이 없음 | i-0f2a71ce011c6a... | CPUUtilization | 경보 없음 |
| ☐ | 지정된 이름이 없음 | i-01aede3852feff... | CPUUtilization | 경보 없음 |
| ☐ | 지정된 이름이 없음 | i-0d701f540c170... | CPUUtilization | 경보 없음 |
| ☐ | 지정된 이름이 없음 | i-0f45330e14fa67... | CPUUtilization | 경보 없음 |
| ☐ | 지정된 이름이 없음 | i-035dff0ffa725f5... | CPUUtilization | 경보 없음 |
| ☑ | jscode-server | i-07650121eeb9d... | CPUUtilization | 경보 없음 |
| ☐ | k6-server | i-0814b9302156a... | CPUUtilization | 경보 없음 |

취소 | 위젯 생성

지표 그래프 추가

제목 없는 그래프

시간 범위 지속 | 1시간 3시간 12시간 1일 3일 1주 사용자 지정 | 현지 시간대 | 행 | 10초

Percent
6.87
3.62
0.365
12:45 13:00 13:15 13:30 13:45 14:00 14:15 14:30 14:45 15:00 15:15 15:30
CPUUtilization

찾아보기 | 다중 소스 쿼리 | 그래프로 표시된 지표(1) | 옵션 | 소스 =

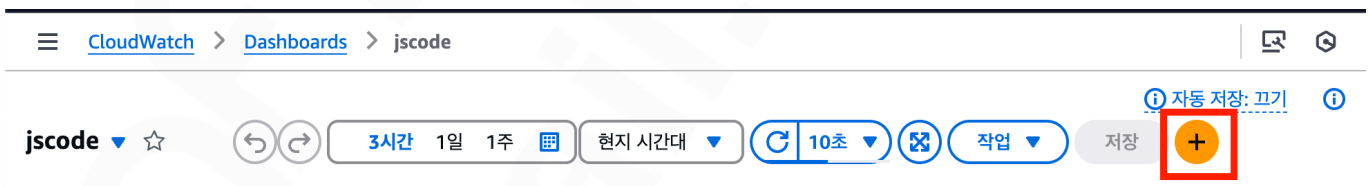
수학 추가 | 쿼리 추가

동적 레이블 추가

통계: 평균 | 기간: 1분 | 그래프 지우기

| | 레이블 | 세부 정보 | 통계 | 기간 | Y축 | 작업 |
|---|---|---|---|---|---|---|
| ☑ | CPUUtilization | EC2 • CPUUtilization • InstanceId: i-0765( | 평균 | 1분 | | |

4. **위젯(EC2 메모리 사용률) 추가하기**




khb302@naver.com

위젯 추가

데이터 소스 유형 - *신규*

- Cloudwatch
- 기타 콘텐츠 유형
- 데이터 소스 생성

위젯 구성

데이터 유형

지표 | 로그 | 경보

위젯 유형

- **행** 시간별 지표 비교
- **데이터 테이블** 테이블에서 시간 경과에 따른 지표 값 비교
- **번호** 즉각적으로 지표 최신 값 확인 — 75 %
- **게이지** 범위 내 지표의 최신 값 보기
- **누적 면적** 시간별 합계 비교
- **막대** 데이터 범주 비교
- **파이** 백분율 또는 비례 데이터 표시
- **탐색기** 여러 개의 태그 기반 그래프가 있는 단일 위젯

취소 | 다음

지표 그래프 추가

제목 없는 그래프

1시간 | 3시간 | 12시간 | 1일 | 3일 | 1주 | 사용자 지정 | 현지 시간대 | 행 | 10초

1
0.5
0

CloudWatch 그래프가 비어 있습니다.
여기에 표시할 일부 지표를 선택하십시오.

12:45 13:00 13:15 13:30 13:45 14:00 14:15 14:30 14:45 15:00 15:15 15:30

찾아보기 | 다중 소스 쿼리 | 그래프로 표시된 지표 | 옵션 | 소스

수학 추가 | 쿼리 추가

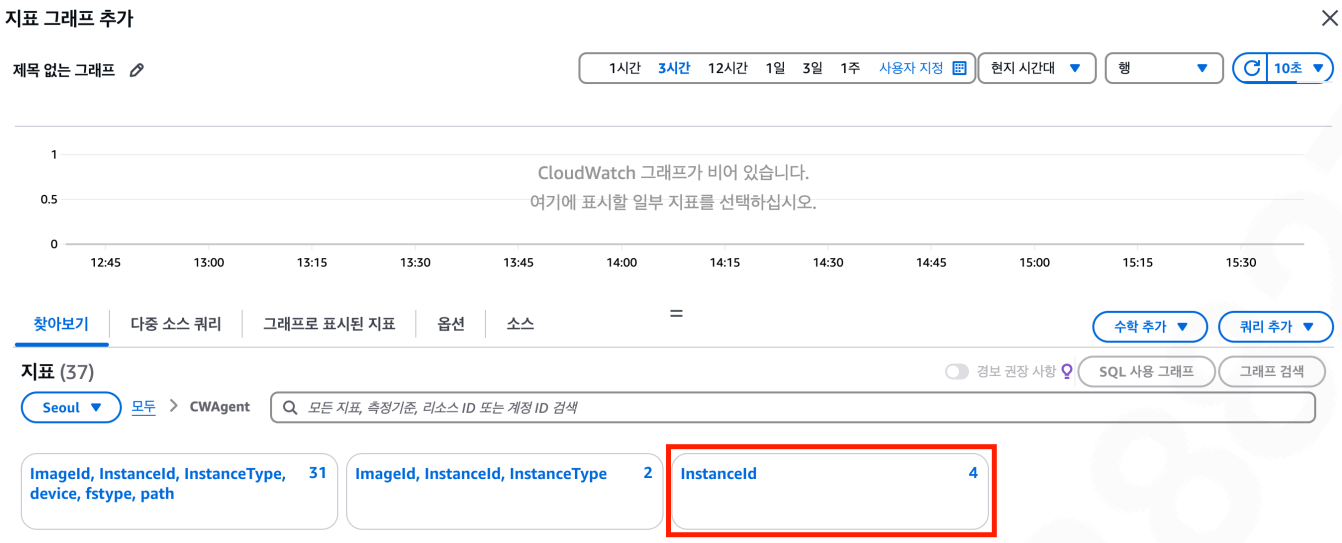
지표 (752)

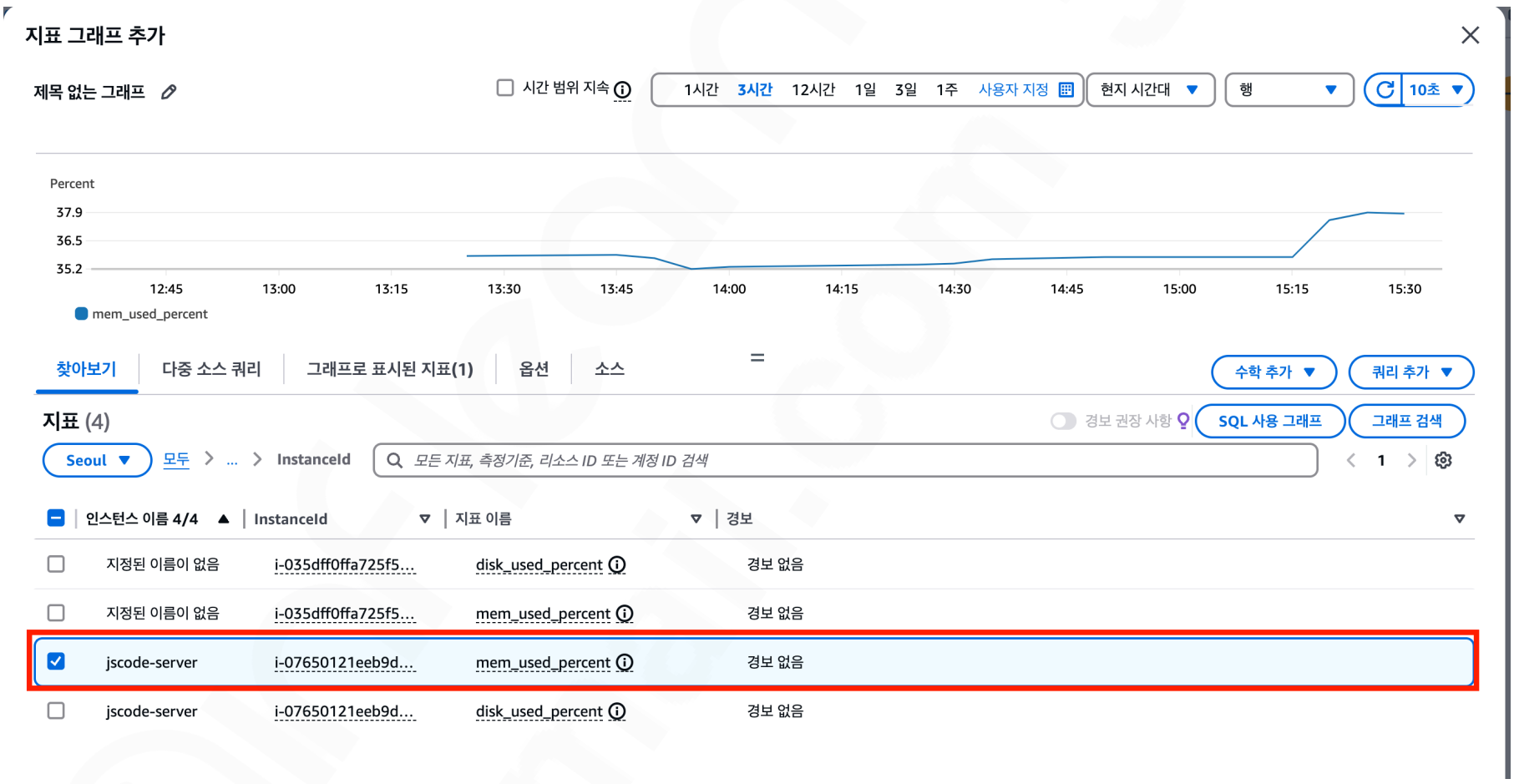
경보 권장 사항 | SQL 사용 그래프 | 그래프 검색

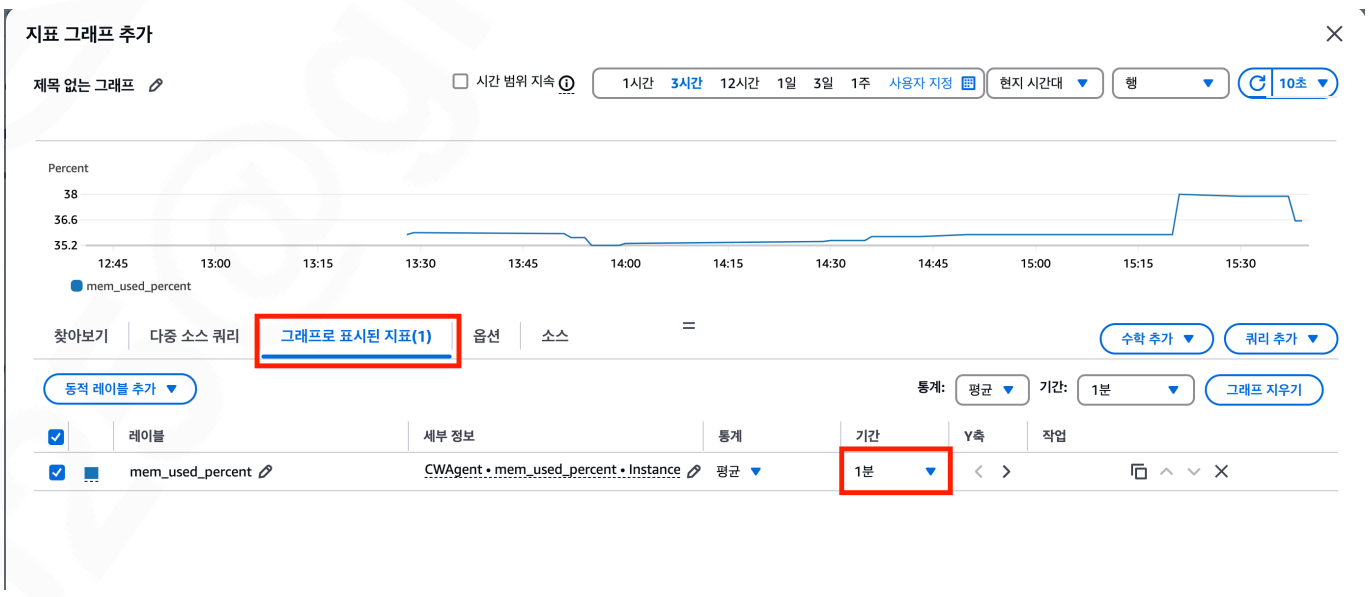
Seoul | 모든 지표, 측정기준, 리소스 ID 또는 계정 ID 검색

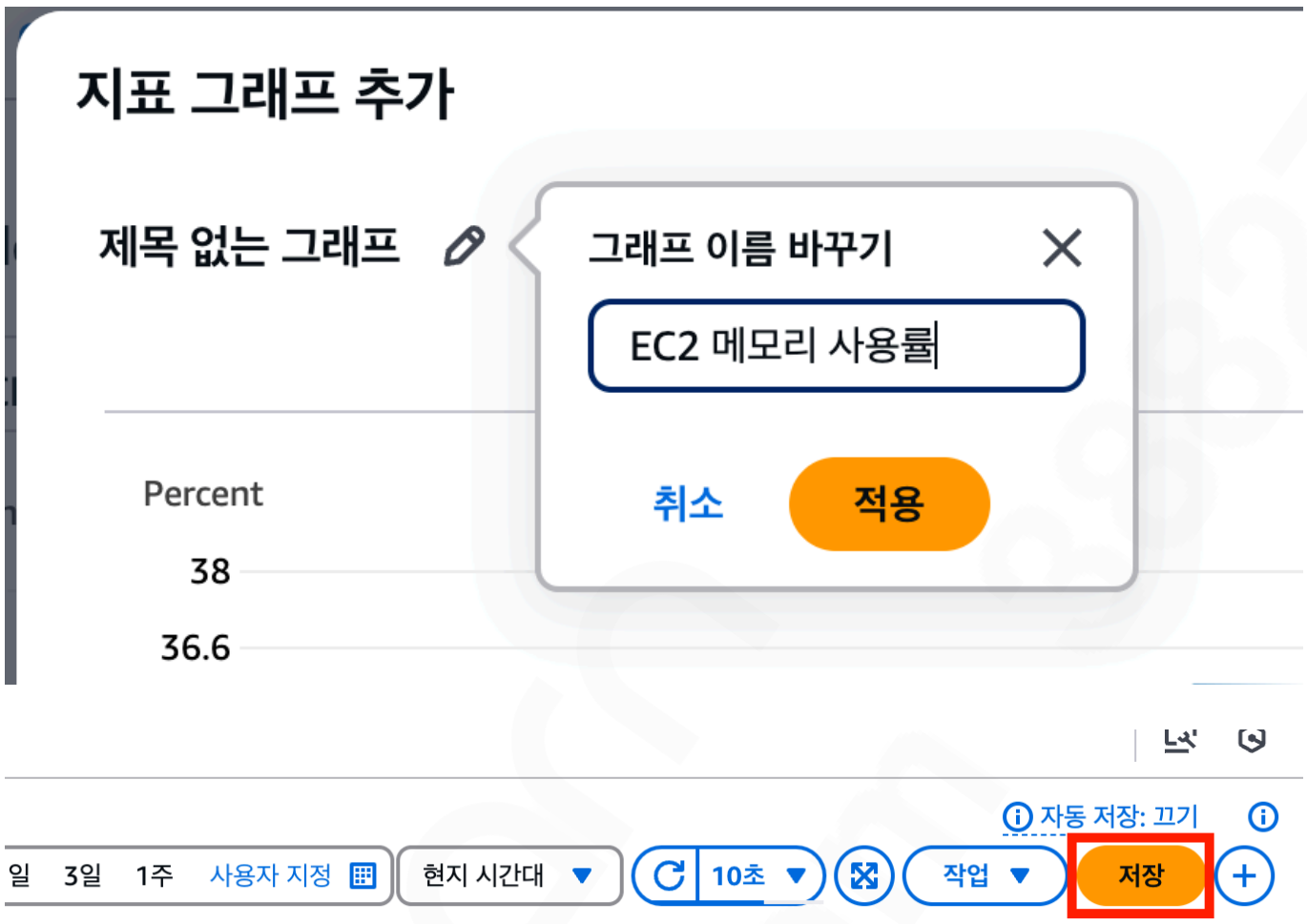
▼ 사용자 지정 네임스페이스

CWAgent 37

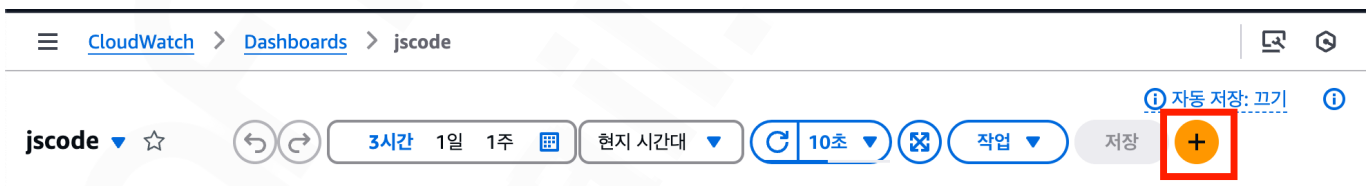

지표 그래프 추가
제목 없는 그래프
1시간 3시간 12시간 1일 3일 1주 사용자 지정
현지 시간대
행
10초
1
0.5
0
CloudWatch 그래프가 비어 있습니다.
여기에 표시할 일부 지표를 선택하십시오.
12:45 13:00 13:15 13:30 13:45 14:00 14:15 14:30 14:45 15:00 15:15 15:30
찾아보기
다중 소스 쿼리
그래프로 표시된 지표
옵션
소스
=
수학 추가
쿼리 추가
지표 (37)
경보 권장 사항
SQL 사용 그래프
그래프 검색
Seoul
모두
CWAgent
모든 지표, 측정기준, 리소스 ID 또는 계정 ID 검색
ImageId, InstanceId, InstanceType, device, fstype, path 31
ImageId, InstanceId, InstanceType 2
InstanceId 4


지표 그래프 추가
제목 없는 그래프
시간 범위 지속
1시간 3시간 12시간 1일 3일 1주 사용자 지정
현지 시간대
행
10초
Percent
37.9
36.5
35.2
12:45 13:00 13:15 13:30 13:45 14:00 14:15 14:30 14:45 15:00 15:15 15:30
mem_used_percent
찾아보기
다중 소스 쿼리
그래프로 표시된 지표(1)
옵션
소스
=
수학 추가
쿼리 추가
지표 (4)
경보 권장 사항
SQL 사용 그래프
그래프 검색
Seoul
모두
...
InstanceId
모든 지표, 측정기준, 리소스 ID 또는 계정 ID 검색
1
인스턴스 이름 4/4
InstanceId
지표 이름
경보
지정된 이름이 없음 i-035dff0ffa725f5... disk_used_percent 경보 없음
지정된 이름이 없음 i-035dff0ffa725f5... mem_used_percent 경보 없음
jscode-server i-07650121eeb9d... mem_used_percent 경보 없음
jscode-server i-07650121eeb9d... disk_used_percent 경보 없음


지표 그래프 추가
제목 없는 그래프
시간 범위 지속
1시간 3시간 12시간 1일 3일 1주 사용자 지정
현지 시간대
행
10초
Percent
38
36.6
35.2
12:45 13:00 13:15 13:30 13:45 14:00 14:15 14:30 14:45 15:00 15:15 15:30
mem_used_percent
찾아보기
다중 소스 쿼리
그래프로 표시된 지표(1)
옵션
소스
=
수학 추가
쿼리 추가
동적 레이블 추가
통계: 평균
기간: 1분
그래프 지우기
레이블
세부 정보
통계
기간
Y축
작업
mem_used_percent
CWAgent • mem_used_percent • Instance
평균
1분




5. **위젯(RDS CPU 사용률) 추가하기**




khb302500@gmail.com

위젯 추가

데이터 소스 유형 - *신규*

- Cloudwatch
- 기타 콘텐츠 유형
- 데이터 소스 생성

위젯 구성

데이터 유형

지표 | 로그 | 경보

위젯 유형

- **행** 시간별 지표 비교
- **데이터 테이블** 테이블에서 시간 경과에 따른 지표 값 비교
- **번호** 즉각적으로 지표 최신 값 확인 (75 %)
- **게이지** 범위 내 지표의 최신 값 보기
- **누적 면적** 시간별 합계 비교
- **막대** 데이터 범주 비교
- **파이** 백분율 또는 비례 데이터 표시
- **탐색기** 여러 개의 태그 기반 그래프가 있는 단일 위젯

취소 | 다음

찾아보기 | 다중 소스 쿼리 | 그래프로 표시된 지표 | 옵션 | 소스

수학 추가 | 쿼리 추가

지표 (752)

경보 권장 사항 | SQL 사용 그래프 | 그래프 검색

Seoul | 모든 지표, 측정기준, 리소스 ID 또는 계정 ID 검색

▼ 사용자 지정 네임스페이스

| | |
|---|---|
| CWAgent | 37 |

▼ AWS 네임스페이스

| | | | |
|---|---|---|---|
| ApplicationELB 76 | CertificateManager 1 | EBS 113 | EC2 214 |
| 이벤트 1 | Firehose 6 | IPAM 1 | 로그 3 |
| RDS 119 | S3 10 | AWS/SecretsManager 1 | 사용량 170 |

찾아보기 | 다중 소스 쿼리 | 그래프로 표시된 지표 | 옵션 | 소스

수학 추가 | 쿼리 추가

지표 (119)

경보 권장 사항 | SQL 사용 그래프 | 그래프 검색

Seoul | 모두 > RDS | 모든 지표, 측정기준, 리소스 ID 또는 계정 ID 검색

| DBInstanceIdentifier | 47 | DatabaseClass | 24 | EngineName | 24 | 전체 데이터베이스 | 24 |
|---|---|---|---|---|---|---|---|

지표 그래프 추가

제목 없는 그래프

시간 범위 지속 | 1시간 | 3시간 | 12시간 | 1일 | 3일 | 1주 | 사용자 지정 | 현지 시간대 | 행 | 10초

찾아보기 | 다중 소스 쿼리 | 그래프로 표시된 지표(1) | 옵션 | 소스

수학 추가 | 쿼리 추가

지표 (2)

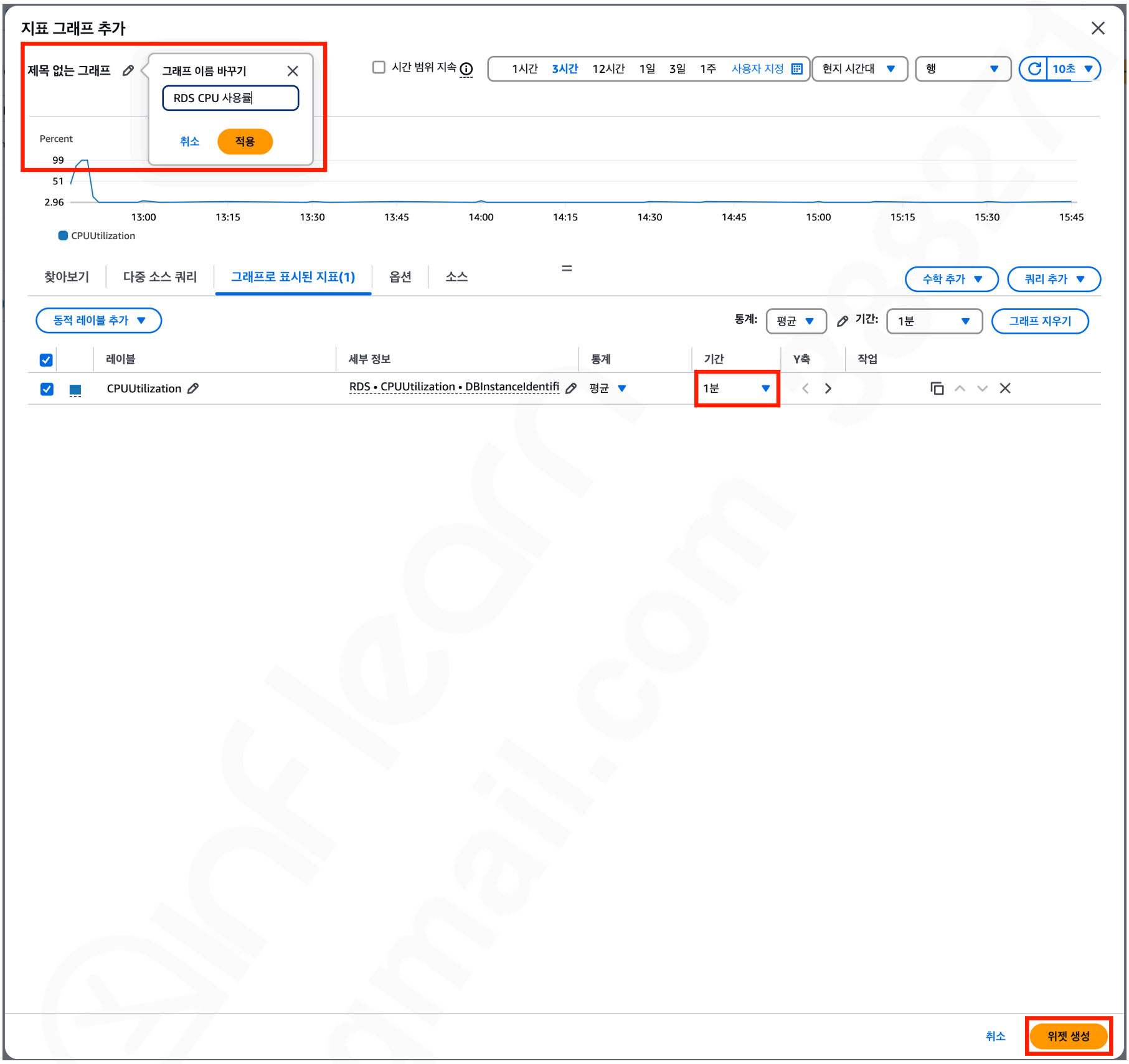
경보 권장 사항 | SQL 사용 그래프 | 그래프 검색

Seoul | 모두 > ... > DBInstanceIdentifier | cpuutilization | 1

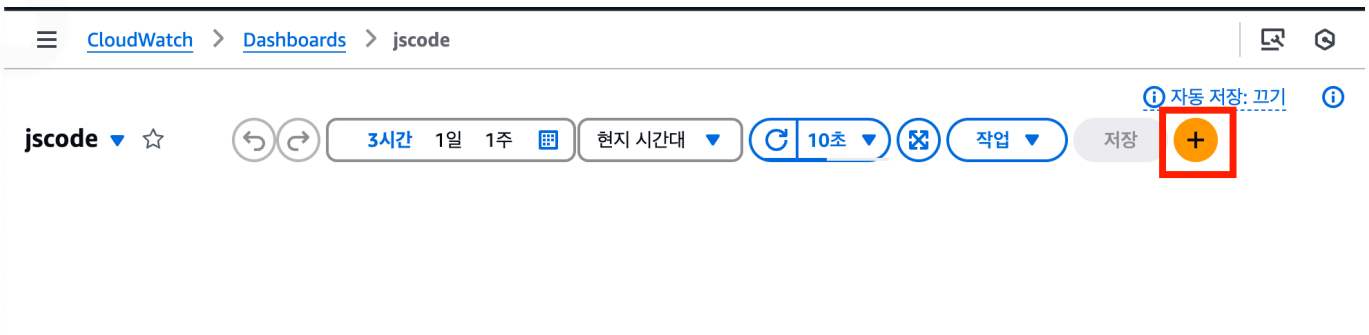
cpuutilization

| DBInstanceIdentifier 2/2 | 지표 이름 | 경보 |
|---|---|---|
| jscode-database | CPUUtilization | 경보 없음 |
| kube-database | CPUUtilization | 경보 없음 |

kb302500@gmail.com



6. **위젯(사용 가능한 RDS 메모리양) 추가하기**



위젯 추가

데이터 소스 유형 - *신규*

- Cloudwatch
- 기타 콘텐츠 유형
- 데이터 소스 생성

위젯 구성

데이터 유형

지표 | 로그 | 경보

위젯 유형

- **행** 시간별 지표 비교
- **데이터 테이블** 테이블에서 시간 경과에 따른 지표 값 비교
- **번호** 즉각적으로 지표 최신 값 확인 (75 %)
- **게이지** 범위 내 지표의 최신 값 보기
- **누적 면적** 시간별 합계 비교
- **막대** 데이터 범주 비교
- **파이** 백분율 또는 비례 데이터 표시
- **탐색기** 여러 개의 태그 기반 그래프가 있는 단일 위젯

취소 | 다음

찾아보기 | 다중 소스 쿼리 | 그래프로 표시된 지표 | 옵션 | 소스

수학 추가 | 쿼리 추가

지표 (752)

경보 권장 사항 | SQL 사용 그래프 | 그래프 검색

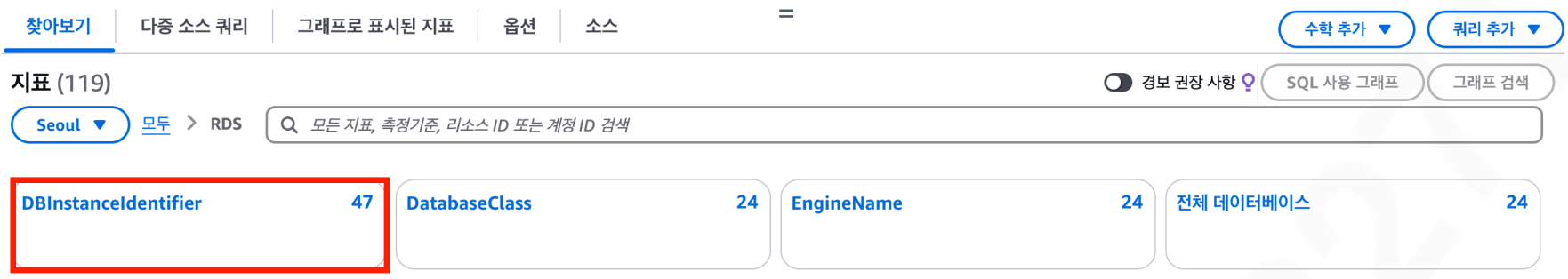
Seoul | 모든 지표, 측정기준, 리소스 ID 또는 계정 ID 검색

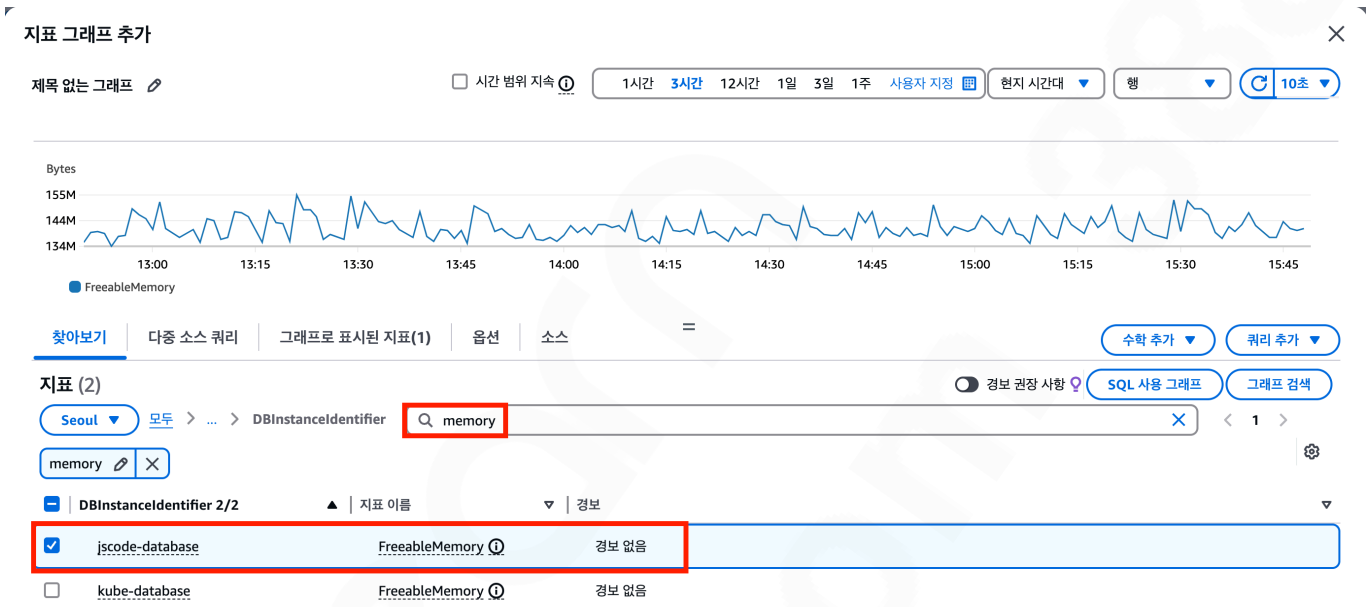
▼ 사용자 지정 네임스페이스

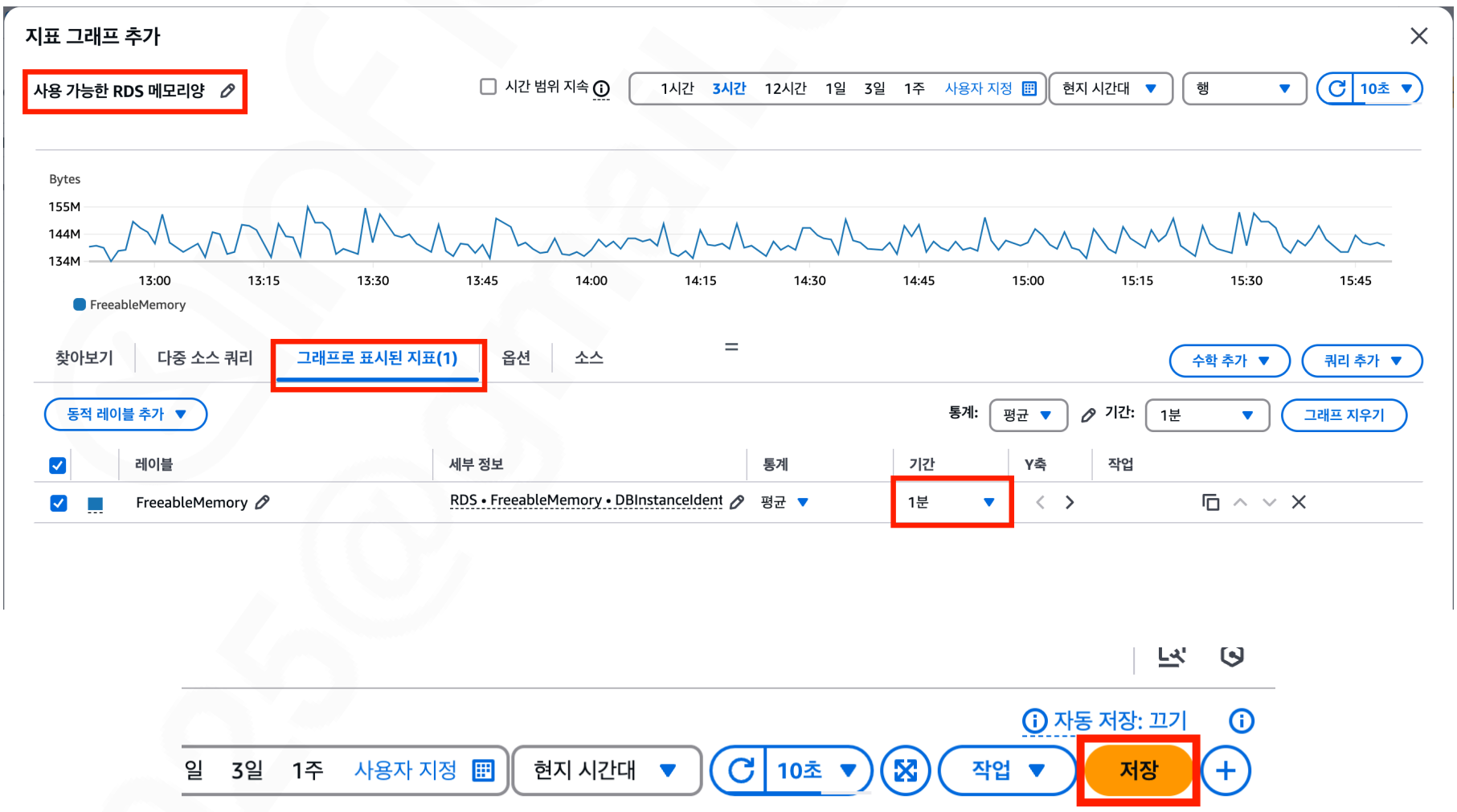
| | |
|---|---|
| CWAgent | 37 |

▼ AWS 네임스페이스

| | | | |
|---|---|---|---|
| ApplicationELB 76 | CertificateManager 1 | EBS 113 | EC2 214 |
| 이벤트 1 | Firehose 6 | IPAM 1 | 로그 3 |
| RDS 119 | S3 10 | AWS/SecretsManager 1 | 사용량 170 |



이제 다시 부하 테스트를 진행해보자. 그러고 어느 리소스에서 병목 지점이 발생하는 지 파악해보자.

# 🎯 [실습] 부하 테스트를 통해 병목 지점 진단하기

## ✅ 부하 테스트 시작하기

1. **기존 k6 스크립트 다시 확인하기**

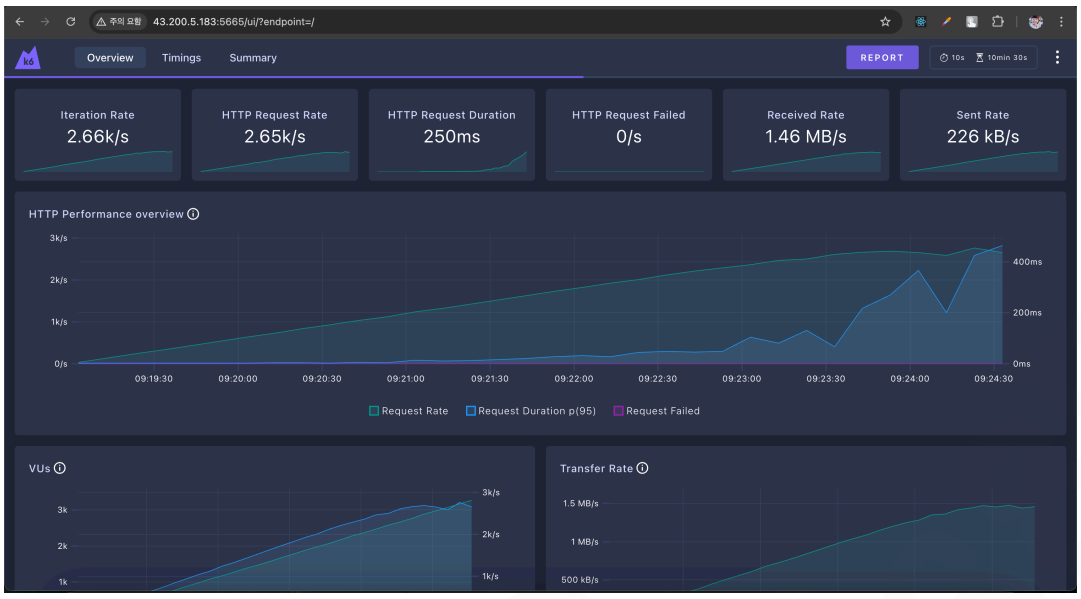
```
import http from 'k6/http';
import { sleep } from 'k6';

export const options = {
  // 부하를 생성하는 단계(stages)를 설정
  stages: [
    // 10분에 걸쳐 vus(virtual users, 가상 유저수)가 50에 도달하도록 설정
    { duration: '10m', target: 50 }
  ],
};

export default function () {
  let random = Math.random();
  // 100명 중 5명의 비율로 게시글을 작성
  if (random < 0.05) {
    const data = { title: '제목', content: '내용' };
    http.post('http://{ELB 주소}/boards', JSON.stringify(data), {
      headers: { 'Content-Type': 'application/json' },
    });

  // 100명 중 95명의 비율로 게시글을 조회
  } else {
    http.get('http://{ELB 주소}/boards');
  }

  // 1초 휴식
  sleep(1);
}
```

2. **부하 테스트 시작하기**

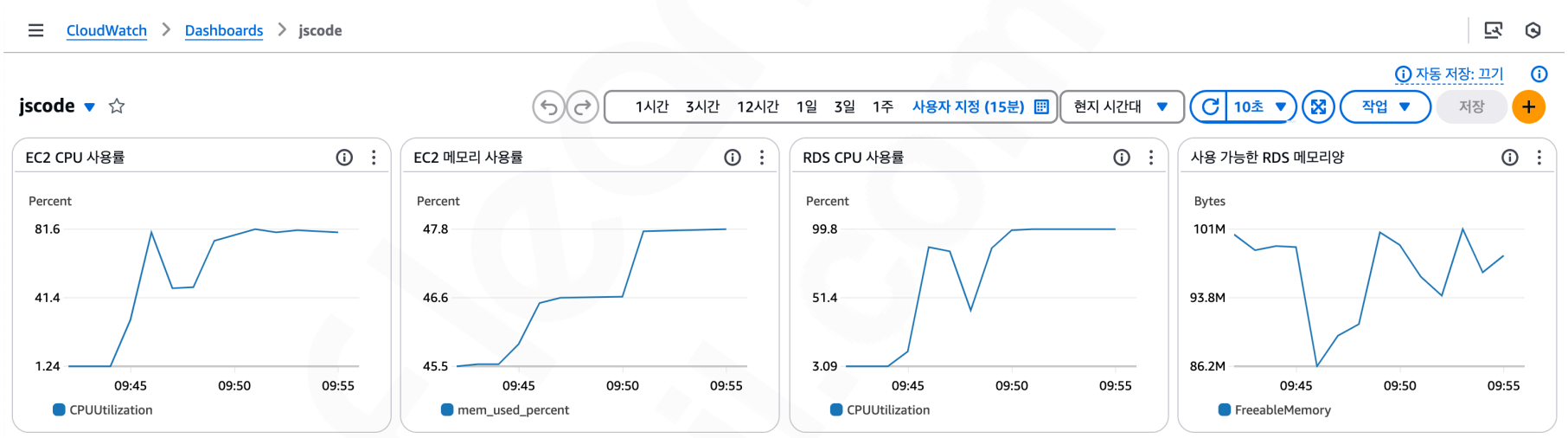
```
$ K6_WEB_DASHBOARD=true k6 run script.js
```

3. **k6의 웹 대시보드 열기**

`http://{k6가 실행되고 있는 EC2 IP 주소}:5665`으로 접속하기

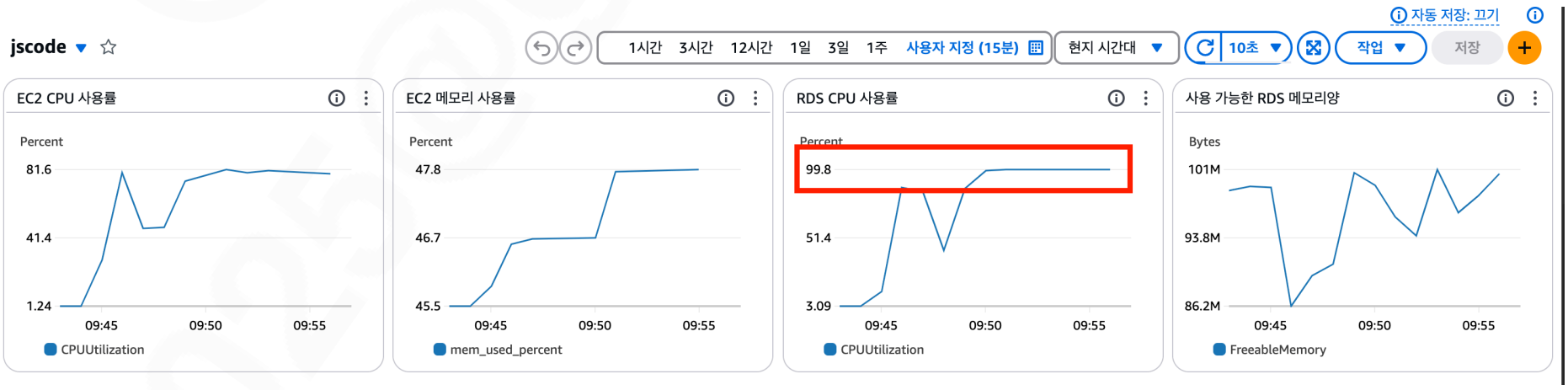



## 4. CloudWatch 대시보드 열기




## ✅ 모니터링 지표를 보고 병목 지점 진단하기

지표를 살펴보면 나머지 리소스들은 괜찮은데 RDS의 CPU 사용률이 100%에 가깝게 측정된 걸 확인할 수 있다. 많은 트래픽을 처리할 때 RDS CPU의 사양이 부족하다는 뜻이다. 즉, RDS가 병목 지점이라는 걸 알 수 있다.



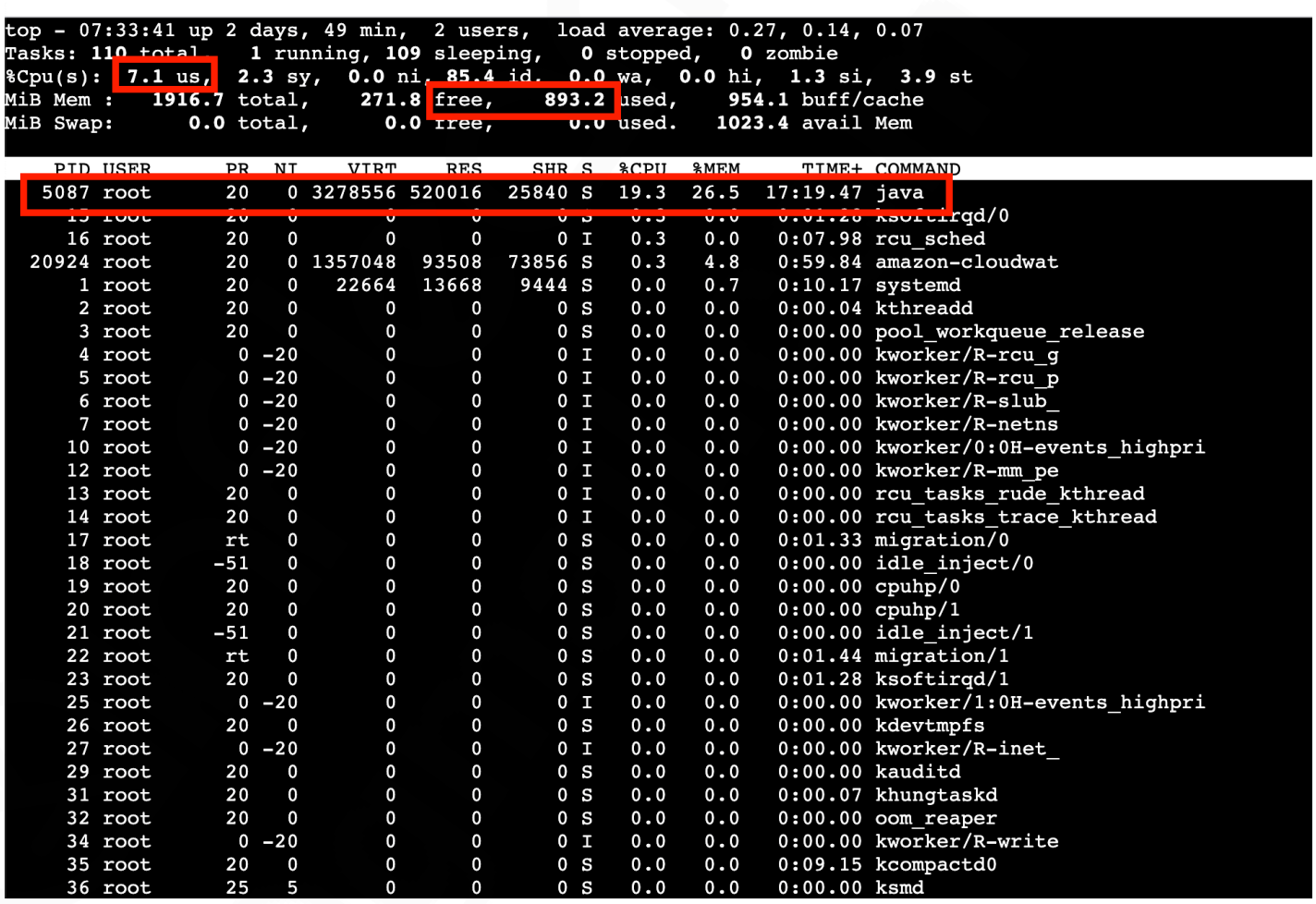
병목 지점이 어디인지 파악했으니 이제 그 병목 지점을 개선해야 한다. 어떤 방식으로 개선해야 하는 지 다음 강의에서 알아보자.

# [보충 강의] 실시간으로 CPU와 메모리를 관측하는 방법

👩🏻‍🏫 CPU, 메모리를 1초 간격으로 관측하고 싶을 수도 있다. 하지만 CloudWatch는 1분 간격으로만 측정을 하기 때문에 한계가 있다. 그래서 실시간으로 CPU, 메모리를 관측할 수 있는 방법을 알려줄 예정이다.

## ✅ 실시간으로 EC2의 CPU와 메모리를 관측하는 방법

```
$ top
```

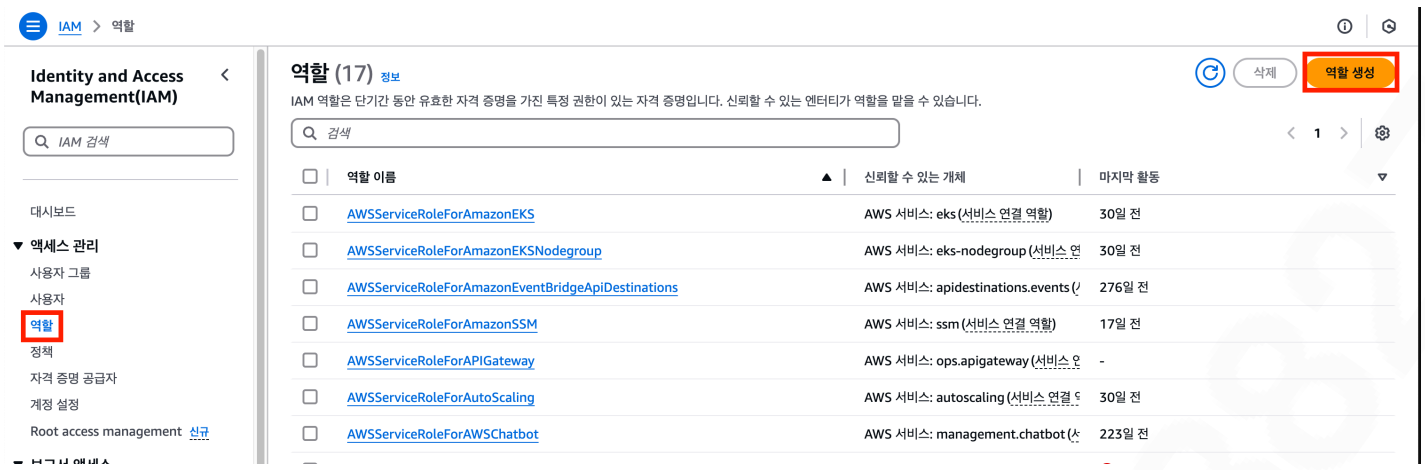



- `7.1 us` : 사용자 프로세스(애플리케이션 프로그램 등)의 CPU 사용률 (%)
- `free, 893.2` : 미사용 메모리 양 (MB)

## ✅ 실시간으로 RDS의 CPU와 메모리를 관측하는 방법

데이터베이스가 설치되어 있는 RDS의 컴퓨터에는 접속할 수 있는 방법이 없다. 따라서 `top` 명령어를 통해 실시간으로 CPU, 메모리를 관측할 수는 없다. 만약 1초 간격으로 RDS의 CPU, 메모리를 관측하고 싶다면 '향상된 모니터링' 옵션을 활용하면 된다. 그러면 CloudWatch에서 1초 간격으로 메트릭을 측정해서 확인할 수 있게 된다.

1. **IAM 역할 생성**




## 신뢰할 수 있는 엔터티 선택 정보

### 신뢰할 수 있는 엔터티 유형

- (●) **AWS 서비스**
  EC2, Lambda 등의 AWS 서비스가 이 계정에서 작업을 수행하도록 허용합니다.
- ( ) **AWS 계정**
  사용자 또는 서드 파티에 속한 다른 AWS 계정의 엔터티가 이 계정에서 작업을 수행하도록 허용합니다.
- ( ) **웹 자격 증명**
  지정된 외부 웹 자격 증명 공급자와 연동된 사용자가 이 역할을 맡아 이 계정에서 작업을 수행하도록 허용합니다.
- ( ) **SAML 2.0 연동**
  기업 디렉터리에서 SAML 2.0과 연동된 사용자가 이 계정에서 작업을 수행할 수 있도록 허용합니다.
- ( ) **사용자 지정 신뢰 정책**
  다른 사용자가 이 계정에서 작업을 수행할 수 있도록 사용자 지정 신뢰 정책을 생성합니다.

### 사용 사례

EC2, Lambda 등의 AWS 서비스가 이 계정에서 작업을 수행하도록 허용합니다.

**서비스 또는 사용 사례**

RDS ▼

지정된 서비스에 대한 사용 사례를 선택합니다.

**사용 사례**

- ( ) **RDS - Directory Service**
  Allows RDS to manage Directory Service resources on your behalf.
- (●) **RDS - Enhanced Monitoring**
  Allows RDS to manage CloudWatch Logs resources for Enhanced Monitoring on your behalf.
- ( ) **RDS - Add Role to Database**
  Allows you to grant RDS access to additional resources on your behalf.
- ( ) **RDS**
  Allows RDS to perform operations using AWS resources on your behalf.
- ( ) **RDS - Beta**
  Allows RDS to perform operations using AWS resources on your behalf in the Beta region.
- ( ) **RDS - Preview**
  Allows RDS Preview to manage AWS resources on your behalf.

취소 다음

## 권한 추가 정보

### 권한 정책 (1) 정보

선택한 역할 유형에는 다음 정책이 필요합니다.

| 정책 이름 | 유형 |
|---|---|
| AmazonRDSEnhancedMonitoringRole | AWS 관리형 |

▶ **권한 경계 설정 - *선택 사항***

취소 이전 다음

## 이름 지정, 검토 및 생성

### 역할 세부 정보

**역할 이름**
이 역할을 식별하는 의미 있는 이름을 입력합니다.

rds-enhanced-monitoring

최대 64자입니다. 영숫자 및 '+=,.@-_' 문자를 사용하세요.

**설명**
이 역할에 대하여 간단한 설명을 추가합니다.

Allows RDS to manage CloudWatch Logs resources for Enhanced Monitoring on your behalf.

최대 문자 수: 1000. 문자(A~Z 및 a~z), 숫자(0~9), 탭, 새 줄 또는 다음 문자 중 하나를 사용합니다: _+=,. @-/\[{}]!#$%^*();:"'`

### 1단계: 신뢰할 수 있는 엔터티 선택

편집

**신뢰 정책**

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "",
            "Effect": "Allow",
            "Principal": {
                "Service": [
                    "monitoring.rds.amazonaws.com"
                ]
            },
            "Action": [
                "sts:AssumeRole"
            ]
        }
    ]
}
```

### 2단계: 권한 추가

편집

**권한 정책 요약**

| 정책 이름 | 유형 | 다음으로서 연결됨 |
|---|---|---|
| AmazonRDSEnhancedMonitoringRole | AWS 관리형 | 권한 정책 |

### 3단계: 태그 추가

**태그 추가 - *선택 사항*** 정보

태그는 리소스를 식별, 정리 또는 검색하는 데 도움이 되도록 AWS 리소스에 추가할 수 있는 키-값 페어입니다.

리소스와 연결된 태그가 없습니다.

새 태그 추가

최대 50개의 태그를 더 추가할 수 있습니다.

취소 이전 역할 생성

## 2. 향상된 모니터링 활성화하기

RDS > 데이터베이스 > jscode-database

Amazon RDS

- 대시보드
- 데이터베이스
- 쿼리 편집기
- 성능 개선 도우미
- 스냅샷
- Amazon S3에서 내보내기
- 자동 백업
- 예약 인스턴스
- 프록시

- 서브넷 그룹
- 파라미터 그룹
- 옵션 그룹
- 사용자 지정 엔진 버전
- 제로 ETL 통합 신규

### jscode-database

수정 작업

**요약**

| DB 식별자 | 상태 | 역할 | 엔진 | 권장 사항 |
|---|---|---|---|---|
| jscode-database | 사용 가능 | 인스턴스 | MySQL Community | 2 유용 |

| CPU | 클래스 | 현재 활동 | 리전 및 AZ |
|---|---|---|---|
| 97.07% | db.t4g.micro | 10 연결 | ap-northeast-2c |

연결 및 보안 | 모니터링 | 로그 및 이벤트 | 구성 | 제로 ETL 통합 | 유지 관리 및 백업 | 데이터 마이그레이션 - *신규* | 태그 | 권장 사항

**새 모니터링 보기 사용 가능**
이제 RDS가 성능 개선 도우미 및 CloudWatch 지표를 포함하는 새로운 모니터링 보기를 지원합니다. 새로운 모니터링 보기에 액세스하려면 **수정**을 선택하여 데이터베이스를 수정하고 성능 개선 도우미를 켭니다.

수정

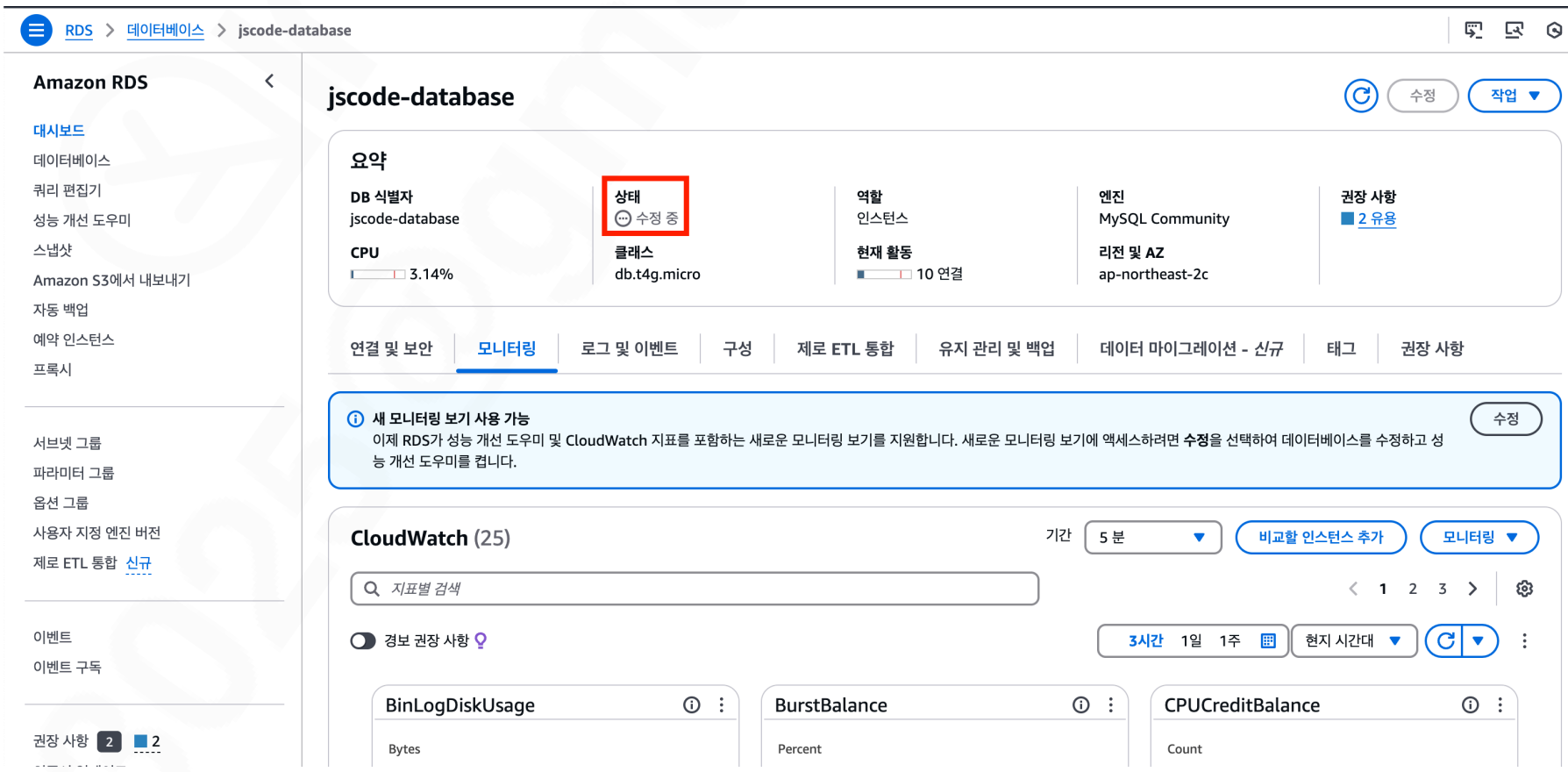
**CloudWatch (25)**

기간 5 분 비교할 인스턴스 추가 모니터링

지표별 검색

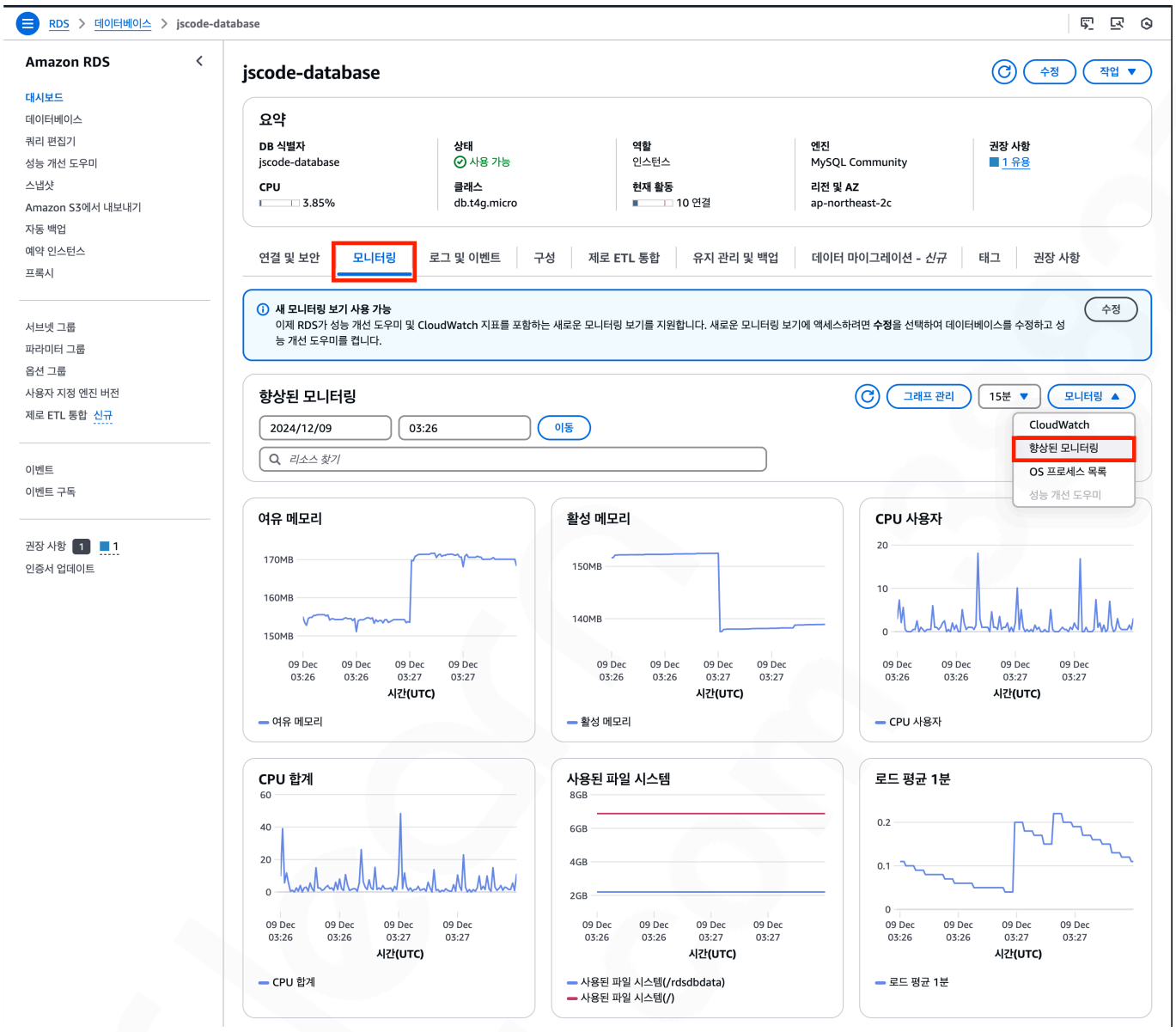
1 2 3

**참고)** 메트릭을 자주 수집할 수록 비용이 증가할 수 있으니 유의하자.




**참고)** 설정값이 반영되는 데 어느 정도 시간이 걸린다.

### 3. 1초 간격으로 메트릭을 수집하는 지 확인하기




👩🏻‍🏫 이 강의에서는 RDS의 '향상된 모니터링' 기능을 쓸 필요가 없으니 비용 절약을 위해 해제해두자.

khb3025@gmail.com

📖

# 가용성(Availability), 시스템 이중화

👩🏻‍🏫 대규모 트래픽을 처리하기 위해 시스템을 고도화해야 한다. 고도화 할 때 성능(Throughput, Latency)만 신경쓰면 안 된다. '**가용성(Availability)**'도 같이 신경쓰면서 시스템을 고도화해야 한다. 그럼 지금부터 **가용성(Availability)**이 뭔지 알아보자.

## ✅ 가용성(Availability)

**가용성(Availability)**이란 **시스템이 서비스를 정상적으로 제공할 수 있는 가능성**을 의미한다. 서비스에 장애가 발생할 가능성이 극히 작은 시스템을 가용성이 높은 시스템이라고 말할 수 있다. 반대로 서비스가 다운되는 시간이 긴 시스템을 가용성이 낮은 시스템이라고 한다.

가용성이 높고 낮음은 서비스의 정상 가동률(%)로 표시된다. 가용성 99.99%라는 경우 99.99% 시간을 정상적으로 이용 가능한 시스템을 말한다. 이를 바꿔 말하면 1년에 53분 정도는 서비스가 다운된다고 말할 수 있다.

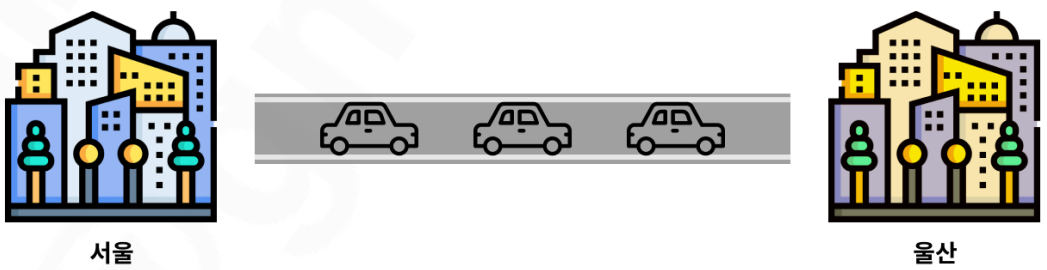
**[높은 가용성을 가진 시스템 설계 방법]**

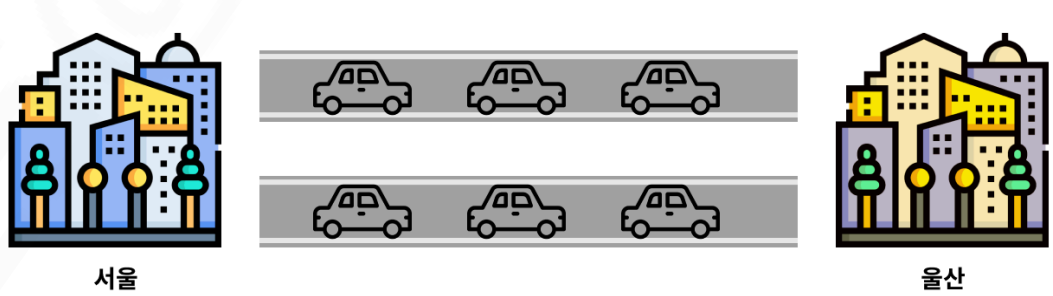
가용성을 높이기 위해서는 서비스 사용 불가능 시간을 최대한 발생시키지 않게 하고 설령 발생하더라도 그 시간을 짧게 만들어야 한다. 다운 타임을 줄이는 데 가장 중요한 설계 방법 중 하나가 **시스템 이중화**이다.

## ✅ 시스템 이중화

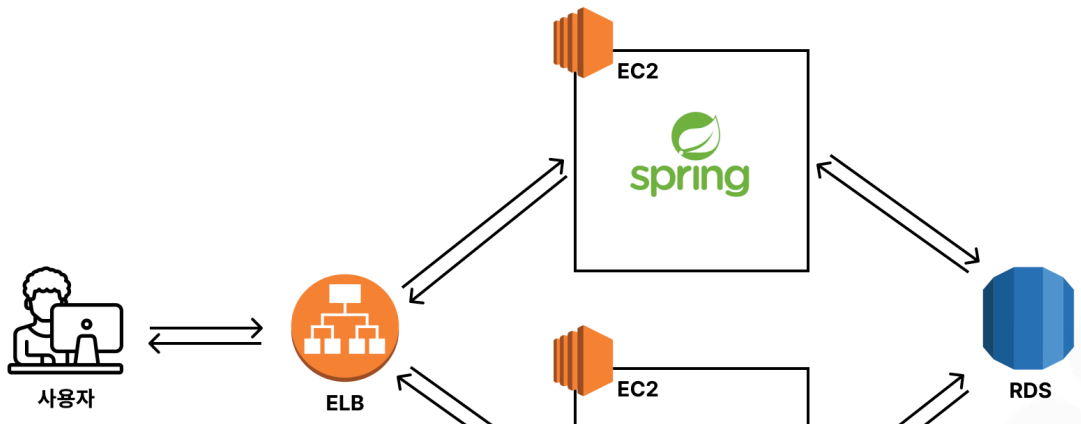
**시스템 이중화**라는 건 시스템의 일부분을 사용할 수 없게 되어도 다른 시스템을 이용하여 서비스를 계속 이용할 수 있게 만드는 걸 의미한다.

예를 들어, 도시를 연결하는 도로가 하나가 아닌 두 개가 있다면 둘 중 하나의 도로에서 사고가 발생하더라도 다른 도로를 이용하여 수송할 수 있다. 또한, 두 개의 도로가 존재함으로써 도로 공사 중에도 정상적으로 수송을 할 수 있다.




시스템에서 이중화는 아래와 같이 설계할 수 있다. 이와 같이 이중화 설계를 하면 다운 타임을 줄일 수 있게 된다. 즉, 서비스의 가용성을 높일 수 있게 된다.



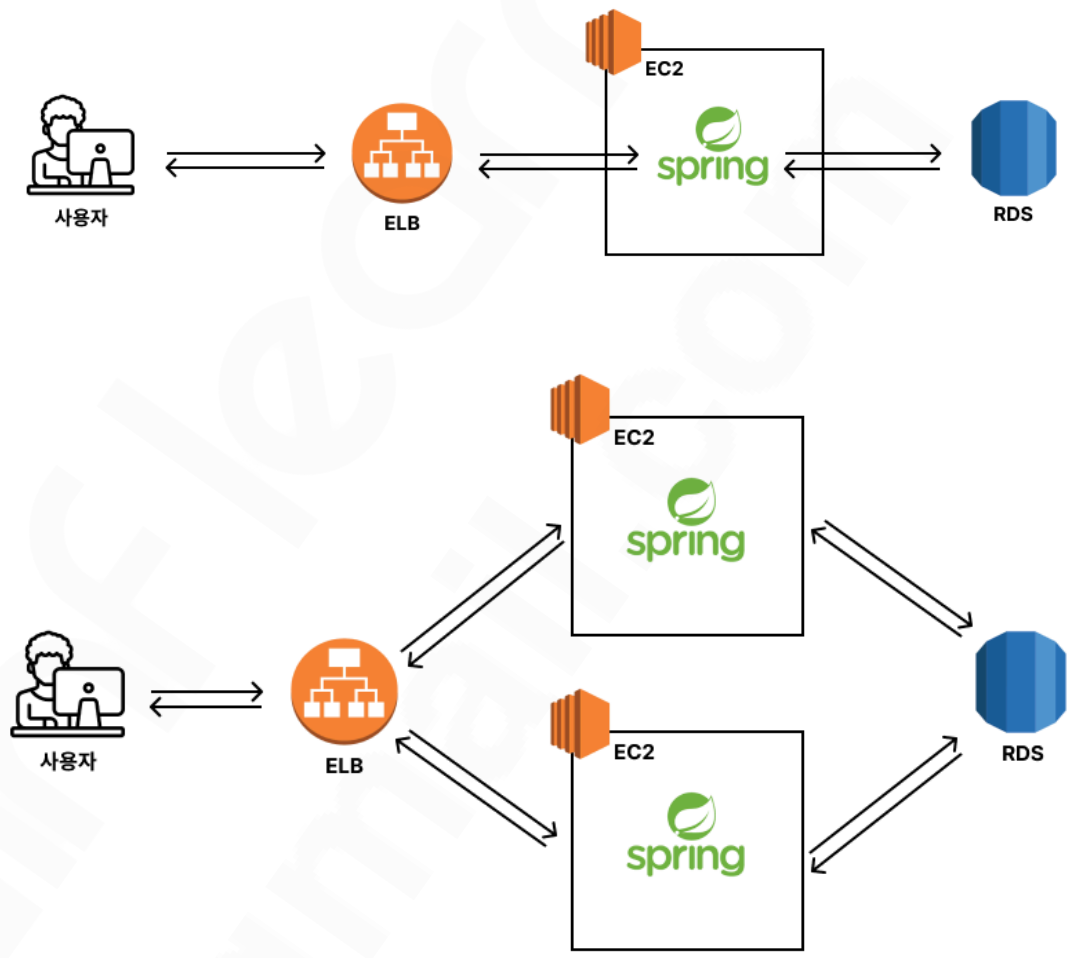

khb302571@gmail.com

# 수평적 확장, 수직적 확장, 캐싱

👩🏻‍🏫 시스템의 성능을 개선할 때 가장 많이 활용하는 3가지 전략을 알아보자.

## ✅ 수평적 확장

특정 시스템 성능을 올리기 위해 **시스템 개수를 늘리는 걸** **수평적 확장**이라고 한다.




**[장점]**

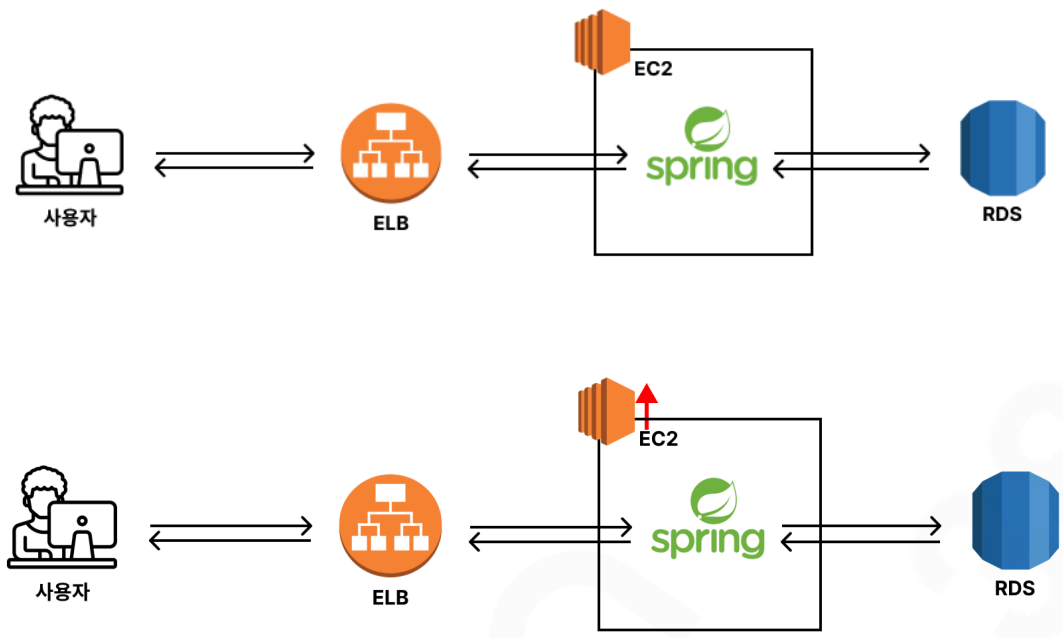
- 시스템 이중화 → 가용성 증가
- 확장에 제한이 없음
- 서버 축소/확장이 쉬움

**[단점]**

- 여러 대의 서버를 한 서버처럼 사용하기 위한 추가 작업이 필요함
  - ex) 로드 밸런서

## ✅ 수직적 확장

특정 시스템 성능을 올리기 위해 시스템 개수 변경 없이 **시스템의 스펙(CPU, 메모리 등)을 업그레이드 하는 방식**을 **수직적 확장**이라고 한다.




**[장점]**

- 여러 대의 서버를 한 서버처럼 사용하기 위한 추가 작업이 필요 없음
    - 심플하게 인프라 구성을 할 수 있어 관리 비용(시간, 노력, 비용)이 적게 듬

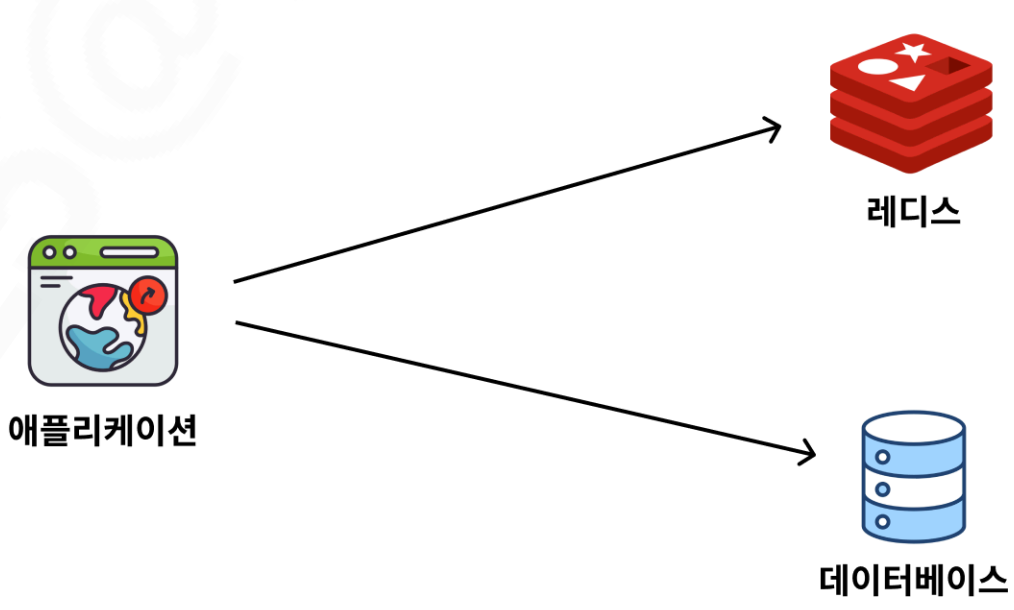
**[단점]**

- 시스템 이중화 X → 가용성 감소
- 시스템 스펙에는 제한이 있으므로 확장에 제한이 생김
- 서버 축소/확장을 할 때마다 서비스 중단(다운 타임)이 불가피함 → 가용성 감소

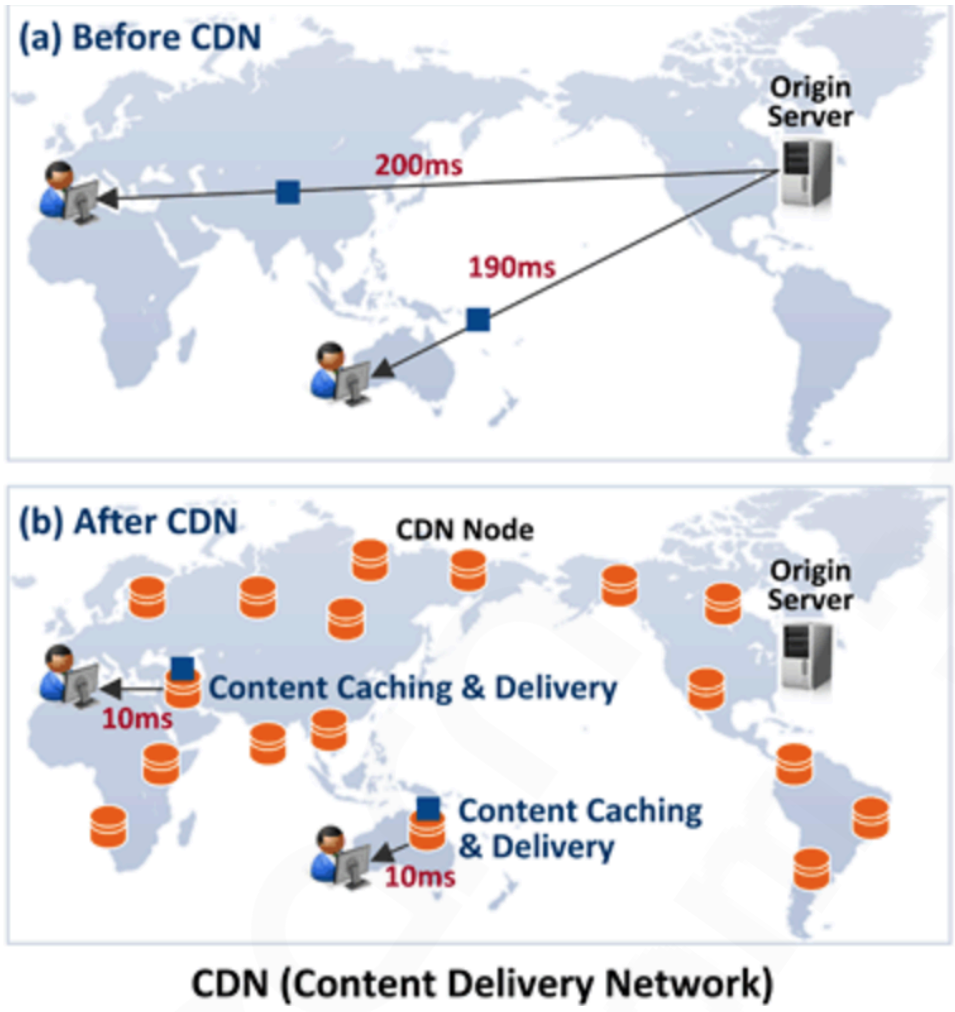
## ✅ 캐싱(Caching)

**캐싱(Caching)**이란 **<u>데이터를 더 빠르게 액세스(조회)할 수 있는 곳에 임시로 저장하는 방식</u>**을 의미한다.

**예시 1)**




**예시 2)**




**CDN (Content Delivery Network)**

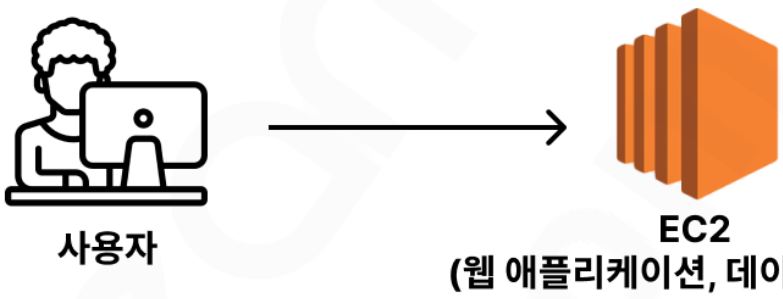
출처 : https://mygumi.tistory.com/67

khb3025@gmail.com

# 트래픽 증가에 따른 시스템 설계 및 확장 방법

부하 테스트를 하면서 목표로 설정해놓은 Throughput, Latency를 달성하기 위해 성능 개선을 해야 한다. 병목 지점이 어디서 발생하느냐에 따라 성능 개선의 방법이 달라진다. 이 방법에 대해 대략적으로 알아놓아야 성능 개선을 수월하게 할 수 있게 된다.

## ✅ 가장 간단한 형태




EC2 서버 한 대에 프론트엔드, 백엔드, 데이터베이스에 관련된 프로그램을 다 실행시키는 형태의 구성이다. 이 형태의 장점은 하나의 서버에서 모든 리소스를 관리하기 때문에 관리 및 조작하기가 심플하다. 또한 다양한 리소스를 쓰지 않기 때문에 비용이 적게 나오는 편이다.

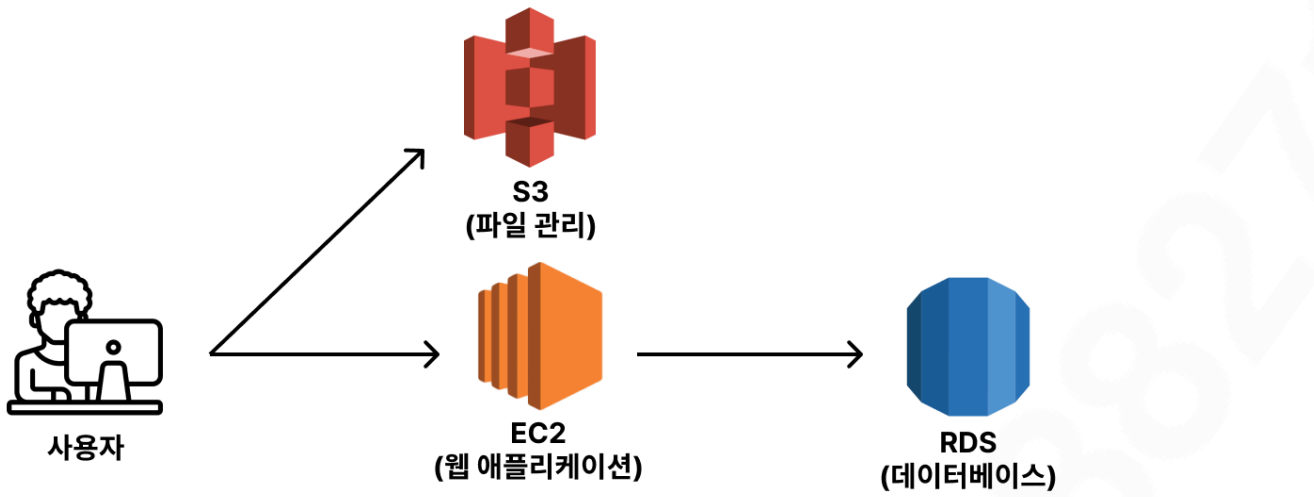
하지만 위의 구성으로 실제 서비스를 운영하다보면 데이터베이스가 생각보다 많은 컴퓨팅 자원(CPU, 메모리, 디스크)을 사용한다. 데이터베이스로 인해 웹 애플리케이션의 성능에 악영향을 줄 수 있기 때문에 별도의 서버를 분리하는 형식을 많이 가져간다.

## ✅ 데이터베이스 분리

웹 서버와 데이터베이스를 분리한 인프라 구성이다. 위 인프라 구성에서 트래픽이 많아지면 정적 파일(이미지, 비디오, 웹 페이지 등)을 제공하는 부분에서 문제가 될 가능성이 크다. 일반적으로 정적 파일은 용량이 크기 때문에 컴퓨팅 자원을 많이 소모해서 서버에 과부하가 걸릴 가능성이 크다.

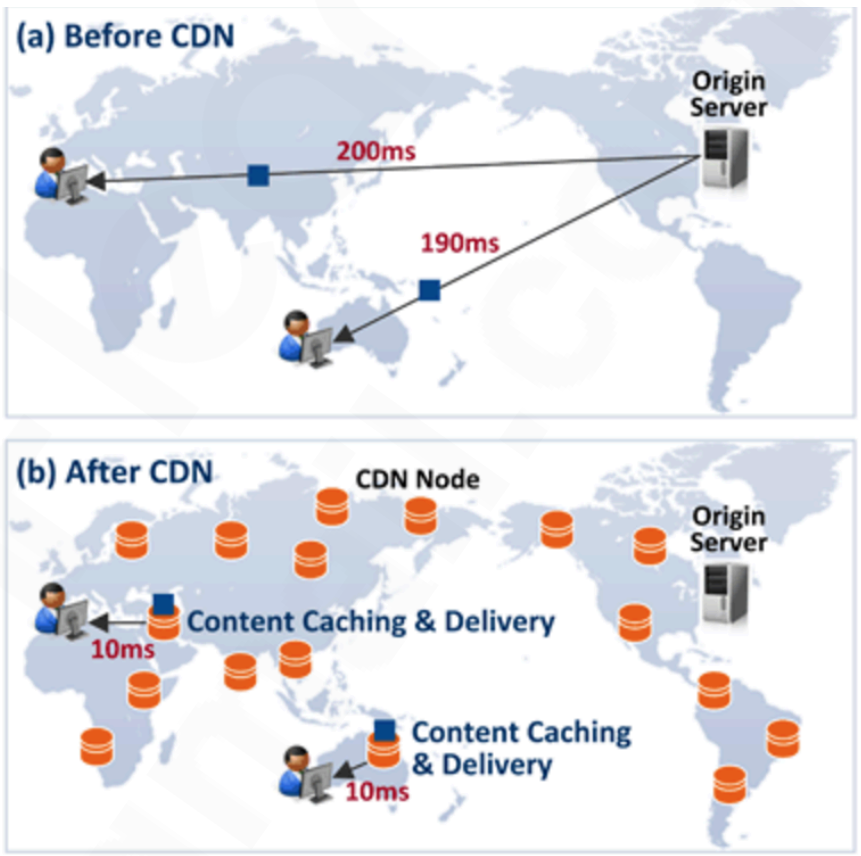
이를 해결하기 위해 정적 파일을 제공하는 서버만 별도로 분리하는 구성을 많이 가져간다.

## ✅ 정적 파일 서버 분리




S3를 활용해 정적 파일만을 제공하는 별도의 서버를 구축했다. 하지만 정적 파일은 용량이 큰 경우가 많기 때문에, 정적 파일을 제공받는 곳이랑 거리가 멀면 멀수록 응답 속도가 오래 걸릴 수 밖에 없다.

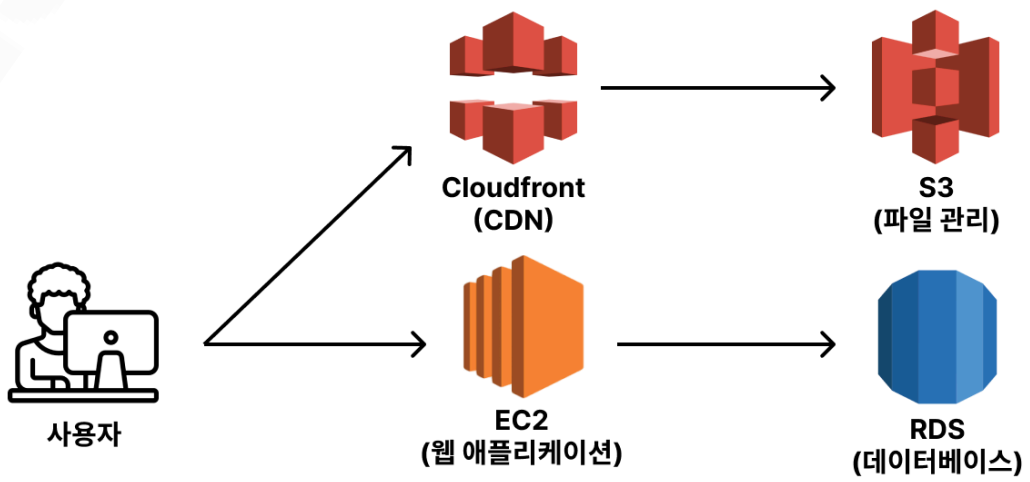
이를 해결하기 위해 캐싱의 원리가 적용된 CDN을 활용해서 정적 파일 전송 속도를 향상시킨다.




CDN (Content Delivery Network)

출처 : https://mygumi.tistory.com/67

## ✅ CDN 서버 활용

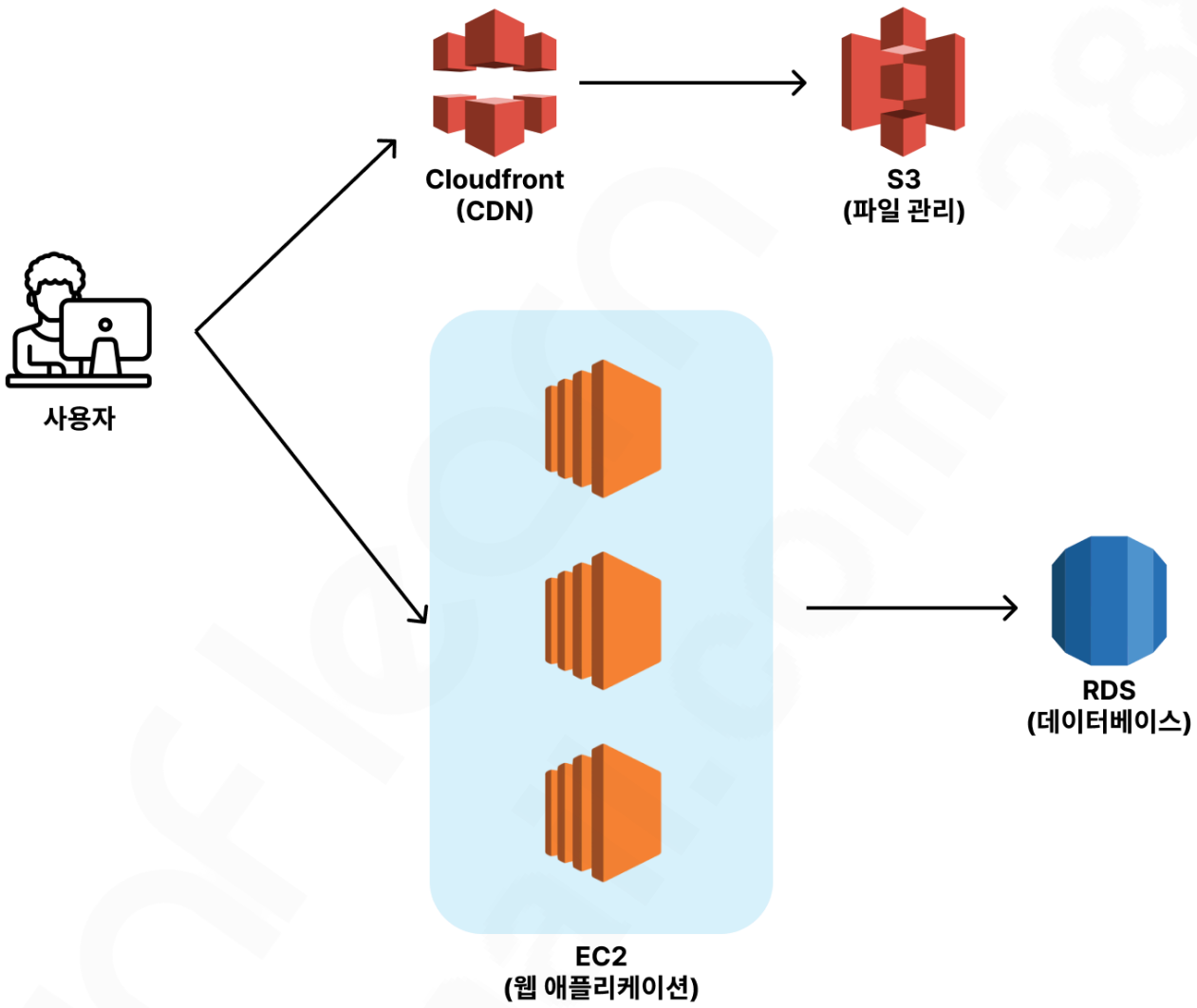



이 구성에서 사용자 요청이 더 많아져서 EC2 인스턴스 한 대로 모든 요청을 처리할 수 없는 상황이 왔다고 가정하자. 이런 경우에는 EC2를 확장해야 한다.

확장 방식에는 수직적 확장과 수평적 확장의 방식이 있다는 걸 배웠다. 하지만 시스템 이중화의 장점 때문에 EC2를 확장할 때는 수평적 확장의 방식을 많이 활용한다.

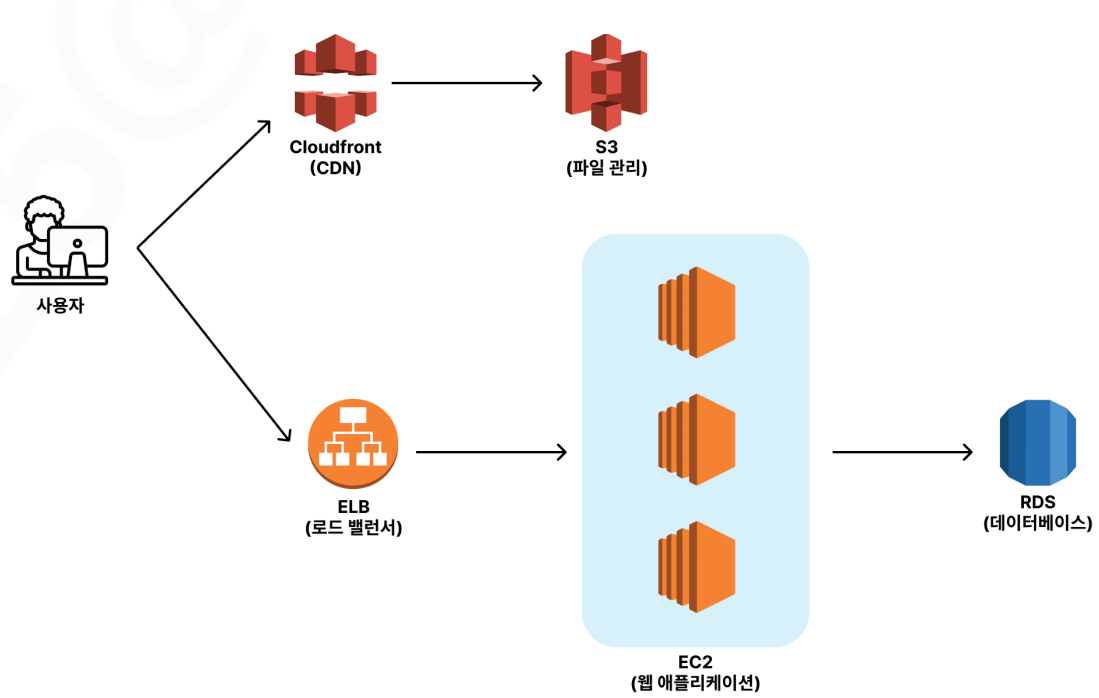
(수직적 확장도 동시에 적용시키는 경우도 많다.)

## ✅ 웹 애플리케이션 서버의 수평적 확장




수평적 확장의 방식으로 웹 애플리케이션 서버(EC2)를 여러 대로 늘렸다. 하지만 사용자보고 여러 서버에 골고루 알아서 요청을 보내라고 시킬 수는 없다. 사용자의 요청을 여러 대의 웹 애플리케이션 서버에 골고루 전달하기 위한 장치가 필요하다. 그게 바로 로드밸런서(ELB)이다.

## ✅ 로드 밸런서 도입




로드 밸런서를 도입함으로써 수평적 확장을 한 여러 대의 EC2에 골고루 트래픽을 분산시킬 수 있게 됐다. 하지만 늘어난 웹 애플리케이션 서버에 따라 많은 수의 요청이 DB에 몰리게 된다.

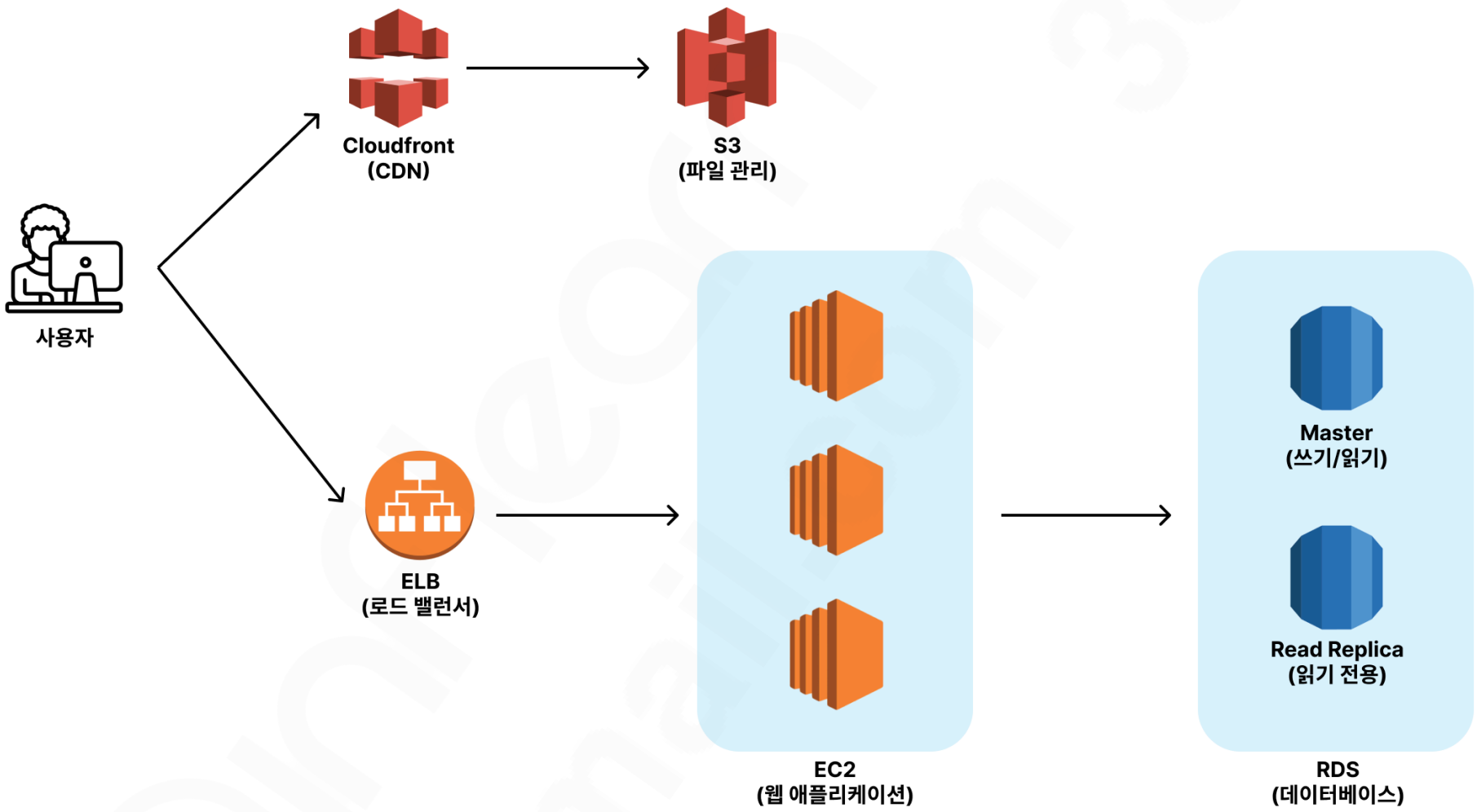
DB에서 병목 현상이 발생하면 DB 자체적으로 성능 개선할 수 있는 부분은 없는 지를 먼저 고려한다. DB 자체적으로 성능 개선하는 방법에는 인덱스 활용, 역정규화, SQL문 튜닝 등이 있다. 이 방식으로 최대한 개선을 했는데도 DB에 병목 현상이 발생할 수도 있다. 그러면 수평적 확장

의 방식을 고려해봐야 한다.

하지만 DB에 수평적 확장의 방식을 적용시키는 건 어려운 점이 많다. 왜냐하면 기존에 저장되어 있는 데이터를 수평적 확장한 모든 DB에 복사해서 관리해야 한다. 데이터의 변경이 일어날 때마다 여러 대의 DB가 동기화 작업을 해야 한다. 동기화 작업은 DB 성능 저하를 유발하기 때문에 오히려 장점보다 단점이 더 큰 확장 방식이라고 판단했다. 이 때문에 DB는 수평적 확장보다는 수직적 확장의 방식으로 성능을 개선한다.

만약 DB를 최대한으로 수직적 확장을 했음에도 추가적인 성능 개선이 필요하다면 읽기 전용 데이터베이스(Read Repica) 도입을 고려한다.

## ✅ 읽기 전용 데이터베이스(Read Replica) 도입

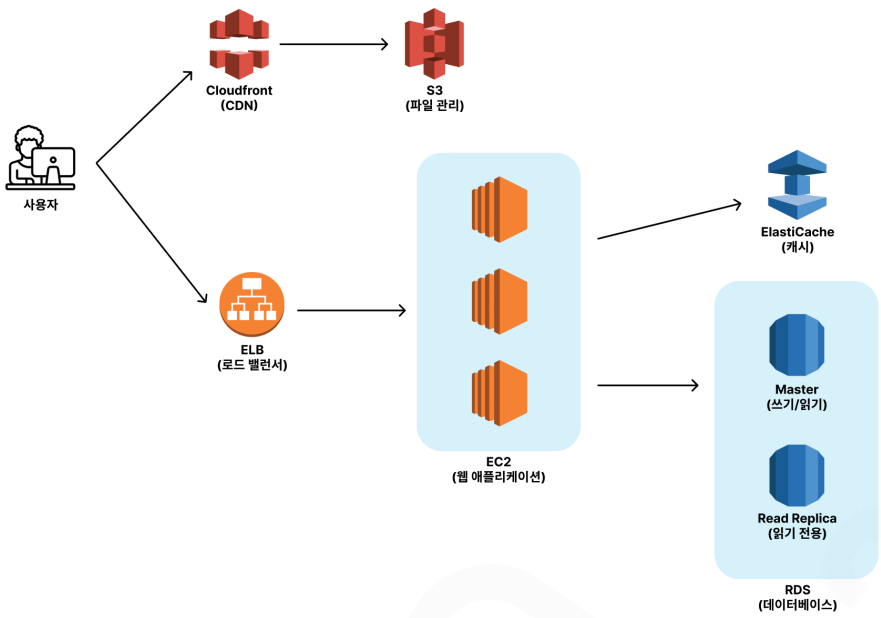



DB를 수평적 확장을 하지 못했던 이유는 데이터의 동기화의 어려움 때문이었다. 하지만 실제 서비스에서 사용하는 쿼리 중 일부는 정밀한 데이터 동기화가 필요 없는 경우가 많다. 따라서 정밀한 데이터 동기화가 필요 없는 쿼리를 실행시킬 때 읽기 전용 데이터베이스(Read Replica)를 많이 활용한다.

원래 하나의 데이터베이스가 모든 트래픽을 처리했어야만 했는데, 읽기 전용 데이터베이스(Read Replica)를 도입함으로써 트래픽을 나눠서 처리할 수 있게 됐다.

이와 같은 방식으로 고사양의 데이터베이스를 사용하다보면 문제가 되는 게 비용이다. AWS의 다양한 서비스 중 RDS(데이터베이스)는 비싼 자원에 속한다. 그래서 비용 절감과 성능 향상을 위해 캐시 서버를 많이 활용한다.

## ✅ 캐시 서버 도입




데이터를 조회할 때 항상 DB로 요청을 보냈었다면, 캐시 서버를 도입함으로써 데이터 조회 요청의 응답을 DB 서버와 캐시 서버가 나눠서 처리할 수 있게 된다. 즉, 캐시 서버를 활용해서 DB 부하를 줄일 수 있다.

## ✅ 병목 지점에 따른 성능 개선 방법

> 이 정도의 시스템 설계 및 확장 방법만 알고 있어도 충분하다. 이번에는 병목 지점의 관점으로 성능 개선 방법을 정리해보자.

1. **EC2(웹 애플리케이션 서버)가 병목 지점일 경우**
    a. 애플리케이션 로직에서 비효율적인 부분 개선하기
    b. 정적 파일 서버(S3, Cloudfront) 분리하기
    c. 로드밸런서(ELB)를 활용해 수평적 확장하기
    d. 수직적 확장하기

2. **RDS(데이터베이스 서버)가 병목 지점일 경우**
    a. 비효율적인 쿼리 개선하기 (인덱스 활용, SQL문 튜닝, 역정규화 등)
    b. 수직적 확장하기
    c. 읽기 전용 데이터베이스(Read Replica) 도입하기
    d. 캐시 서버 도입하기

khb3023@gmail.com

# 🎯 [실습] 병목 지점(DB) 해결하기

🎯 [실습] 내가 구성한 인프라는 1초당 몇 개의 요청을 견뎌낼 수 있을까?

🎯 [실습] 부하 테스트를 통해 병목 지점 진단하기

이전 실습에서 부하 테스트를 했을 때 Throughput이 3 TPS임을 측정했고, RDS가 병목 지점이라는 것까지 확인했다. 지금부터 RDS의 병목 지점을 해결해서 성능 개선을 시도해볼 것이다.

## ✅ 인덱스를 활용해 RDS 성능 개선

1. **성능 개선 방향성 판단하기**

   DB를 성능 개선 할 때는 크게 4가지 방향성이 있다고 설명했었다. 그 중 비효율적인 쿼리 개선을 가장 먼저 시도해봐야 한다.

   a. **비효율적인 쿼리 개선하기 (인덱스 활용, SQL문 튜닝, 역정규화 등)**
   b. 수직적 확정
   c. 읽기 전용 데이터베이스(Read Replica) 도입하기
   d. 캐시 서버 도입하기

2. **인덱스 생성하기**

   Spring Boot 프로젝트에 있는 `GET /boards 게시글 조회`에서 `created_at` 컬럼을 기준으로 정렬한 뒤에 조회한다. 조금 더 빠르게 조회하게 만들기 위해 `created_at` 컬럼을 기준으로 인덱스를 생성해주자.

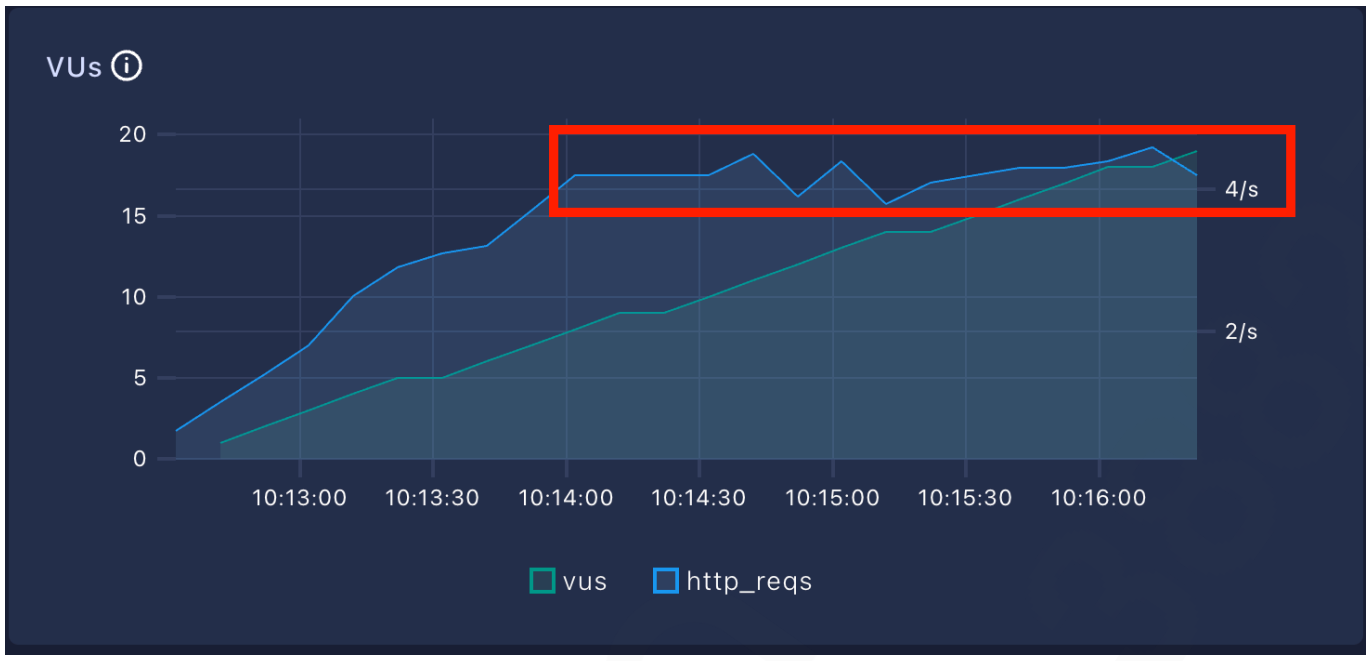
   (SQL 튜닝에 관련된 강의가 아니기 때문에 자세한 설명은 생략한다. 이 부분에 대해 자세히 공부해보고 싶다면 **이 강의**를 참고하자.)

   ```
   CREATE INDEX idx_created_at ON boards (created_at);
   SHOW INDEX FROM boards;
   ```

2. **부하 테스트 시작하기**

   ```
   $ K6_WEB_DASHBOARD=true k6 run script.js
   ```

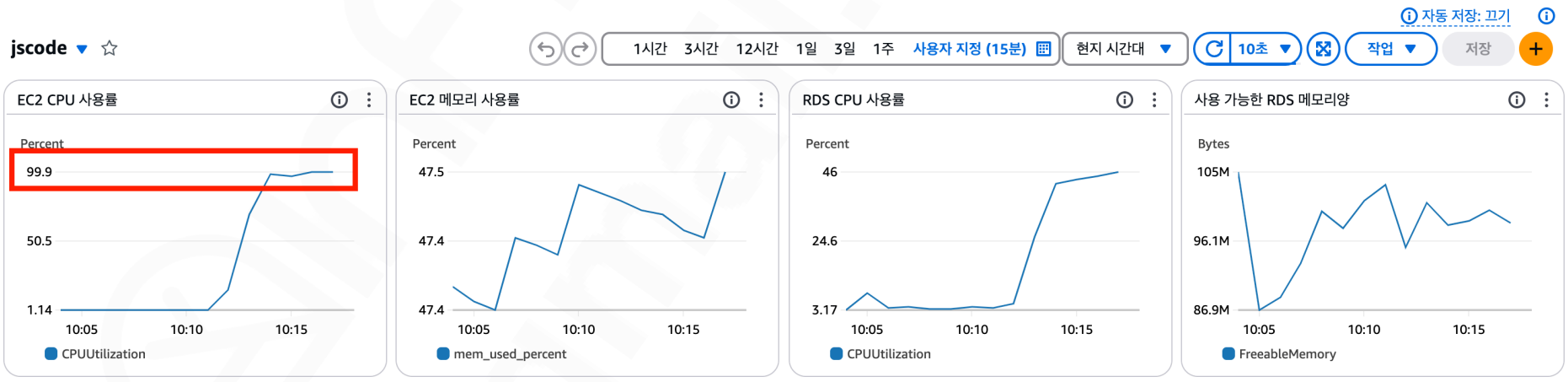
3. **k6의 웹 대시보드 확인하기**




최대 Throughput은 약 **4 TPS** 정도로 측정된다. 기존 Throughput(3 TPS)에 비해 1.25배 정도 증가했다. 즉, 1초에 처리할 수 있는 요청량이 3개에서 4개로 증가했다는 뜻이다.

## ✅ 추가적인 성능 개선

원래 설정해둔 목표 Throughput이 4 TPS보다 더 높았다고 가정하자. 그러면 추가적인 성능 개선이 더 필요하다. 성능 개선을 하기 위해서 가장 먼저 해야 하는 건 '**병목 지점을 찾는 것**'이다. 모니터링 지표를 살펴보니 이번에는 EC2의 CPU 사용률만 100%에 가깝게 사용되고 있었다. 즉, 병목 지점이 EC2라는 걸 알 수 있다.




> 다음 강의에서 EC2의 병목 지점을 해결해보자.

khb302500

# 🎯 [실습] 병목 지점(웹 애플리케이션 서버) 해결하기

## ✅ EC2의 성능 개선 방향 잡기

병목 지점이 EC2라는 건 확인했다. EC2를 성능 개선할 때는 크게 4가지 방향성이 있다고 설명했다. a와 b에 대해서는 해당 사항이 없으므로, 시스템 이중화의 장점을 가지고 있는 '**수평적 확장**'의 방식으로 성능을 개선해보자.

그 중에서 시스템 이중화의 이점을 가져가기 위해 수평적 확장을 시도해보자.

a. 애플리케이션 로직에서 비효율적인 로직 개선하기

b. 정적 파일 서버(S3, Cloudfront) 분리하기

c. 로드밸런서(ELB)를 활용해 수평적 확장하기

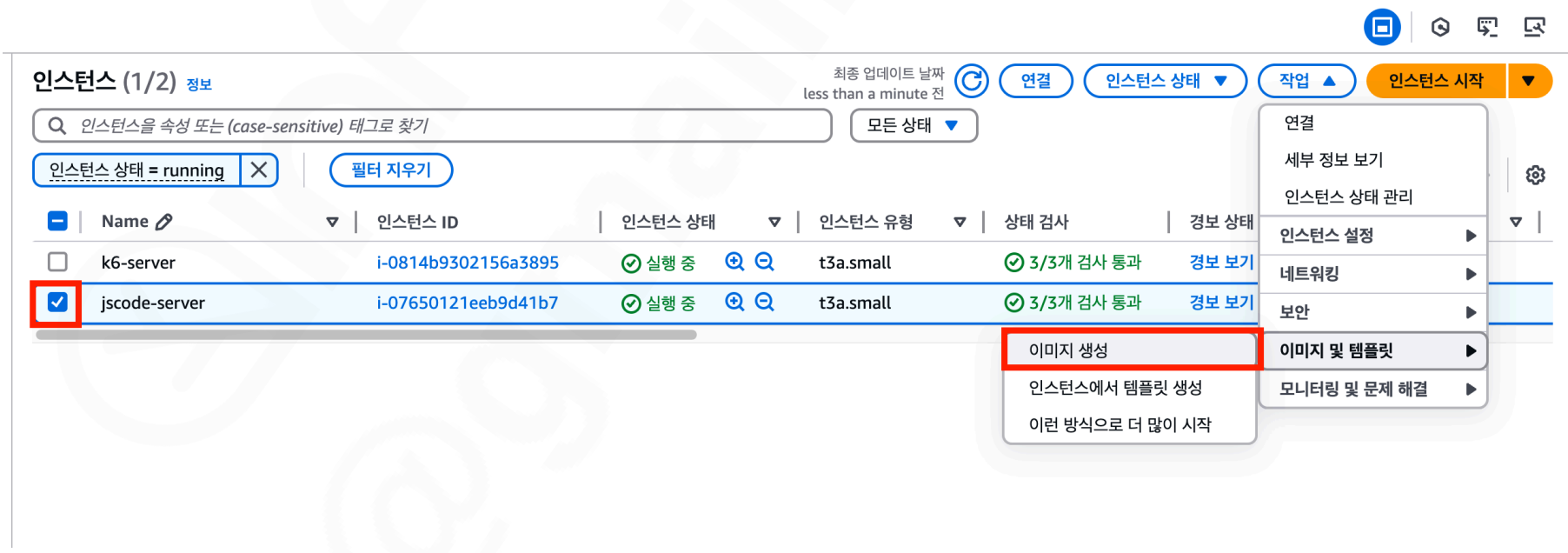
d. 수직적 확장하기

## ✅ EC2 수평적 확장하기

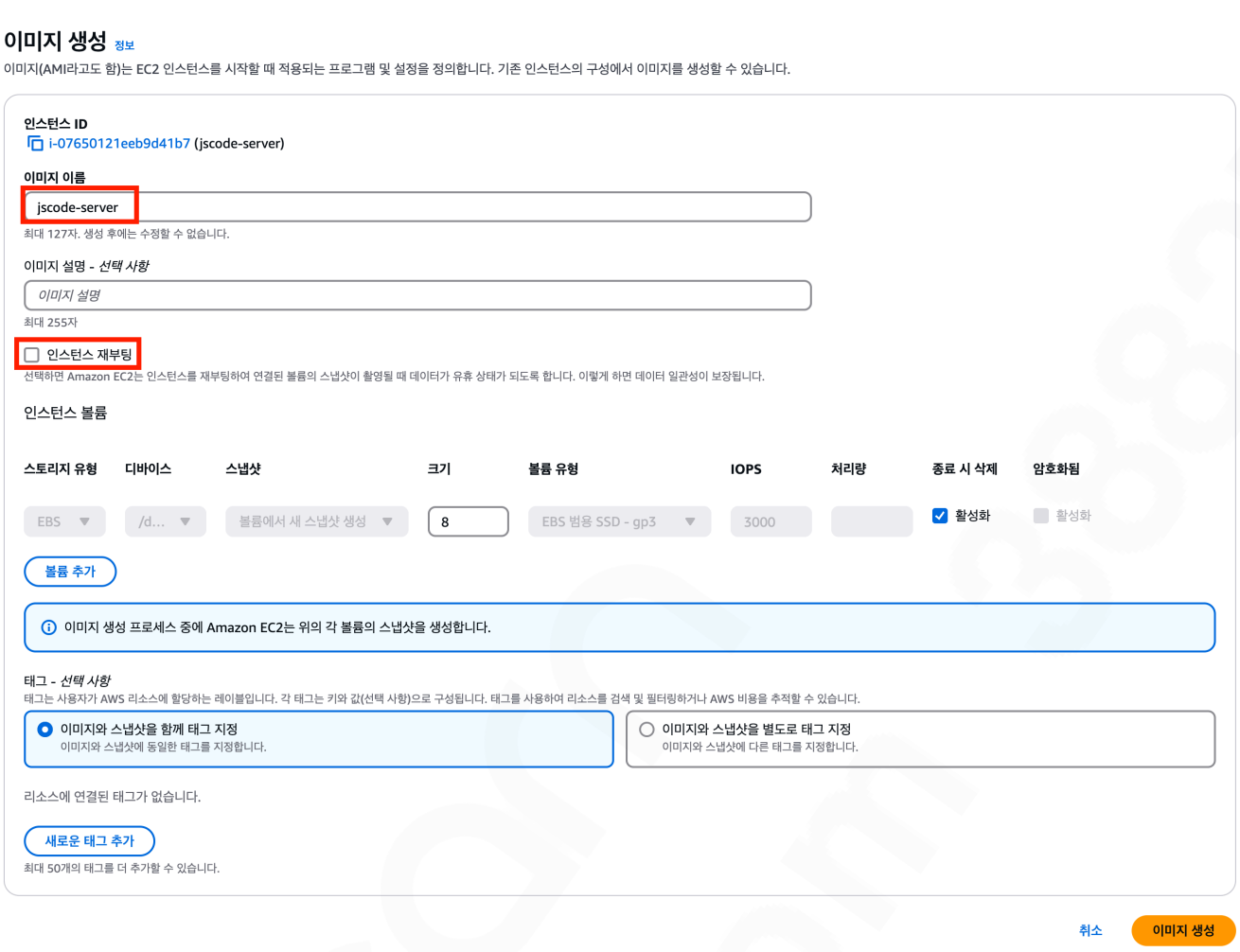
**참고)** EC2에는 오토 스케일링(Auto Scaling)이라는 기능을 활용해 트래픽에 따라 자동으로 수평적 확장을 해주는 기능이 있다. 하지만 학습을 위해 수동으로 수평적 확장을 할 예정이다.

1. **이미지 생성하기**

   **이미지(Image)**란 '**기존 EC2 인스턴스의 복제본**'이라고 생각하면 이해하기 쉽다.




khb3025@gmail.com




## 2. 이미지로 EC2 인스턴스 생성하기

khb3025@gmail.com

## Launch an instance 정보

Amazon EC2를 사용하면 AWS 클라우드에서 실행되는 가상 머신 또는 인스턴스를 생성할 수 있습니다. 아래의 간단한 단계에 따라 빠르게 시작할 수 있습니다.

### 이름 및 태그 정보

**이름**

jscode-server2

추가 태그 추가

### ▼ 애플리케이션 및 OS 이미지(Amazon Machine Image) 정보

AMI는 인스턴스를 시작하는 데 필요한 소프트웨어 구성(운영 체제, 애플리케이션 서버 및 애플리케이션)이 포함된 템플릿입니다. 아래에서 찾고 있는 항목이 보이지 않으면 AMI를 검색하거나 찾아보세요.

*수천 개의 애플리케이션 및 OS 이미지를 포함하는 전체 카탈로그 검색*

카탈로그의 AMI | 최근 사용 | 내 AMI | Quick Start

**이름**

jscode-server

**설명**

-

**이미지 ID**

ami-0373ed9399926f26d

**사용자 이름**

root

| 게시됨 | 아키텍처 | 가상화 | 루트 디바이스 유형 | ENA 활성화됨 |
|---|---|---|---|---|
| 2024-12-11T01:36:13.000Z | x86_64 | hvm | ebs | 예 |

더 많은 AMI 찾아보기

AWS, Marketplace 및 커뮤니티의 AMI 포함

### ▼ 인스턴스 유형 정보 | 조언 받기

**인스턴스 유형**

t3a.small

패밀리: t3a 2 vCPU 2 GiB 메모리 현재 세대: true
온디맨드 Windows 기본 요금: 0.0418 USD 시간당
온디맨드 Ubuntu Pro 기본 요금: 0.0269 USD 시간당 온디맨드 Linux 기본 요금: 0.0234 USD 시간당
온디맨드 SUSE 기본 요금: 0.0544 USD 시간당 온디맨드 RHEL 기본 요금: 0.0522 USD 시간당

모든 세대

인스턴스 유형 비교

소프트웨어가 사전 설치된 AMI에는 추가 비용이 적용됩니다.

khb302500@gmail.com

## ▼ 키 페어(로그인) 정보

키 페어를 사용하여 인스턴스에 안전하게 연결할 수 있습니다. 인스턴스를 시작하기 전에 선택한 키 페어에 대한 액세스 권한이 있는지 확인하세요.

**키 페어 이름 - *필수***

키 페어 없이 계속 진행(권장되지 않음) 기본값 ▼ 새 키 페어 생성

## ▼ 네트워크 설정 정보

편집

**네트워크** | 정보

vpc-d0897abb

**서브넷** | 정보

기본 설정 없음(가용 영역의 기본 서브넷)

**퍼블릭 IP 자동 할당** | 정보

활성화

프리 티어 허용 범위를 벗어나는 경우 추가 요금이 적용됩니다.

**방화벽(보안 그룹)** | 정보

보안 그룹은 인스턴스에 대한 트래픽을 제어하는 방화벽 규칙 세트입니다. 특정 트래픽이 인스턴스에 도달하도록 허용하는 규칙을 추가합니다.

◉ 보안 그룹 생성 ○ 기존 보안 그룹 선택

다음 규칙을 사용하여 '**launch-wizard-21**'(이)라는 새 보안 그룹을 생성합니다.

☑ 다음에서 SSH 트래픽 허용 — 위치 무관 0.0.0.0/0
인스턴스 연결에 도움

☐ 인터넷에서 HTTPS 트래픽 허용
예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면

☑ 인터넷에서 HTTP 트래픽 허용
예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면

⚠ 소스가 0.0.0.0/0인 규칙은 모든 IP 주소에서 인스턴스에 액세스하도록 허용합니다. 알려진 IP 주소의 액세스만 허용하도록 보안 그룹을 설정하는 것이 좋습니다. ✕

## ▼ 스토리지 구성 정보

고급

1x 8 GiB gp3 ▼ 루트 볼륨 (암호화되지 않음)

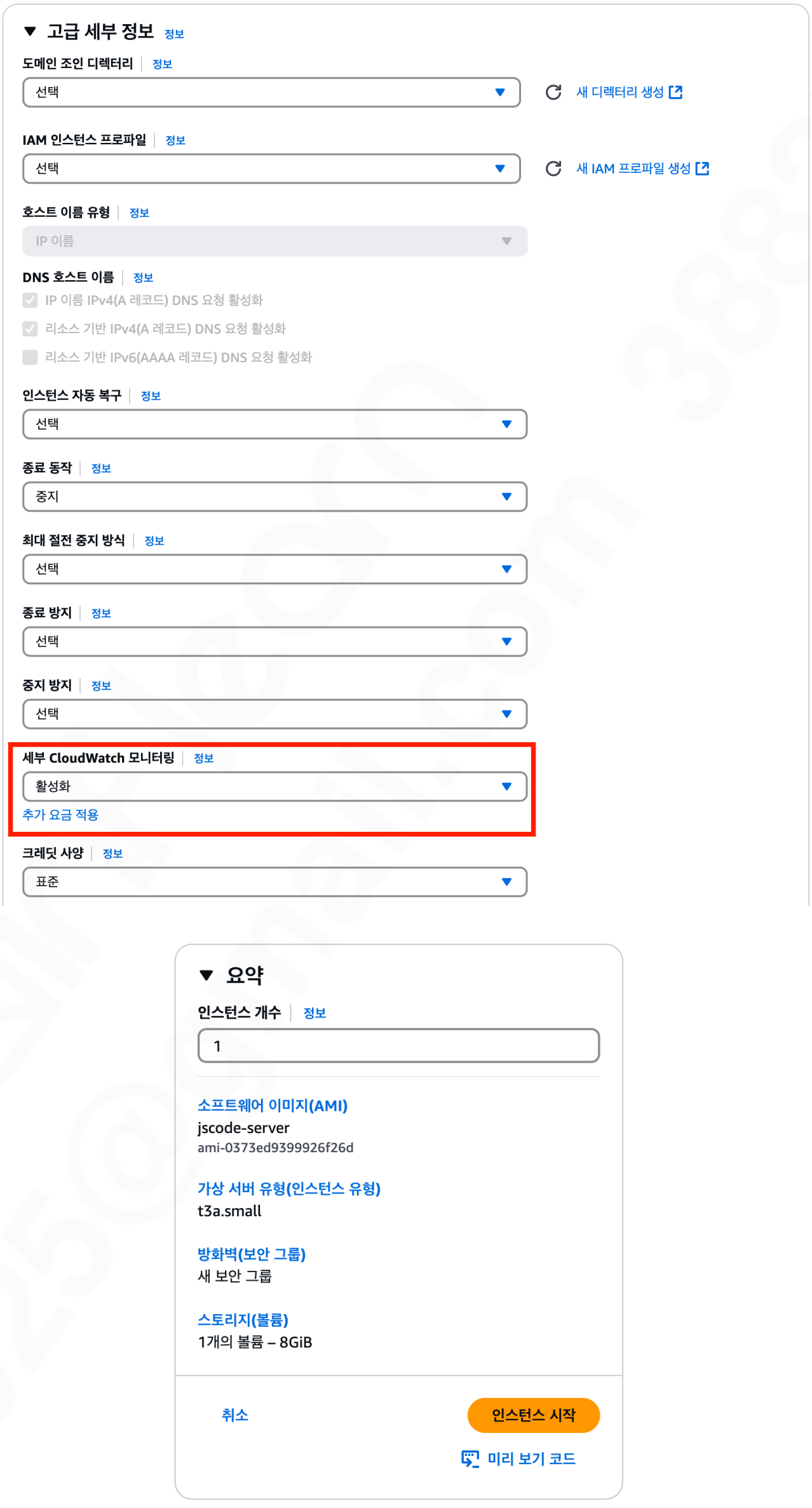
새 볼륨 추가

선택한 AMI에 인스턴스가 허용하는 것보다 많은 인스턴스 스토어 볼륨이 포함되어 있습니다. AMI에서 처음 0개의 인스턴스 스토어 볼륨에만 액세스할 수 있습니다.

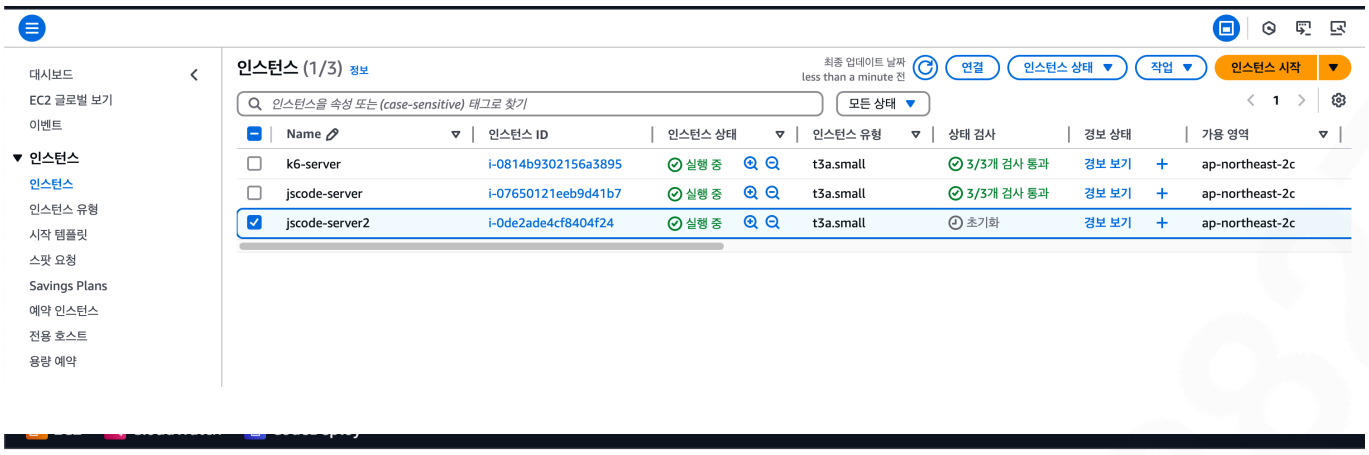
백업 정보를 보려면 새로 고침 클릭
할당한 태그에 따라 Data Lifecycle Manager 정책으로 인스턴스를 백업할지 여부가 결정됩니다.

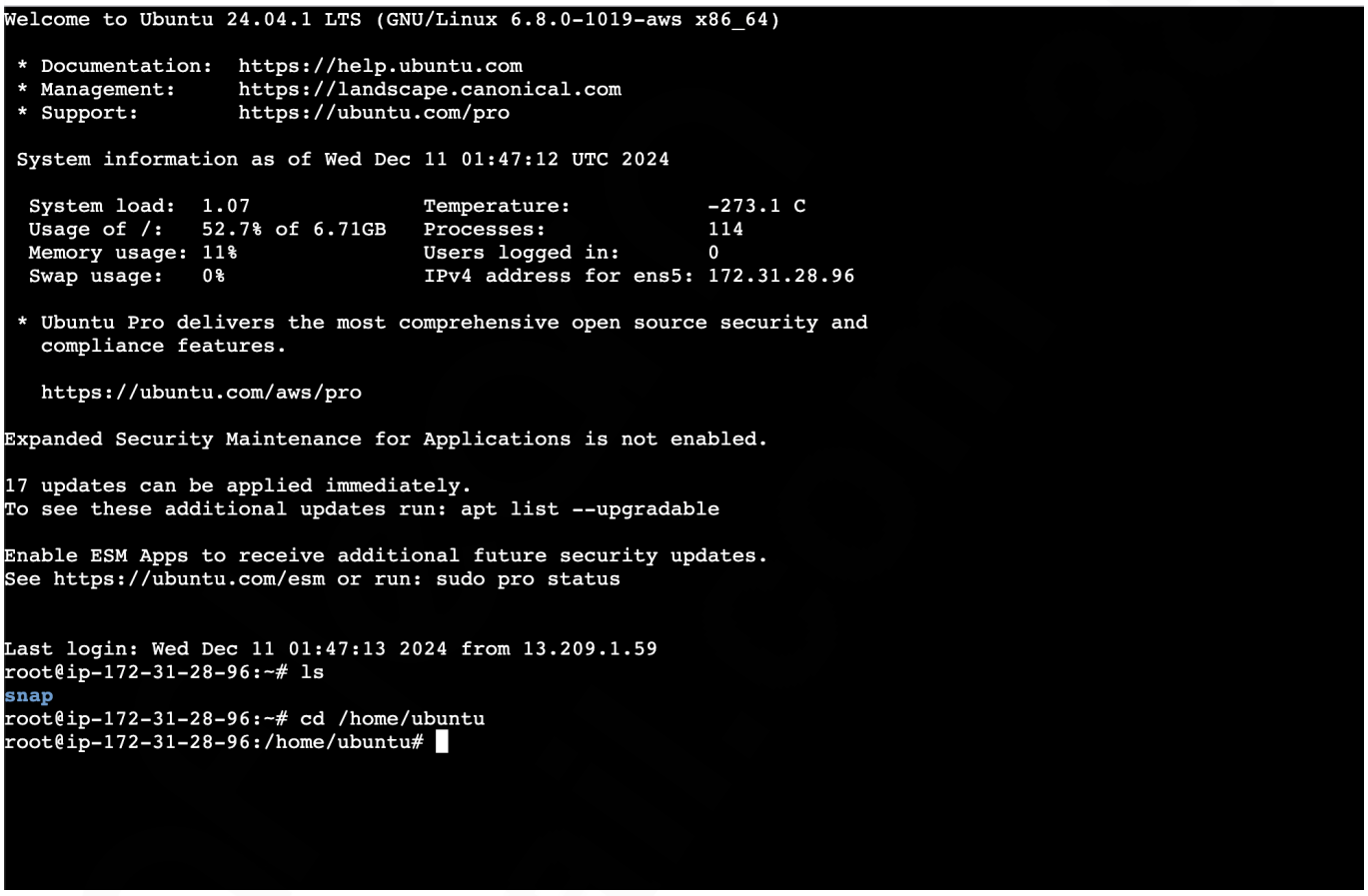
0 x 파일 시스템 편집

khb302507




3. **EC2 인스턴스로 접속해 Spring Boot 서버 실행시키기**

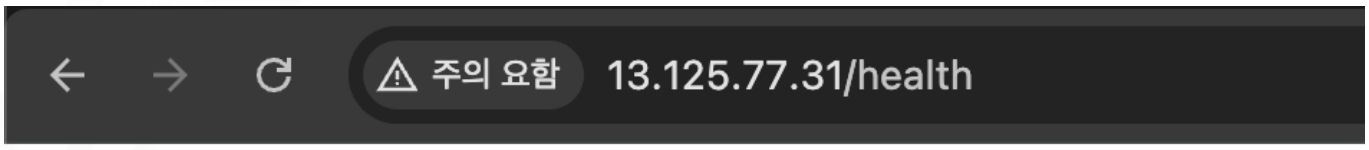



```
$ cd /home/ubuntu
$ cd load-testing-server
$ cd build/libs
$ sudo nohup java -jar jscode-0.0.1-SNAPSHOT.jar &
```

4. **Spring Boot가 잘 실행되는 지 확인하기**

`http://{EC2 IP 주소}/health`로 접속해서 Spring Boot 서버가 잘 작동하는 지 확인해보자.



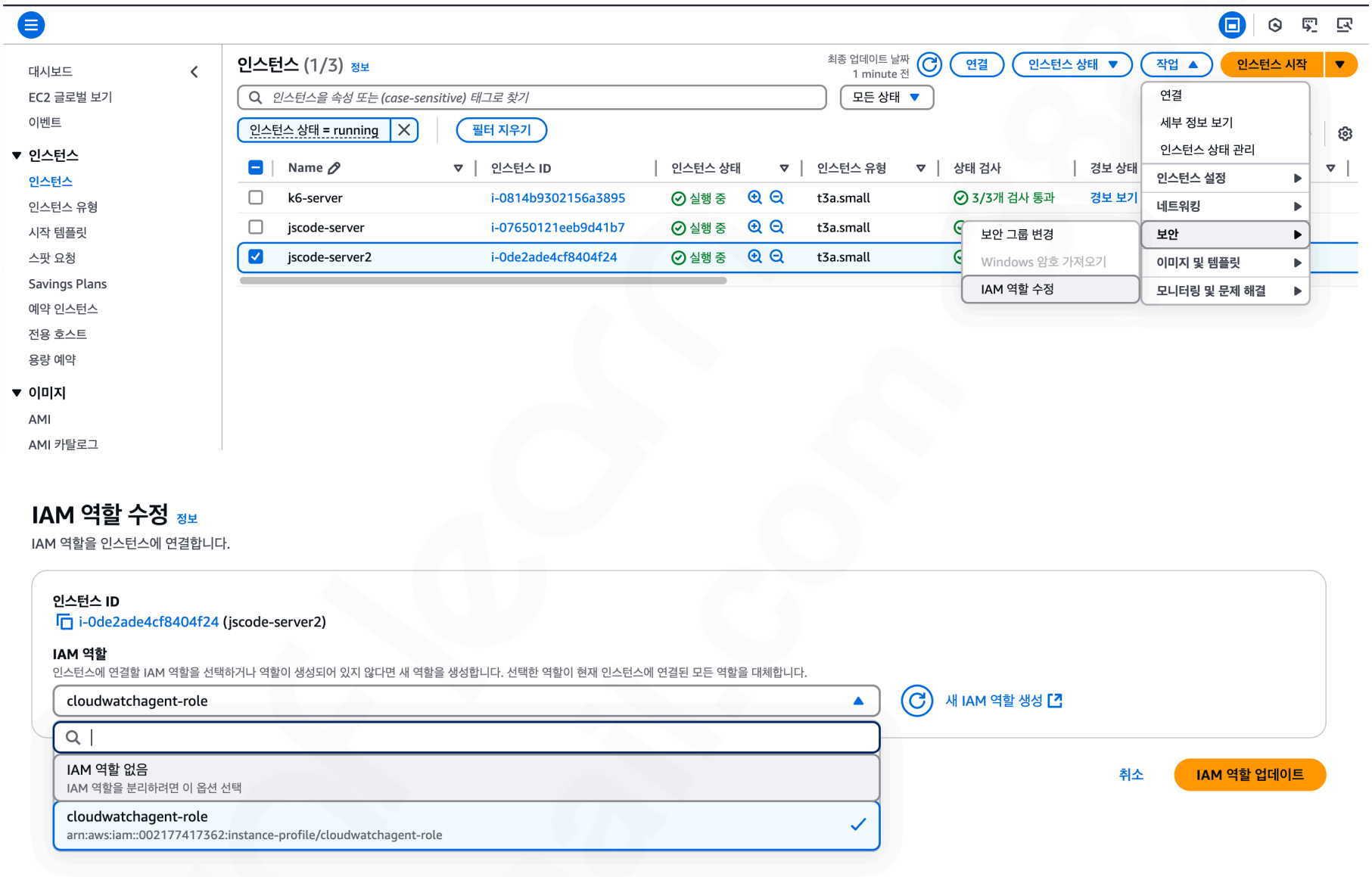

Success Health Check

5. **CloudWatch Agent 실행시키기**

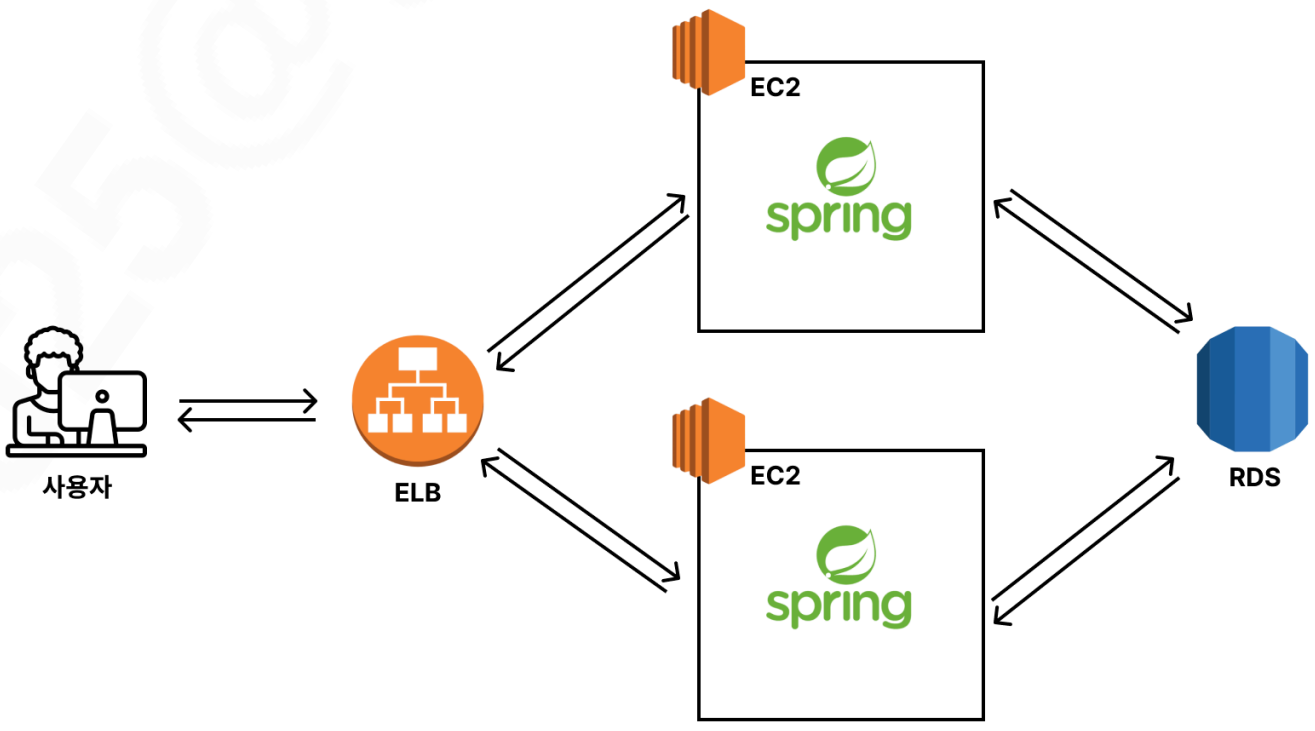
```
# CloudWatch Agent 실행
$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -s -c file:/opt/
```

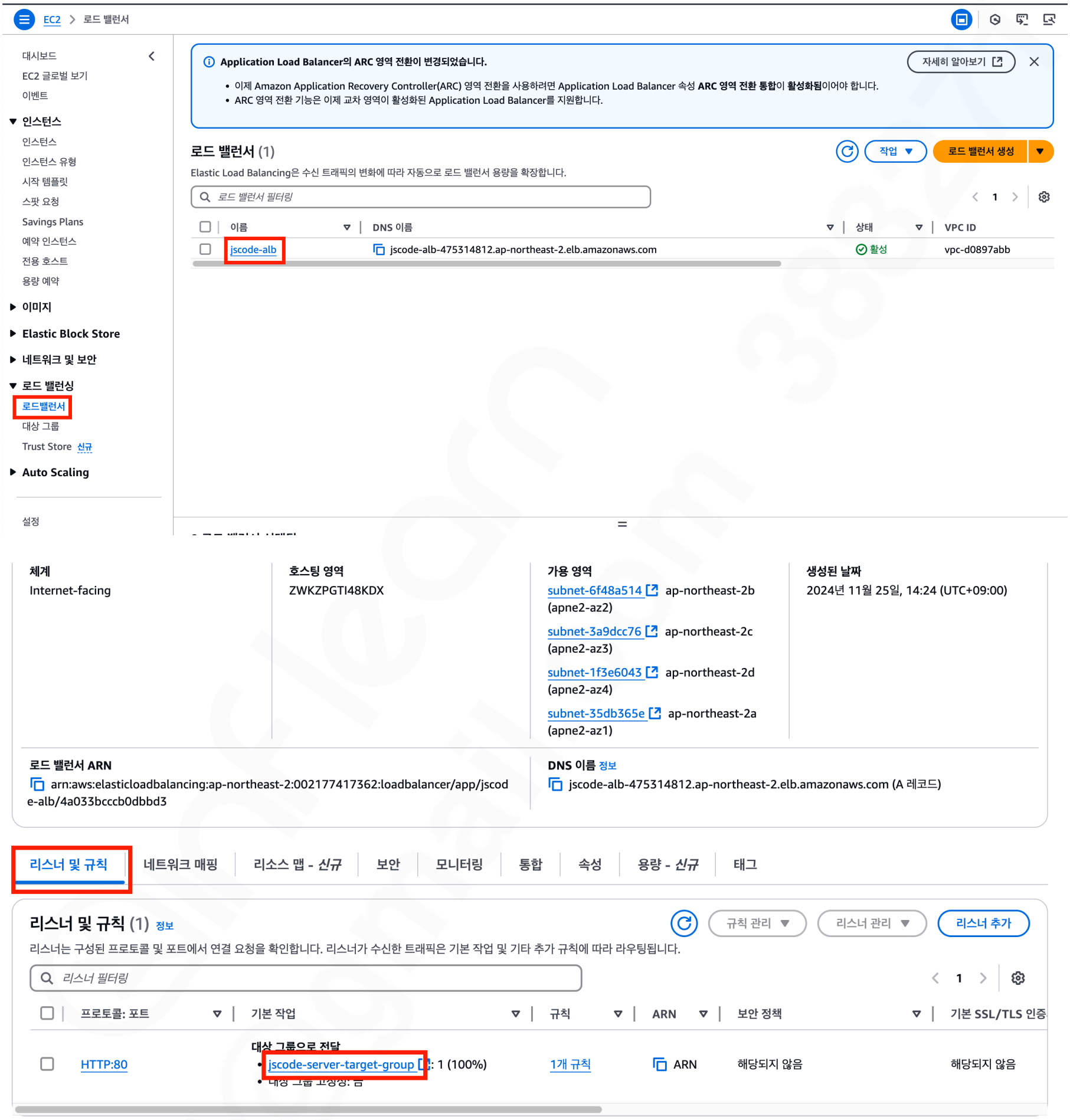
```
# 잘 실행되고 있는 지 확인
$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -m ec2 -a status
```

## 4. EC2 인스턴스에 IAM Role 연결




## 5. ELB의 대상 그룹에 추가하기




EC2 > 로드 밸런서
대시보드
EC2 글로벌 보기
이벤트
인스턴스
인스턴스
인스턴스 유형
시작 템플릿
스팟 요청
Savings Plans
예약 인스턴스
전용 호스트
용량 예약
이미지
Elastic Block Store
네트워크 및 보안
로드 밸런싱
로드밸런서
대상 그룹
Trust Store 신규
Auto Scaling
설정
Application Load Balancer의 ARC 영역 전환이 변경되었습니다.
자세히 알아보기
이제 Amazon Application Recovery Controller(ARC) 영역 전환을 사용하려면 Application Load Balancer 속성 ARC 영역 전환 통합이 활성화됨이어야 합니다.
ARC 영역 전환 기능은 이제 교차 영역이 활성화된 Application Load Balancer를 지원합니다.
로드 밸런서 (1)
작업
로드 밸런서 생성
Elastic Load Balancing은 수신 트래픽의 변화에 따라 자동으로 로드 밸런서 용량을 확장합니다.
로드 밸런서 필터링
이름
DNS 이름
상태
VPC ID
jscode-alb
jscode-alb-475314812.ap-northeast-2.elb.amazonaws.com
활성
vpc-d0897abb
체계
Internet-facing
호스팅 영역
ZWKZPGTI48KDX
가용 영역
subnet-6f48a514 ap-northeast-2b (apne2-az2)
subnet-3a9dcc76 ap-northeast-2c (apne2-az3)
subnet-1f3e6043 ap-northeast-2d (apne2-az4)
subnet-35db365e ap-northeast-2a (apne2-az1)
생성된 날짜
2024년 11월 25일, 14:24 (UTC+09:00)
로드 밸런서 ARN
arn:aws:elasticloadbalancing:ap-northeast-2:002177417362:loadbalancer/app/jscode-alb/4a033bcccb0dbbd3
DNS 이름 정보
jscode-alb-475314812.ap-northeast-2.elb.amazonaws.com (A 레코드)
리스너 및 규칙
네트워크 매핑
리소스 맵 - 신규
보안
모니터링
통합
속성
용량 - 신규
태그
리스너 및 규칙 (1) 정보
규칙 관리
리스너 관리
리스너 추가
리스너는 구성된 프로토콜 및 포트에서 연결 요청을 확인합니다. 리스너가 수신한 트래픽은 기본 작업 및 기타 추가 규칙에 따라 라우팅됩니다.
리스너 필터링
프로토콜: 포트
기본 작업
규칙
ARN
보안 정책
기본 SSL/TLS 인증
HTTP:80
대상 그룹으로 전달
jscode-server-target-group : 1 (100%)
대상 그룹 고정성: 끔
1개 규칙
ARN
해당되지 않음
해당되지 않음

khb302500

인스턴스

**IP 주소 유형**
IPv4

HTTP: 80

**로드 밸런서**
jscode-alb

HTTP1

vpc-d0897abb

| 1 | 1 | 0 | 0 | 0 | 0 |
|---|---|---|---|---|---|
| 대상 합계 | 정상 | 비정상 | 사용되지 않음 | 초기 | 드레이닝 |
| | 0 이상 | | | | |

▶ **가용 영역별 대상 배포**
아래의 등록된 대상 테이블에 적용된 해당 필터를 보려면 이 테이블에서 값을 선택합니다.

대상 | 모니터링 | 상태 검사 | 속성 | 태그

## 등록된 대상 (1) 정보

이상 완화: 해당되지 않음 | 등록 취소 | **대상 등록**

대상 그룹은 지정한 프로토콜 및 포트 번호를 사용하여 등록된 개별 대상으로 요청을 라우팅합니다. 상태 확인은 대상 그룹의 상태 확인 설정에 따라 등록된 모든 대상에 대해 수행됩니다. 이상 탐지는 정상 대상이 3개 이상 있는 HTTP/HTTPS 대상 그룹에 자동으로 적용됩니다.

대상 필터링

| 인스턴스 ID | 이름 | 포트 | 영역 | 상태 확인 | 상태 확인 세부 정보 | 관리상 ... | 재정의 |
|---|---|---|---|---|---|---|---|
| i-07650121eeb9d41b7 | jscode-server | 80 | ap-northeast-... | Healthy | - | No override. | No ov |

## 대상 등록

인스턴스를 선택하고 포트를 지정한 후 보류 중인 대상 목록에 인스턴스를 추가합니다. 이 작업을 반복하여 보류 중인 대상 목록에 인스턴스와 포트의 조합을 더 추가합니다. 선택 내용에 만족하면 [보류 중인 대상 등록]을 클릭합니다.

### 사용 가능한 인스턴스 (1/3)

인스턴스 필터링

| 인스턴스 ID | 이름 | 상태 | 보안 그룹 | 영역 | 프라이빗 IPv4 주소 |
|---|---|---|---|---|---|
| i-0de2ade4cf8404f24 | jscode-server2 | 실행 중 | launch-wizard-21 | ap-northeast-2c | 172.31.28.96 |
| i-07650121eeb9d41b7 | jscode-server | 실행 중 | launch-wizard-19 | ap-northeast-2c | 172.31.16.55 |
| i-0814b9302156a3895 | k6-server | 실행 중 | launch-wizard-17 | ap-northeast-2c | 172.31.20.91 |

**1개 선택됨**

**선택한 인스턴스를 위한 포트**
선택한 인스턴스로 트래픽을 라우팅하기 위한 포트입니다.

80

1-65535(쉼표로 여러 포트 구분)

아래에 보류 중인 것으로 포함

1개의 선택 항목이 현재 아래에 보류 중입니다. 준비가 되면 대상을 더 포함하거나 등록하십시오.

### 대상 보기

**대상 (2)**

보류 중인 모든 항목 제거

대상 필터링 | 대기 중인 항목만 보기

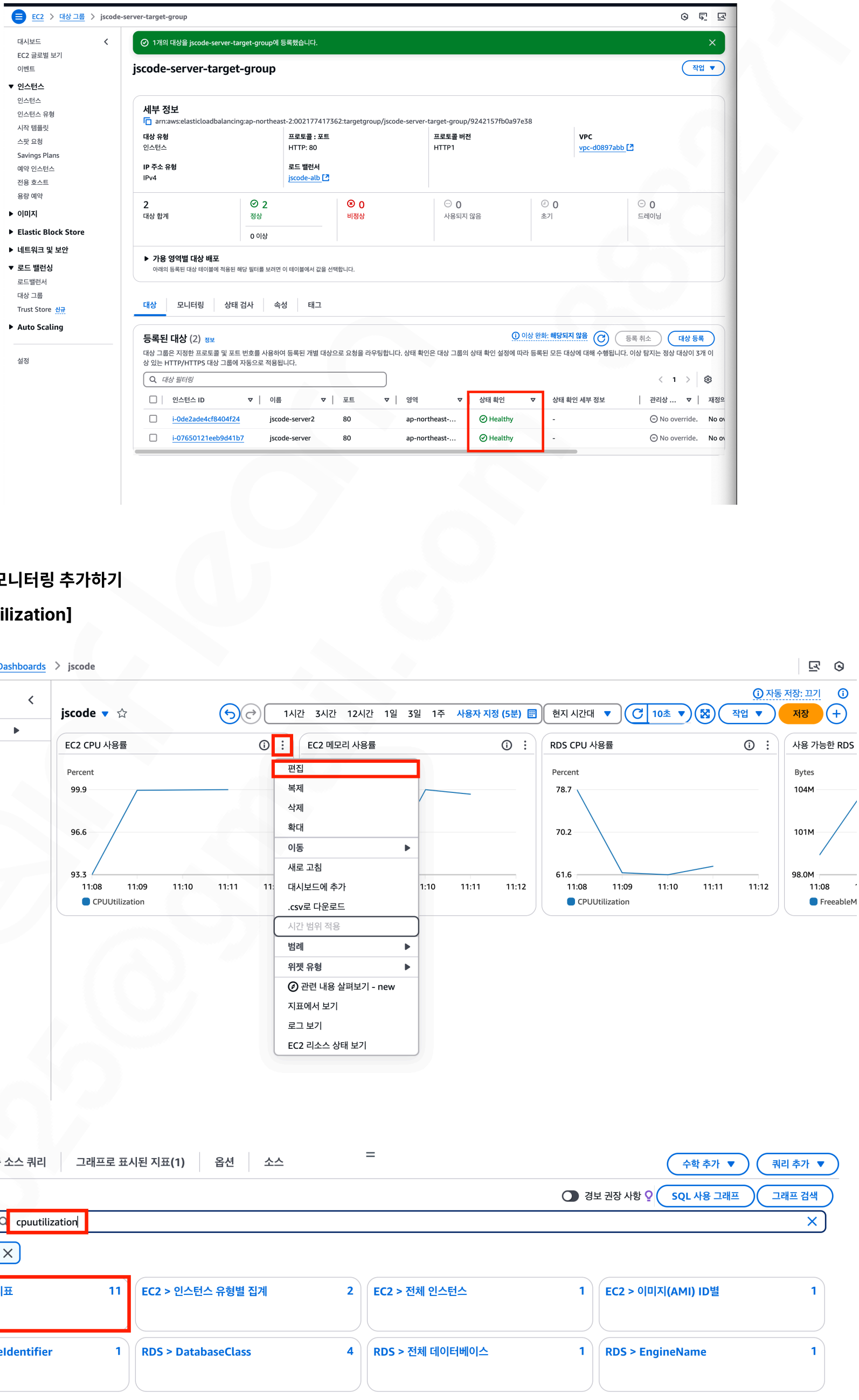
| 인스턴스 ID | 이름 | 포트 | 상태 | 보안 그룹 | 영역 | 프라이빗 IPv4 주소 | 서브넷 ID | 시작 시간 |
|---|---|---|---|---|---|---|---|---|
| i-0de2ade4cf8404f24 | jscode-server2 | 80 | 실행 중 | launch-wizard-21 | ap-northeast-2c | 172.31.28.96 | subnet-3a9dcc76 | 2024년 12월 11일, 10:45 (UTC+09:00) |
| i-07650121eeb9d41b7 | jscode-server | 80 | 실행 중 | launch-wizard-19 | ap-northeast-2c | 172.31.16.55 | subnet-3a9dcc76 | 2024년 12월 10일, 17:21 (UTC+09:00) |

**1개 대기 중**

취소 | **보류 중인 대상 등록**

6. **CloudWatch 모니터링 추가하기**

**[EC2의 CPUUtilization]**



찾아보기 | 다중 소스 쿼리 | 그래프로 표시된 지표(2) | 옵션 | 소스

수학 추가 | 쿼리 추가

지표 (11)

경보 권장 사항 | SQL 사용 그래프 | 그래프 검색

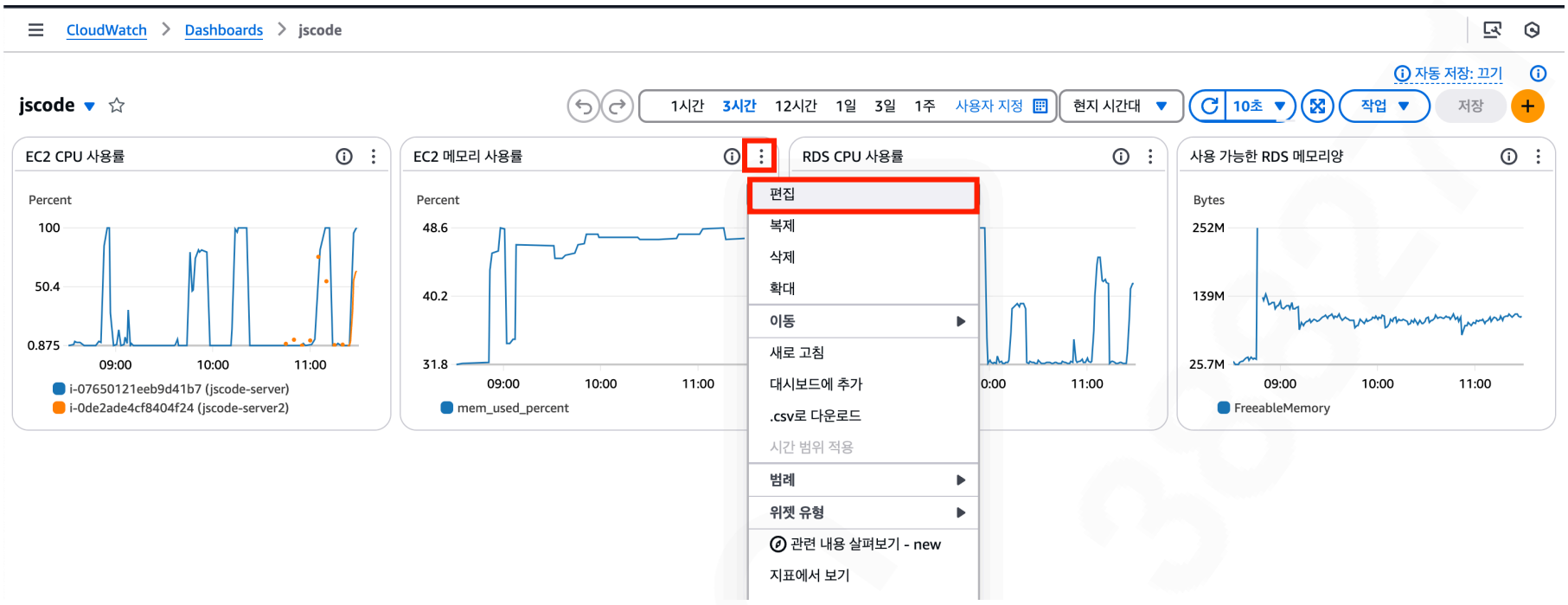
Seoul | 모두 > ... > 인스턴스별 지표 | cpuutilization | 1

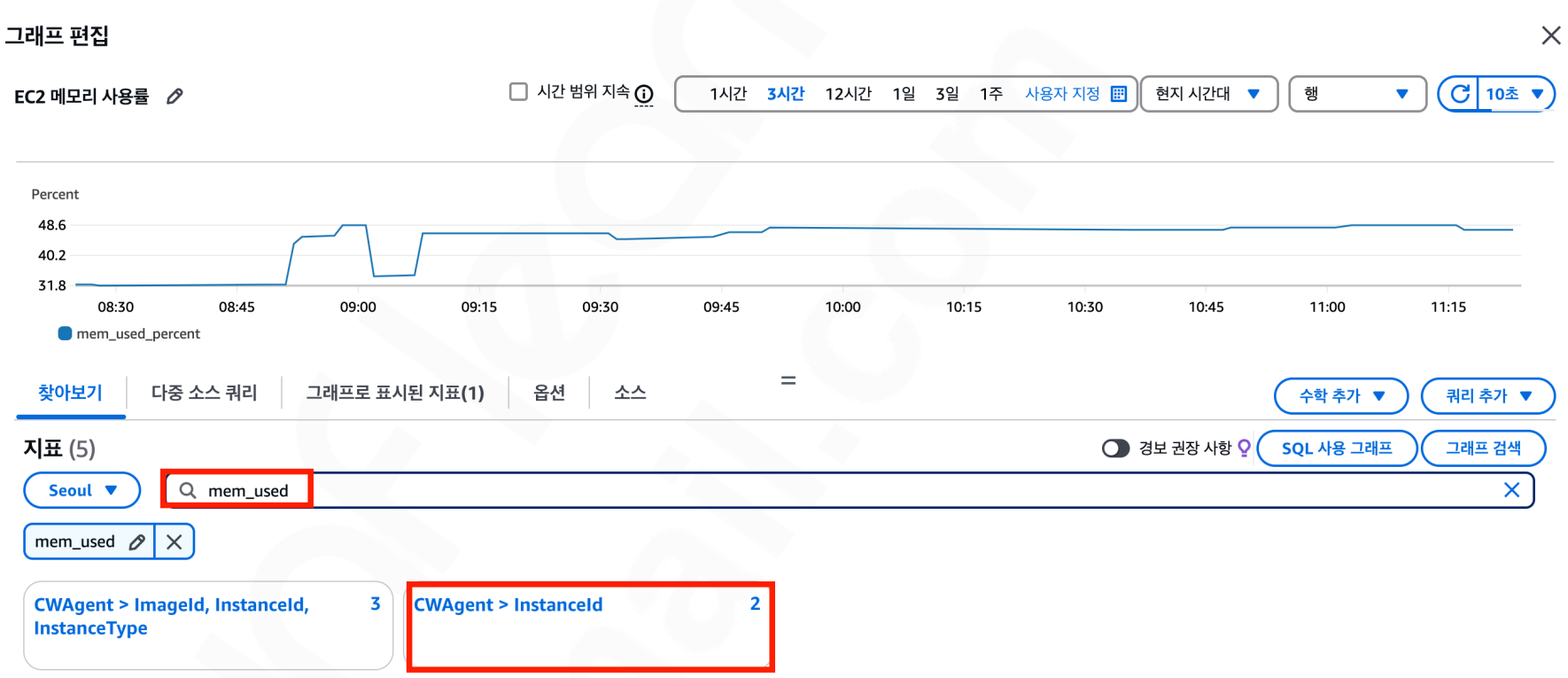
cpuutilization

| 인스턴스 이름 11/11 | InstanceId | 지표 이름 | 경보 |
|---|---|---|---|
| 지정된 이름이 없음 | i-08ccc4969a1fee... | CPUUtilization | 경보 없음 |
| 지정된 이름이 없음 | i-0fb1d938a9061... | CPUUtilization | 경보 없음 |
| 지정된 이름이 없음 | i-0e4cd9ecbec5c0... | CPUUtilization | 경보 없음 |
| 지정된 이름이 없음 | i-0f2a71ce011c6a... | CPUUtilization | 경보 없음 |
| 지정된 이름이 없음 | i-01aede3852feff... | CPUUtilization | 경보 없음 |
| 지정된 이름이 없음 | i-0d701f540c170... | CPUUtilization | 경보 없음 |
| 지정된 이름이 없음 | i-035dff0ffa725f5... | CPUUtilization | 경보 없음 |
| 지정된 이름이 없음 | i-0307778bfd301... | CPUUtilization | 경보 없음 |
| jscode-server | i-07650121eeb9d... | CPUUtilization | 경보 없음 |
| jscode-server2 | i-0de2ade4cf840... | CPUUtilization | 경보 없음 |
| k6-server | i-0814b9302156a... | CPUUtilization | 경보 없음 |

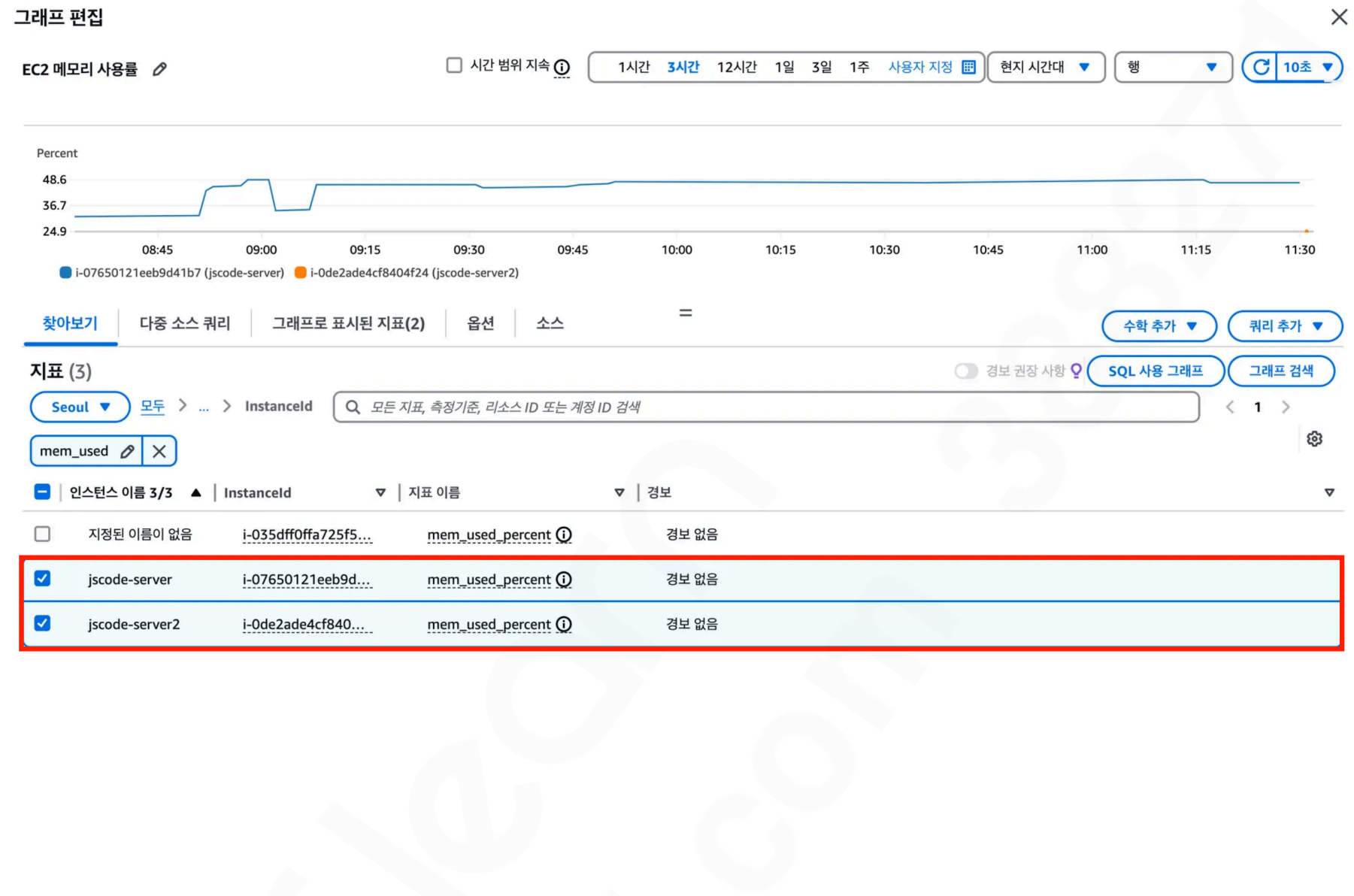
취소 | 위젯 업데이트

**[EC2의 메모리 사용량]**




khb3025@gmail.com



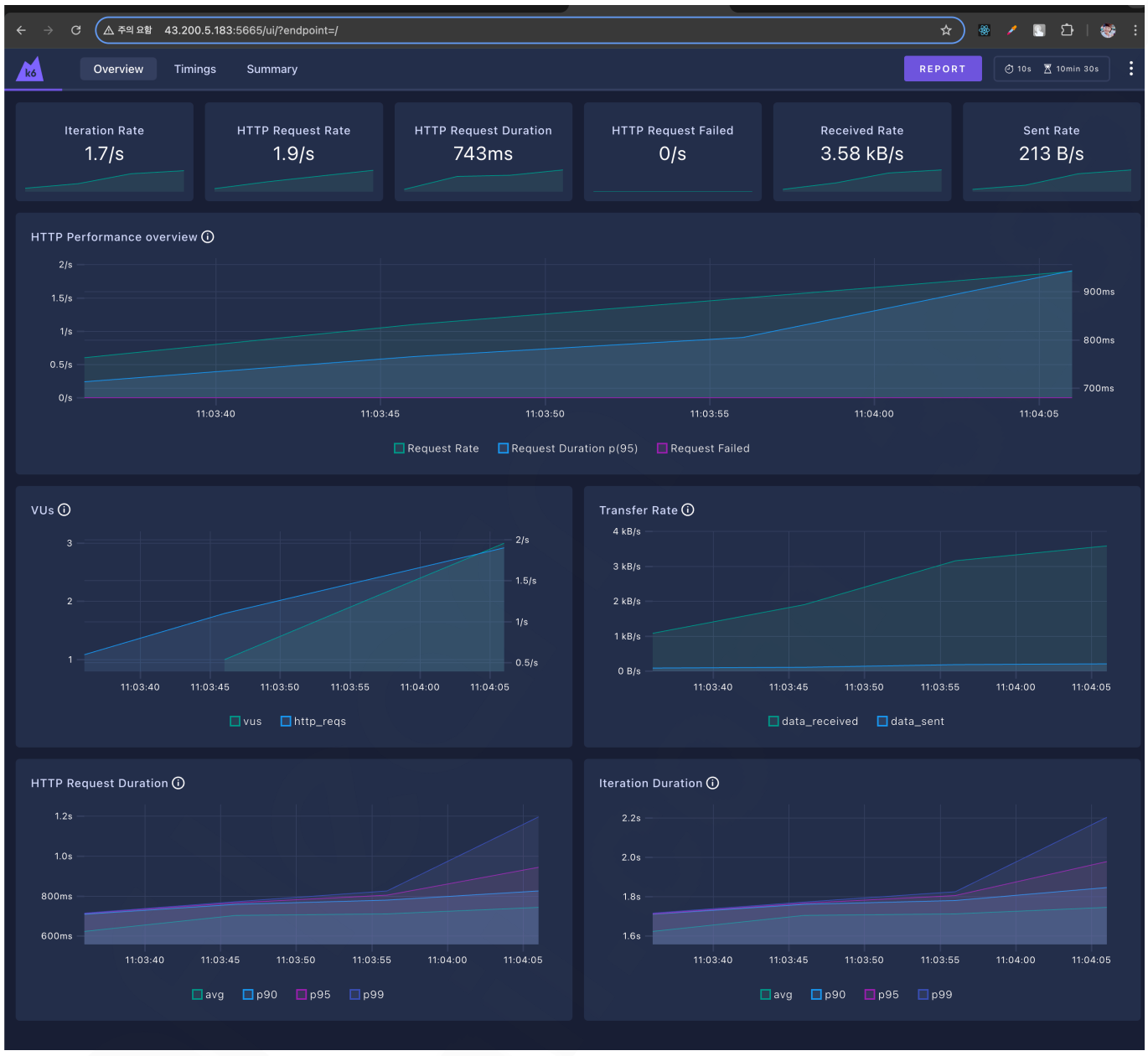
## ✅ 부하 테스트 다시 진행하기

1. **부하 테스트 시작하기**

```
$ K6_WEB_DASHBOARD=true k6 run script.js
```

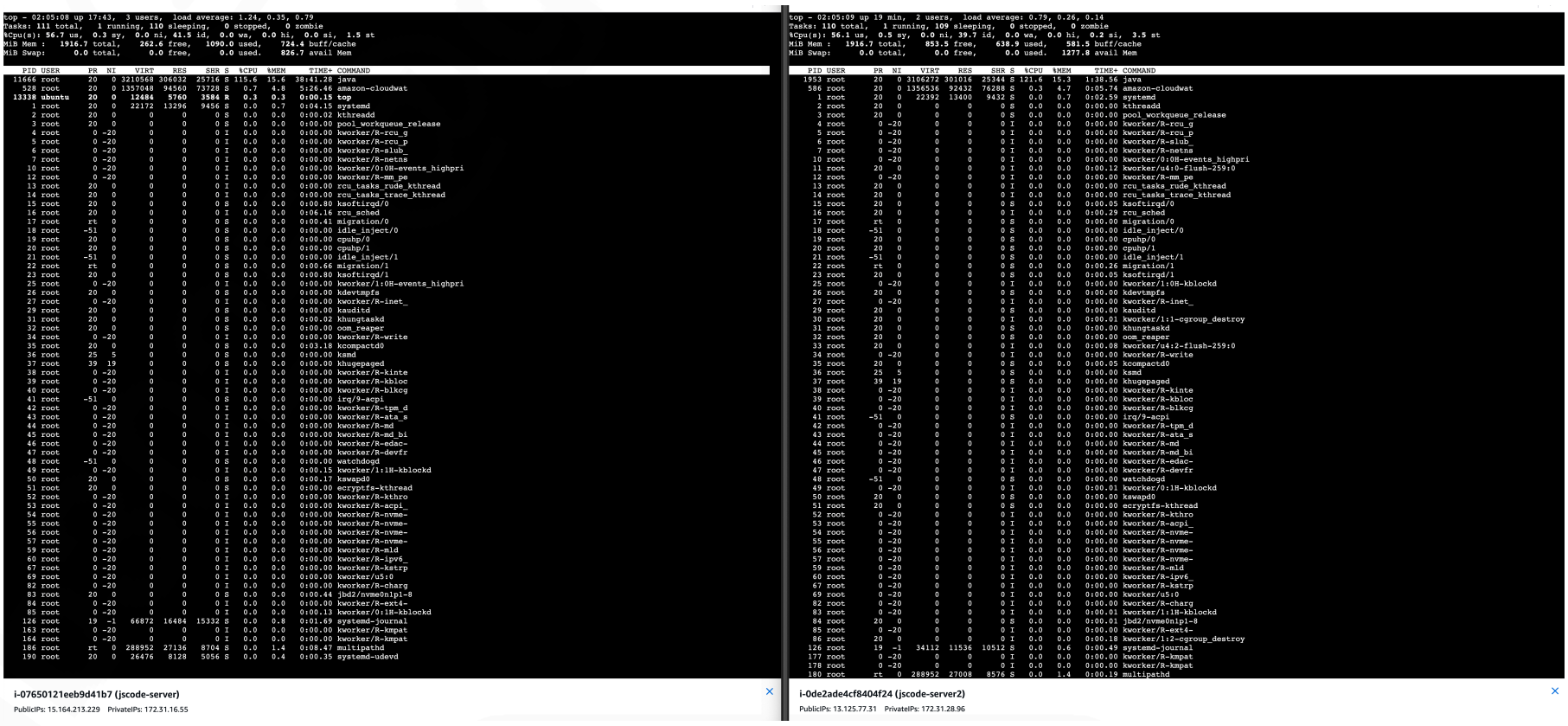
2. **k6의 웹 대시보드 열기**

http://{k6가 실행되고 있는 EC2 IP 주소}:5665으로 접속하기



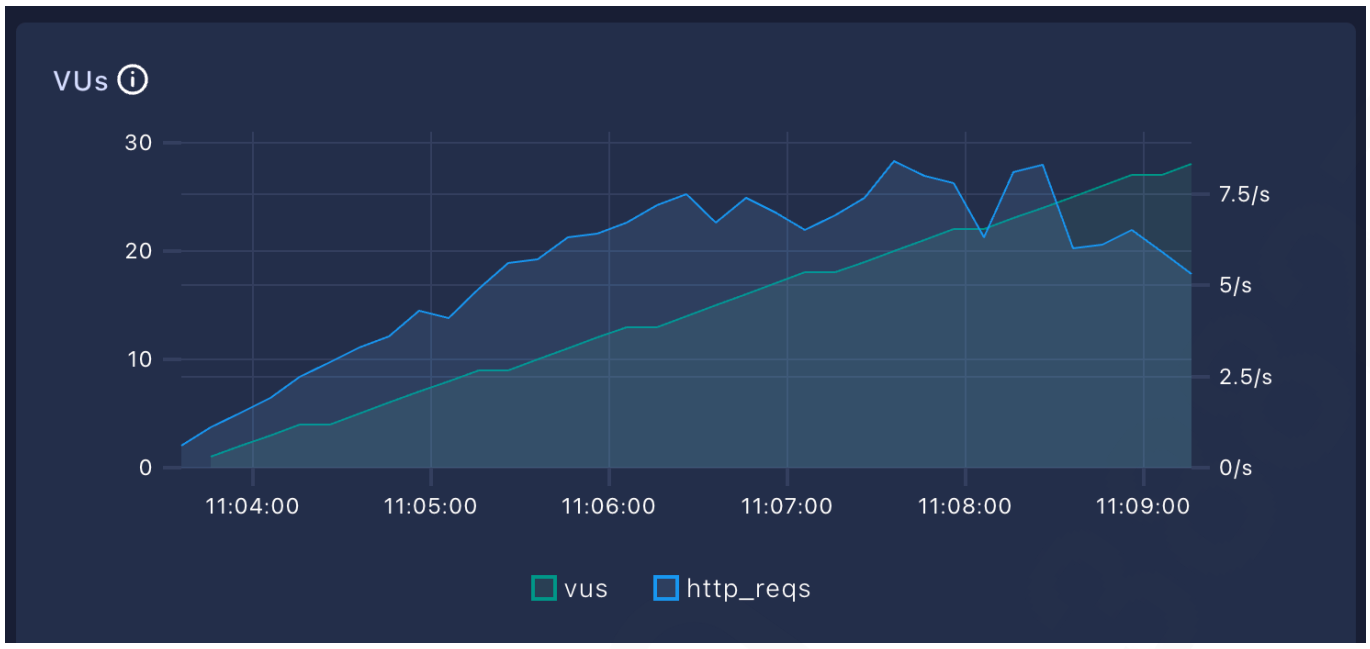

### 3. top 명령어 활용해 실시간 모니터링하기

```
$ top
```



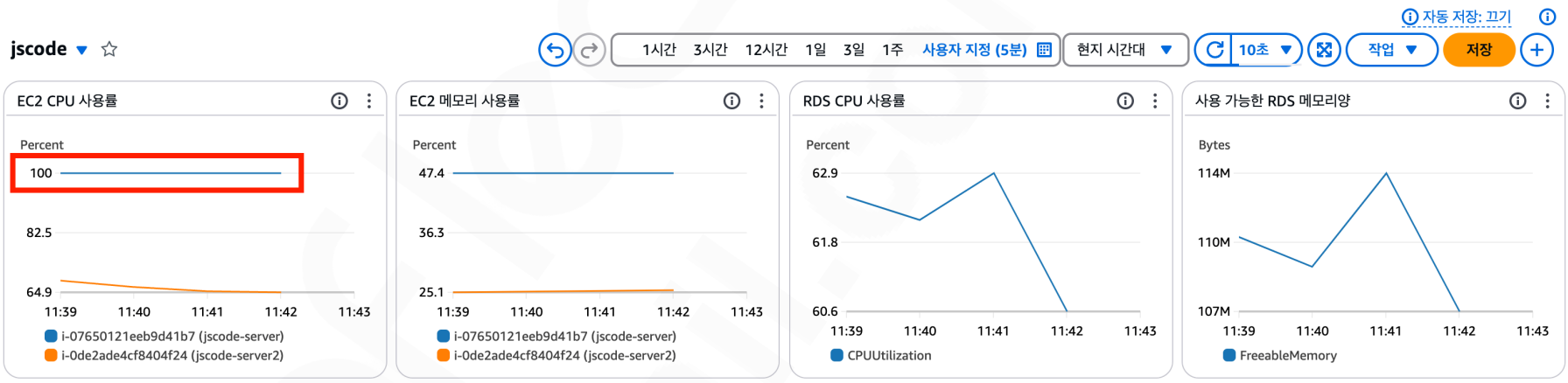

### 4. 부하 테스트 결과 해석

최대 Throughput은 약 **7 TPS** 정도로 측정된다. 기존 Throughput(4 TPS)에 비해 1.75배 정도 증가했다. 즉, 1초에 처리할 수 있는 요청량이 4개에서 7개로 증가했다.




5. **병목 지점 확인하기**

   추가적인 성능 개선을 하고 싶다면 아래 병목 지점을 해결한 뒤에 또 부하 테스트를 하는 과정을 반복하면 된다.




khb3025@gmail.com

# 부하 테스트의 전체 흐름 다시 점검

## ✅ 1. 부하 테스트 필요성 인식

“서비스를 오픈하려고 하는데 사용자들이 몰려서 서버가 터지면 어떡하지?”

“내 서버는 어느 정도 사용자 요청을 견딜 수 있는 거지?”

“개발자님, 이번에 치킨 이벤트를 하려하는데 사용자가 1만명 정도가 들어와도 서버가 괜찮을까요?”

## ✅ 2. 부하 테스트 목표 설정하기

부하 테스트 목표는 주로 **Throughput**과 **Latency**를 활용해 설정한다.

**[예시]**

- **목표 TPS : 2000TPS**

  → 서비스에 예상 접속자 수, 예상 요청 수 등을 활용해 TPS 목표를 설정한다.

- **평균 Latency : 800ms**

  → 서비스의 특성에 맞게 Latency를 설정한다.

## ✅ 3. 현재 시스템이 어느 정도 트래픽까지 견딜 수 있는 지 부하 테스트 진행

부하 테스트 툴(k6, ngrinder, jmeter 등)을 활용해 시스템에 부하를 준다. 부하의 정도를 올려가면서 어느 정도 부하까지 견딜 수 있는 지를 측정한다. 즉, 시스템의 최대 Throughput을 측정한다.

## ✅ 4. 병목 지점 파악 후 성능 개선하기

목표로 설정한 Throughput, Latency를 달성하기 위해, 기존 시스템의 성능을 개선해야 할 수도 있다. 성능 개선을 하려면 가장 먼저 병목 지점을 파악해서 개선해야 한다.

## ✅ 5. 개선한 시스템이 어느 정도 트래픽까지 견딜 수 있는 지 부하 테스트 진행

개선한 시스템이 어느 정도 트래픽까지 견딜 수 있는 지 부하 테스트를 진행한다. 부하 테스트를 통해 최대 Throughput과 평균 Latency를 측정해서 목표치를 달성했는 지 체크한다.

만약 목표치를 달성하지 못했다면 다시 4번 과정(병목 지점 파악 후 성능 개선)을 거친 후 부하 테스트를 진행해서 목표치 달성 여부를 확인한다. 이 과정을 목표 달성 때까지 반복한다.